



Department of Informatics
Technical University of Munich



TECHNICAL UNIVERSITY OF MUNICH

DEPARTMENT OF INFORMATICS

BACHELOR'S THESIS IN INFORMATICS

Benchmarking Secure Multiparty Computation Protocols

Jacob Obrman

TECHNICAL UNIVERSITY OF MUNICH
DEPARTMENT OF INFORMATICS

Bachelor's Thesis in Informatics

**Benchmarking Secure Multiparty Computation
Protocols**

**Leistungsvergleich von Secure Multiparty
Computation Protokollen**

Author:	Jacob Obrman
Supervisor:	Prof. Dr.-Ing. Georg Carle
Advisor:	Christopher Harth-Kitzerow, Manuel Simon, Eric Hauser
Date:	August 15, 2022

I confirm that this Bachelor's Thesis is my own work and I have documented all sources and material used.

Garching, August 15, 2022

Location, Date

Signature

ABSTRACT

Secure Multiparty Computation (SMC) enables a group of secret holders to evaluate a function over their secrets while concealing all inputs and revealing nothing except the function's result. Since SMC's inception in the early 80's, over 40 implementations have been created, optimized for most diverse situations. Finding a suitable SMC protocol for a specific problem is a process which includes many steps. One of them is narrowing down the various existing generic SMC protocol candidates to fit a specific purpose. By filtering impractical candidates, protocol designers can focus their work on the most useful variants.

This practical thesis results from the project of implementing *sevarebench*, a benchmarking tool that assists software developers on their path of selecting a SMC protocol that is optimized for their specific hardware and use case. The developed benchmarking framework is utilized to answer how currently available SMC protocol implementations are performing for different reference use cases, how their performance fares compared to other types of SMC protocols, and to what degree host environment parameters are impacting their performance. After introducing the modules of *sevarebench* and the SMC protocol collection *MP-SPDZ*, this Bachelor's Thesis presents a variety of measurement results obtained by running numerous experiments for reference problems including set operations and arithmetic matrix operations on integers and fixed-point numbers. On the basis of the generated data and its visualization, an analysis gives reasoning in classifying the strengths and weaknesses of concrete protocols and the category they belong to.

ZUSAMMENFASSUNG

Secure Multiparty Computation (SMC), *sichere Mehrparteienberechnung* aus dem Englischen, ermöglicht einer Gruppe von Geheimnisträgern eine Funktion über deren Geheimnissen auszuwerten während deren Eingabewerte verborgen werden und nichts außer dem Berechnungsergebnis bekannt gegeben wird. Seit dem Ursprung von SMC in den frühen 80er Jahre, wurden über 40 Implementierungen geschaffen, optimiert für unterschiedlichste Situationen. Ein geeignetes SMC-Protokoll für ein spezifisches Problem zu finden ist ein Prozess der viele Schritte erfordert. Einer dieser Schritte besteht darin, die verschiedenen bestehenden generischen SMC-Protokoll-Kandidaten für einen bestimmten Einsatzzweck einzugrenzen. Durch das Ausschließen ungeeigneter Kandidaten können Entwickler von Protokollen ihre Arbeit auf die nützlichsten Varianten fokussieren.

Diese praktische Bachelorarbeit resultiert aus dem Projekt der Implementierungen von *sevarebench*, ein Benchmarking-Tool das Software Entwicklern auf der Suche nach einem SMC-Protokoll, das angepasst an deren konkrete Hardware und für ihre Einsatzzwecke ist, unterstützt. Das entwickelte Benchmarking Rahmenprogramm wird eingesetzt um folgende Fragen zu beantworten: Welche Leistung erbringen aktuell verfügbare Implementierungen von SMC-Protokollen für verschiedene Referenz Einsatzzwecke, wie schneidet deren Leistung im Vergleich zu anderen Typen von SMC-Protokollen ab, und bis zu welchem Grad wirken sich Host-Umgebungsparameter auf die Leistung aus. Nach einer Einführung in die Module von *sevarebench* und der SMC-Protokoll Sammlung *MP-SPDZ* präsentiert diese Bachelor Arbeit eine Vielfalt an Messergebnissen aus zahlreichen durchgeführten Experimenten für Referenz Probleme, einschließlich Mengenoperationen und arithmetische Matrix Operationen auf ganzen Zahlen und Festkommazahlen. Auf Grundlage der generierten Daten und ihrer Visualisierung gibt eine Analyse eine Klassifizierung der Stärken und Schwächen konkreter Protokolle und der zugehörigen Kategorie.

CONTENTS

1	Introduction	1
1.1	Definition	1
1.2	Research Questions	3
1.3	Thesis Outline	3
2	Background	5
2.1	SMC History	5
2.2	SMC Categories	7
2.2.1	Security Types of the Adversary Models	7
2.2.2	Computation Domains	8
2.3	Experiment Testing Environment	9
3	Related Work	11
3.1	Assessment of Secret Sharing Based SMC	11
3.2	Multi-Protocol SPDZ Collection	13
4	sevarebench: Benchmarking Framework for SMC Protocols	15
4.1	Functionality and Modules Overview	15
4.2	Measurement Module	17
4.2.1	Unconcealed Verification Computation	17
4.2.2	SMC in a Manipulated Host Environment	18
4.3	Testresults Module	18
4.4	Implemented Solutions for Reference SMC Problems	19
5	Performance Assessment of SMC protocols	23
5.1	Electing Promising Protocols for Reference SMC Problems	23
5.1.1	Initial Exclusion of Underperforming Candidates	24
5.1.2	Comparing Protocols of the Same Adversary Model	26
5.2	Metrics of CPU parameters for Reference SMC Problems	31

5.2.1	Frequency	32
5.2.2	Quota	34
5.2.3	Cores	36
5.3	Analyzing the Measurement Results	38
5.3.1	Implications of Observed Behavior	39
5.3.2	Comparing Protocols of a Computational Domain	41
6	Conclusion	43
6.1	Contributions	44
6.2	Future Work	44
A	Appendix	47
A.1	Measurements for Chapter 5.1	47
A.1.1	Measurement Tables for Section 5.1.1	47
A.1.2	Measurement Plots for Section A.1.2	55
A.1.3	Measurement Plots for Section 5.2	68
A.2	TTP Experiment Measurement Results	80
A.3	List of Acronyms	81
	Bibliography	83

LIST OF FIGURES

1.1	Secret sharing method [3]	2
2.1	SMC Timeline	6
2.2	Available testbed topologies	10
4.1	Benchmarking framework posSMCbench	16
5.1	Dishonest majority protocols with their runtimes for scalable search on valga	27
5.2	Honest majority protocols with their runtimes for scalable search on valga	27
5.3	Dishonest majority protocols with their peak RAM usage for scalable search on valga	28
5.4	Honest majority protocols with their peak RAM usage for scalable search on valga	28
5.5	Dishonest majority protocols with their runtimes for scalable search based on frequencies with fixed input size 500 on valga	29
5.6	Honest majority protocols with their runtimes for scalable search based on frequencies with fixed input size 2400 on valga	29
5.7	Dishonest majority protocol finalists reaction to CPU frequency change (Scalable Search)	33
5.8	Honest majority protocol finalists reaction to CPU frequency change (Scalable Search)	33
5.9	Garbled circuit protocol yaos reaction to CPU frequency change (Scalable Search)	34
5.10	Dishonest majority protocol finalists reaction to CPU quota limit change (Scalable Search)	35
5.11	Honest majority protocol finalists reaction to CPU quota limit change (Scalable Search)	35

5.12	Garbled circuit protocol yaos reaction to CPU quota limit change (Scalable Search)	36
5.13	Dishonest majority protocol finalists reaction to available CPU thread number change (Scalable Search)	37
5.14	Honest majority protocol finalists reaction to available CPU thread number change (Scalable Search)	37
5.15	Garbled circuit protocol yaos reaction to available CPU thread number change (Scalable Search)	38
A.1	Dishonest majority protocols with their runtimes for matrix integer addition on todd	56
A.2	Honest majority protocols with their runtimes for matrix integer addition on todd	56
A.3	Dishonest majority protocols with their peak RAM usage for matrix integer addition on todd	57
A.4	Honest majority protocols with their peak RAM usage for matrix integer addition on todd	57
A.5	Dishonest majority protocols with their runtimes for matrix integer addition based on frequencies with fixed input size 33^2 on todd	58
A.6	Honest majority protocols with their runtimes for matrix integer addition based on frequencies with fixed input size 65^2 on todd	58
A.7	Dishonest majority protocols with their runtimes for matrix fixed-point addition on dogecoin	59
A.8	Honest majority protocols with their runtimes for matrix fixed-point addition on dogecoin	59
A.9	Dishonest majority protocols with their peak RAM usage for matrix fixed-point addition on dogecoin	60
A.10	Honest majority protocols with their peak RAM usage for matrix fixed-point addition on dogecoin	60
A.11	Dishonest majority protocols with their runtimes for matrix fixed-point addition based on frequencies with fixed input size 33^2 on dogecoin	61
A.12	Honest majority protocols with their runtimes for matrix fixed-point addition based on frequencies with fixed input size 65^2 on dogecoin	61
A.13	Dishonest majority protocols with their runtimes for matrix integer multiplication on bitcoin	62
A.14	Honest majority protocols with their runtimes for matrix integer multiplication on bitcoin	62

A.15 Dishonest majority protocols with their peak RAM usage for matrix integer multiplication on bitcoin	63
A.16 Honest majority protocols with their peak RAM usage for matrix integer multiplication on bitcoin	63
A.17 Dishonest majority protocols with their runtimes for matrix integer multiplication based on frequencies with fixed input size 12^2 on bitcoin	64
A.18 Honest majority protocols with their runtimes for matrix integer multiplication based on frequencies with fixed input size 31^2 on bitcoin	64
A.19 Dishonest majority protocols with their runtimes for set intersection on dogecoin	65
A.20 Honest majority protocols with their runtimes for set intersection on bitcoin	65
A.21 Dishonest majority protocols with their peak RAM usage for set intersection on dogecoin	66
A.22 Honest majority protocols with their on bitcoin	66
A.23 Dishonest majority protocols with their runtimes for set intersection based on frequencies with fixed input size 13 on dogecoin	67
A.24 Honest majority protocols with their runtimes for set intersection based on frequencies with fixed input size 170 on bitcoin	67
A.25 Dishonest majority protocol finalists reaction to CPU frequency change (Integer Addition)	68
A.26 Honest majority protocol finalists reaction to CPU frequency change (Integer Addition)	68
A.27 Dishonest majority protocol finalists reaction to CPU frequency change (Fixed-point Addition)	69
A.28 Honest majority protocol finalists reaction to CPU frequency change (Fixed-point Addition)	69
A.29 Dishonest majority protocol finalists reaction to CPU frequency change (Integer Multiplication)	70
A.30 Honest majority protocol finalists reaction to CPU frequency change (Integer Multiplication)	70
A.31 Dishonest majority protocol finalists reaction to CPU frequency change (Set Intersection)	71
A.32 Honest majority protocol finalists reaction to CPU frequency change (Set Intersection)	71
A.33 Dishonest majority protocol finalists reaction to CPU quota limit change (Integer Addition)	72

A.34 Honest majority protocol finalists reaction to CPU quota limit change (Integer Addition)	72
A.35 Dishonest majority protocol finalists reaction to CPU quota limit change (Fixed-point Addition)	73
A.36 Honest majority protocol finalists reaction to CPU quota limit change (Fixed-point Addition)	73
A.37 Dishonest majority protocol finalists reaction to CPU quota limit change (Integer Multiplication)	74
A.38 Honest majority protocol finalists reaction to CPU quota limit change (Integer Multiplication)	74
A.39 Dishonest majority protocol finalists reaction to CPU quota limit change (Set Intersection)	75
A.40 Honest majority protocol finalists reaction to CPU quota limit change (Set Intersection)	75
A.41 Dishonest majority protocol finalists reaction to available CPU thread number change (Integer Addition)	76
A.42 Honest majority protocol finalists reaction to available CPU thread num- ber change (Integer Addition)	76
A.43 Dishonest majority protocol finalists reaction to available CPU thread number change (Fixed-point Addition)	77
A.44 Honest majority protocol finalists reaction to available CPU thread num- ber change (Fixed-point Addition)	77
A.45 Dishonest majority protocol finalists reaction to available CPU thread number change (Integer Multiplication)	78
A.46 Honest majority protocol finalists reaction to available CPU thread num- ber change (Integer Multiplication)	78
A.47 Dishonest majority protocol finalists reaction to available CPU thread number change (Set Intersection)	79
A.48 Honest majority protocol finalists reaction to available CPU thread num- ber change (Set Intersection)	79

LIST OF TABLES

2.1	Relevant testbed nodes	10
3.1	Computation domains of <i>MP-SPDZ</i> ' protocols	13
5.1	Experiment 31 scalable search preliminary results (Nodes: todd, ned and rod)	25
5.2	Experiment 31 scalable search preliminary results interpretation	26
A.1	Experiment 32 matrix integer addition preliminary results (Nodes: todd, ned and rod)	48
A.2	Experiment 32 matrix integer addition preliminary results interpretation	49
A.3	Experiment 33 matrix fixed-point addition preliminary results (Nodes: dogecoin, dogecoincash and dogecoingold)	50
A.4	Experiment 33 matrix fixed-point addition preliminary results interpretation	51
A.5	Experiment 34 matrix integer multiplication preliminary results (Nodes: bitcoin, bitcoincash and bitcoingold)	52
A.6	Experiment 34 matrix integer multiplication preliminary results interpretation	53
A.7	Experiment 35 set intersection preliminary results (Nodes: bitcoin, bitcoincash and bitcoingold)	54
A.8	Experiment 35 set intersection preliminary results interpretation	55
A.9	Experiment 31 scalable search conventional Trusted Third Party (TTP) experiment runtime results	80

CHAPTER 1

INTRODUCTION

A typical scenario when talking about Secure Multiparty Computation (SMC) is: A market survey, commissioned by a company, determines the ideal location for the next investment. But the company has to be careful, because it knows that a competitor is also looking to expand. It is in their equal interest to not accidentally get involved in the same region, so the two companies need a strategy to find out if they both plan with the same place. For competitive reasons, revealing their respective investment plan is not an option, but the information, whether the locations intersect, would be valuable to both [1].

The depicted problem of the two companies, where two or more parties would benefit from evaluating a computation over their secret data, too sensitive even for a trusted third party, can be solved by using SMC. Therefore, this Bachelor's Thesis presents tools and insight for the process of studying and finding an efficient candidate among the many SMC protocols that exist today.

In the following, a formal definition helps to specify the term SMC, while the research questions and thesis outline identify the focus.

1.1 DEFINITION

SMC refers to cryptographic protocols that enable a group of secret holders to use their individual, independent, and private data for distributed computation of a new piece of information based on their combined input. A party member can only infer additional information from what it has provided and the result of the program evaluation, while other inputs are kept concealed at any time. This is achieved with cryptographic tech-

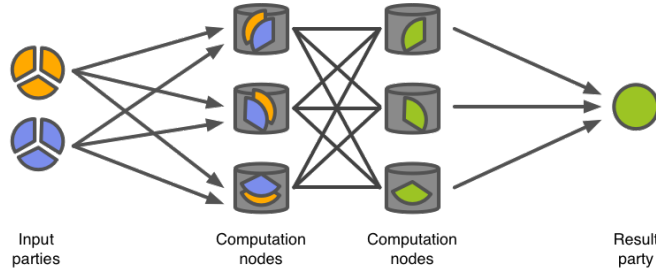


FIGURE 1.1: Secret sharing method [3]

niques such as secret sharing, illustrated in Figure 1.1, where secret inputs are split into parts that are inconclusive on their own, while mathematical operations defined within a computational domain still allow for processing them. The application domain of this technology are scenarios, where this new piece of information is of special value to the secret holders [2].

The relation between the members of a group of secret holders, who could benefit from SMC, is variable, ranging from a mostly honest group actually wishing to learn the outcome of combining their secrets and that follows the protocol, to a group where the majority is instead interested in spying on the secrets of the competitors or sabotaging the calculation with no interest in the final result. For SMC, the general definition of security requirements can be extended by the following properties, as Zhao et al. pointed out[4]:

- *Privacy*: All available information for any participant is limited to its own secret input and the computation result.
- *Correctness*: The computation result is correct
- *Input independence*: There is no dependency between the inputs given by a corrupted participant and the honest participants.
- *Output guarantee*: The result of the computation result for honest participants cannot be prevented by a corrupted participant.
- *Fairness*: A corrupted participant only receives the result if the honest participants received it as well.

For various adversary models, protocol variants have been designed that consider the dishonesty of participants but still try to fulfill the extended security demands by fortifying parts of the protocol run while accepting an efficiency trade-off [4].

1.2 RESEARCH QUESTIONS

This Bachelor's Thesis and its assisting tools have been developed with the following questions in mind:

- What is the performance of state-of-the-art SMC protocols for different reference use cases?
- How does the performance differ when using protocols of different adversary models and computation domains?
- How are environmental parameters on the host, like processing capabilities, impacting the performance of different protocols?

The strategy on tackling the research questions mainly involves the development of a convenient method to test and measure the protocol performances, allowing for comparisons of the protocols. These comparisons are expected to answer the relevant questions and many more from the same family of questions.

1.3 THESIS OUTLINE

In Chapter 2, we present information about the background of SMC and the experiment testing environment. A brief historical background depicts the origin of the theory, its state of the art and the current position of SMC in practical applications. The difference of protocol types lead to the distinction from adversary models and computation domains, which evolved during the decades of research in this field. A short overview gives insight into the categorization of various SMC protocols. The empirical part of this Bachelor's Thesis was supported by the testbed environment of the Chair of Network Architectures and Services. Relevant features of this environment will be outlined to support the understanding of the experiment setting.

In Chapter 3, we introduce work that is related to the topic of benchmarking SMC protocols. The findings of Maltitz [5] offer an orientation on what to expect from the planned experiment settings and Koch's Thesis [6] on benchmarking SMC protocols provides a source of inspiration for the interpretation of the measurement results. While Koch built around the SMC protocol collection SCALE-MAMBA, we incorporate Keller's library MP-SPDZ [7] in the developed benchmarking framework. Keller has created one of the biggest gatherings of SMC protocols, which categorizes the over 40 protocol implementations in their adversary model and computation domain. It will be utilized for the empirical part in Chapter 5.

CHAPTER 1: INTRODUCTION

In Chapter 4, we introduce the benchmarking tool *sevarebench*, a custom made framework that links the plain Orchestrating Service (pos) [8] developed at the Chair of Network Architectures and Services with the proposed SMC library. *Sevarebench* allows for convenient benchmarking of the various protocol choices and analyzing their behavior under different settings. This chapter describes how experiment data is generated, processed and presented. Some programs that solve basic reference problems were implemented and can be used as orientation for developing new, tailored experiments for custom situations.

Chapter 5 represents the empirical part of this thesis, where example experiment results, gained from using the previously introduced framework with the developed programs, are demonstrated and evaluated. Especially the influence of environmental parameters such as CPU frequency, CPU core count and available RAM on the one hand, and the choice of the SMC category on the other hand are investigated.

Chapter 6 concludes the work and gives an outlook for the future of this technology.

CHAPTER 2

BACKGROUND

The conventional way of cooperative computation typically involves a trusted third party. Since Yao and Goldreich introduced their work on SMC in the 1980s, their theory matured into a modern and secure alternative for group computations. It is becoming more practical to achieve high levels of privacy and security by advancements in protocol development and computer hardware [5].

2.1 SMC HISTORY

SMC solves scenarios where mutually distrustful entities would benefit from a computational result utilizing their individual secrets. During the two decades between the formulation of the SMC problem from chapter 1 until now, there have been many advancements in cryptography and communication technology. Researchers came up with many more use cases, resulting from the progress made in improving the efficiency and overall applicability of SMC protocols, but also from the security and privacy demands the digitalized and globalized society requests from their equipment [9].

Figure 2.1 highlights some milestones from the history of SMC. The foundation for Yao's paper "Protocols for Secure Computations" in 1982 have been laid in 1979 by Shamir and 1981 by Rabin, introducing the necessary cryptographic primitives Secret Sharing (SS) and Oblivious Transfer (OT) [10]. In his paper, Yao presents the first recorded SMC protocol for solving the Millionaires Problem for the two-party case [11]. Following it, two people can find out who is richer, without revealing their secret net worth. Five years later, Goldreich formally proved in his "Completeness Theorem for Protocols with Honest Majority" the privacy preserving guarantee for any-sized parties, in the case of an honest behaving strict majority. Shortly after in the timeline, Beaver et al. [11]

CHAPTER 2: BACKGROUND

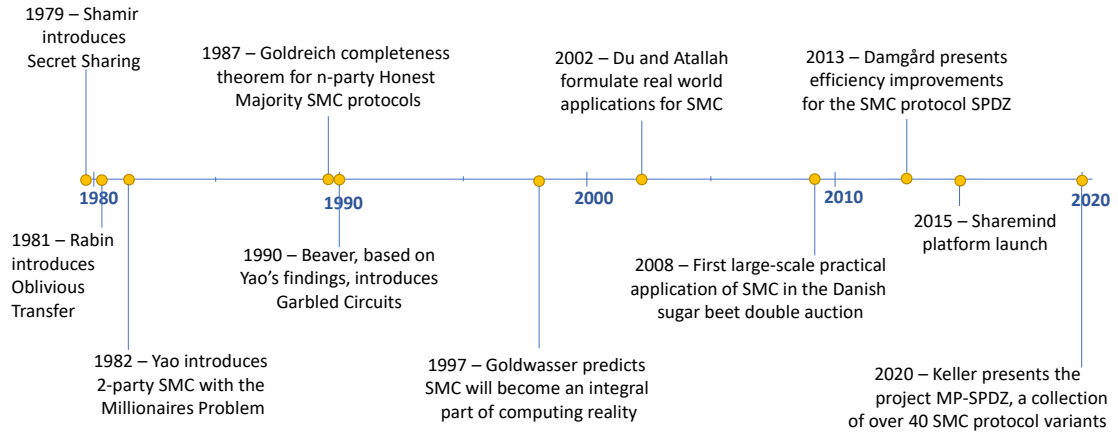


FIGURE 2.1: SMC Timeline

introduced the term “Garbled Circuits”, naming the technique he derived from Yao’s findings and now known as Yao’s Garbled Circuits. By that time, various SMC protocol variations were developed by scientists such as Goldreich, Micali, Wigderson, Ben-Or, Goldwasser, Chaum, Crépeau, Damgård and Rabin [11]. Assembling the most relevant findings in 1997, Goldwasser writes about the beauty of SMC protocols, creatively combining the many cryptographic techniques that existed, some of them originally not intended for the distributed setting contributing a critical part of many protocol variants. To name a few that haven’t been mentioned already, important tools are zero-knowledge proofs, asymmetrical cryptography, probabilistic encryption, error correcting codes, and computation with secret shares. Goldwasser predicted that SMC will become an integral part of our computing reality [12]. Many theoretical and practical examples have been given throughout the mentioned literature, where SMC can be employed. The problem used as a teaser in Chapter 1 was formulated two decades ago in 2002 by Du and Atallah [1]. With the rapid evolution of the Internet and its great potential during that time [13], they could identify even more use cases for the interconnected future [1]. Du and Atallah specify the SMC aptitude, for instance, in the fields of economy, privacy-preserving data mining, geometric and scientific computation, and statistical analysis, medicine where DNA pattern or generally profile matching is a private matter, and traditional computer science problems such as the selection problem, sorting, shortest path problem and private polynomial interpolation. The first successful large-scale practical application of SMC, involving 1229 participants, took place in 2008 [14]. In a double auction between traders of sugar beet contracts, the goal was to find the market clearing price while keeping the individual bids private. The conventional solution would have been to delegate the responsibility to a trusted third party, claiming much higher costs. In 2013, further performance and usability enhancements were accomplished and

presented by Damgård, Keller et al. in their paper [15], increasing practicability for more use cases. The first SMC-as-a-service platform, called Sharemind, launched in the year 2015, marking a major milestone with completing the transition from theory to widespread real world use. One of the platform’s main targets are companies seeking to fulfill the European privacy law [9]. Lastly, in 2020, Keller assembled many protocol variations into one code repository, creating one of the biggest SMC collections for researching and experimenting with [7]. It is still being actively developed as of today and the benchmarking tool created in the context of this Bachelor’s Thesis to analyze SMC protocols behavior in changing environments wraps around the framework.

2.2 SMC CATEGORIES

Combining an adversary model and a computation domain will be defined in the following as a SMC category. For some categories, multiple protocol candidates have been already designed, while in some cases there are none yet. An overview of popular protocols, which are being studied in the scope of this thesis, can be found in Table 3.1. These are generic protocols, meaning that they can be used to compute any discrete function with a fixed size circuit representation. Specialized protocols are being developed for a concrete SMC problem and category combination, which are optimized in performance or other requirements such as low energy or memory consumption [16]. After determining the most suitable generic protocol, the next improvement steps would be the creation of a specialized version for the problem at hand, which is not further discussed in this thesis.

2.2.1 SECURITY TYPES OF THE ADVERSARY MODELS

SMC solves scenarios where mutually distrustful entities would benefit from a computational result utilizing their individual secrets. Therefore, it is initially assumed that any participant distrusts the other members with keeping its secret. To further divide the nature of a participants behavior, security types have been introduced to describe the severity of their corruptions [4]:

- *Semi-honest* behavior

This is the least severe corruption assumption, where corrupted participants are expected to correctly follow the protocol, but want to gather as much additional information as possible. Concerning the participant behavior, SMC protocols of this type are the most optimistic and still account for the possibility that a corrupted participant can passively obtain any information on the internal state of any other corrupted participant. This type is suited for scenarios, where every

participant would benefit from the correct result as opposed to learning the others secrets or sabotaging the computation.

- *Covert* behavior

SMC protocols of this security type include a mechanism that can discover malicious behavior up to a certain probability. Corrupted participants must fear being caught cheating by other participants which could damage their reputation. For scenarios, where participants are expected to be actively malicious and computational performance is important, protocols offering covert security are a compromise if strong security protocols would perform too badly.

- *Malicious* behavior

Security against any form of malicious behavior against the protocol, like deviating from its instructions. This is guaranteed by these SMC protocols and allows a group to have maliciously behaving participants. Since any adversarial attack is promised to fail, this security type is generally the most favored. A scenario, where the computational efficiency is not a requirement or the group consists of vehemently dishonest participants, is applicable for the strongest security type. The increase in security has to be compensated with an increase in protocol algorithm complexity that leads to an expensive performance trade-off.

Together with the security type, or adversarial power, as Lindell [2] defines it, the adversary model also considers the relative size of the subset of corrupted participants from the entire group. This results in the following views:

Let n denote the cardinality of the set of participants and t of the subset of dishonest participants.

- If $t < n/2$, a strict majority of participants behaves honest:
 - *Semi-honest behavior, honest majority*
 - *Malicious behavior, honest majority*
- If $t \geq n/2$, a majority of participants behaves dishonest:
 - *Semi-honest behavior, dishonest majority*
 - *Malicious behavior, dishonest majority*

2.2.2 COMPUTATION DOMAINS

At its core, a computation domain is defined by the property of a number $M \in \mathbb{N}$. Together with M a finite ring *modulo* M is defined, typically with elements from $\mathbb{N}$.

2.3 EXPERIMENT TESTING ENVIRONMENT

Cryptographic tools in a SMC protocol operate with elements of that ring. The following classifications based on M 's property determine the mathematical structure and computational strategy behind the protocols of that domain [7]:

- $M = 2$

A finite ring *modulo 2* operates on the set $\{0, 1\}$ which also satisfies the rules of a finite field since $M = 2 \in \mathbb{P}$ is a prime number. In this computation domain, referred to as *binary* or $\mathbb{F}_2$, the arithmetic operations addition and multiplication can be viewed as evaluating an XOR and AND gate respectively. This domain gets divided into two subdomains based on the cryptographic primitive used. For protocols making use of oblivious transfer and encrypted gates the domain is referred to as *garbled circuits* and protocols implementing secret sharing are called *binary secret sharing* or $\mathbb{F}_2$ *sharing*.

- $M = 2^k, k \in \mathbb{N} > 1$

Since $2^k \notin \mathbb{P}, \forall k \in \mathbb{N} > 1$ this domain only satisfies the rules of a ring, and is therefore referred to as the computational domain *ring, modulo 2^k* or $\mathbb{Z}_{2^k}$

- $M \in \mathbb{P}$

The special case $M = 2$ is handled above. For $M > 2$, the Ring *modulo(M)* again satisfies the rules of a finite field. The domain is known as *field, modulo prime* or $\mathbb{F}_p$ and contains the most protocol candidates.

2.3 EXPERIMENT TESTING ENVIRONMENT

The measurements for the empirical part took place in the testbed environment of the Chair of Network Architectures and Services. For the purpose of conducting networking experiments, it offers a selection of various interconnected test hosts that are controllable over a single management host. In some cases, hosts are connected directly to each other (Figure 2.2a) and for larger networks they are connected over a switch (Figure 2.2b).

The interface for controlling the nodes of a testbed is called *pos* [8]. Its main features are the scriptability and reproducibility of test runs. This is achieved by the procedure of completely resetting and booting of stateless live images before each start of a run and its extensive tools for automated node interaction like uploading files from nodes to an external storage, syncing nodes of a run together and defining variables for global use or running a series of experiments with changing values.

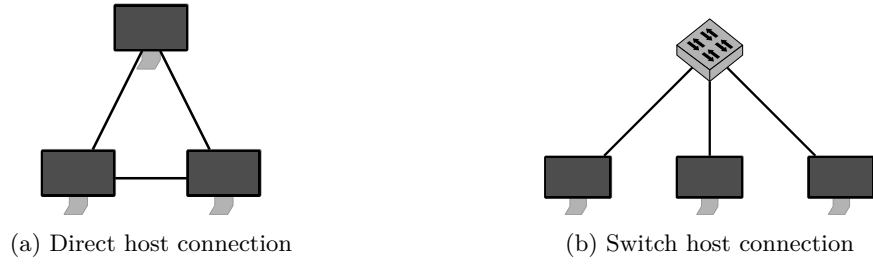


FIGURE 2.2: Available testbed topologies

node	cpu	threads	ram	link type and speed
kaunas testbed:				
valga	2x E5-2630 v4 @ 2.20GHz	2x 20	128GiB	direct 10G
rapla	2x E5-2620 v2 @ 2.10GHz	2x 12	128GiB	direct 10G
tapa	2x E5-2620 v2 @ 2.10GHz	2x 12	128GiB	direct 10G
coinbase testbed nodes bitcoin*, ether*, dogecoin*:				
	D-1518 @ 2.20GHz	8	32GiB	switch 1G
springfield testbed nodes todd, ned, rod:				
	D-1518 @ 2.20GHz	8	128GiB	direct 10G

TABLE 2.1: Relevant testbed nodes

Nodes that were primarily used in the context of this thesis are mentioned with relevant specifications in Table 2.1.

* The used coinbase nodes bitcoin, bitcoincash, bitcoingold, ether, ethercash, ethergold, dogecoin, dogecoincash, dogecoingold are all identical.

CHAPTER 3

RELATED WORK

Today, there exist a number of projects that provide implemented SMC protocols for testing purposes. Rotaru [17] maintains a comprehensive list of software related to SMC. Von Maltitz chose the *Framework for Efficient Secure Computation (FRESKO)* for his research.

3.1 ASSESSMENT OF SECRET SHARING BASED SMC

The authors of the paper *A Performance and Resource Consumption Assessment of Secret Sharing Based Secure Multiparty Computation*, von Maltitz and Carle [5], recognized a need for more thorough benchmarking of SMC solutions. While protocol projects kept improving in algorithmic efficiency, a broader trend of conducting real world performance measurements beyond a few runtime experiments was still lacking. The authors present results for one specific, more complex, real world SMC problem and analyze various parameters. The underlying mathematical problem to solve was the arithmetic mean, with the context being to calculate the average traveled distance of the participants' secrets in a privacy-preserving and secure manner. Interesting findings from their research were:

- Preliminary execution time considerations

Abstracting the participants and their end devices as a network of processors and viewing secret sharing based protocol runs as an alternating sequence of computation and communication, the overall cost of a calculation can be estimated by the sum of the costs of each individual computation plus the communication cost

multiplied with the communication rounds. With one more round for computing the final result, the formula is

$$cost_{overall} = \sum_{i=1}^{m+1} cost_{comp_i} + m \cdot cost_{comm}$$

This concludes that a secret sharing based protocol's performance is dependent on the properties of the participants equipment and the network connecting them in a special way. Because $comp_i$ can only be computed in sequence after $comp_{i-1}$, while the computation and communication happens concurrent, the bottleneck of the entire network is always defined by the weakest node and the slowest connection between a pair of nodes in terms of processing power and networking capabilities.

- SMC vs. TTP

The alternative and conventional way of multi party computation involves a TTP that needs to know every secret input. Comparing the costs with each other, it becomes clear that with a growing number of participants, the costs scale badly for SMC. Just the communication cost for the opening and closing phases are $2n^2 - 2n$ in the BGW SMC protocol for n participants [5], while with a TTP solution it only grows linear with $2 * n$ transfers, only once sending to and receiving the input from for each participant. On top of that, the initialization of secret sharing based protocols requires each participant to generate a polynomial, calculate n shares and evaluate the Lagrange interpolation at the end to reconstruct the final result. These are expensive operations in contrast to none required in a TTP scheme.

In the scope of this Bachelor's Thesis, a simple comparison run has been conducted with the help of an adapted version of the introduced SMC framework. The scalable search problem is implemented in C++ and executed on a node acting as the TTP after the secret inputs were transferred in an encrypted manner using public key cryptography. Measurement results in Table A.9 show that this variant outclasses the SMC version by magnitudes, but the secrets were used in an unconcealed manner within the computation.

- Observed effects of network parameters

While this thesis studies the effects of host parameter changes on runtime, the work by von Maltitz and Carle primarily analyzed how a changing networking environment influences host parameters and runtime. For heap RAM, it was observed that network parameters had negligible impact, but an increase in participants affected the size of the reserved heap space in a linear trend up to 15 peers. It was argued that it stems from the increased number of shares with every additional participant and connection data that have to be stored. With the side effect of

3.2 MULTI-PROTOCOL SPDZ COLLECTION

Adversary model	<i>modulo prime</i> ($\mathbb{F}_p$)	<i>modulo 2^k</i> ($\mathbb{Z}_{2^k}$)	Binary secret sharing ($\mathbb{Z}_2$ SS)	Garbled circuits ($\mathbb{Z}_2$ GC)
Malicious, dishonest majority	MASCOT, LowGear, HighGear	SPDZ2k	Tiny, Tinier	BMR
Covert, dishonest majority	CowGear, ChaiGear	N/A	N/A	N/A
Semi-honest, dishonest majority	Semi, Hemi, Temi, Soho	Semi2k	SemiBin	Yao's GC, BMR
Malicious, honest majority	Shamir, Rep3, PS, SY	Brain, Rep3, PS, SY	Rep3, CCD, PS	BMR
Semi-honest, honest majority	Shamir, ATLAS, Rep3	Rep3	Rep3, CCD	BMR

TABLE 3.1: Computation domains of *MP-SPDZ*' protocols

busy waiting during the setup phase in mind, where waiting nodes consume CPU cycles until all others are ready to continue, an increase in network latency leads to a slightly higher amount of CPU cycles. Later in his dissertation, von Maltitz also assessed the effects of host parameters on SMC protocols [18].

3.2 MULTI-PROTOCOL SPDZ COLLECTION

SPDZ (pronounced “Speedz”) is the nickname of the MPC protocol of Damgård et al. [7] from Crypto 2012. *MP-SPDZ* [7] is a framework by Keller that is being actively developed and offers over 40 implemented and ready to use SMC protocol candidates. It was presented in 2020 at the ACM SIGSAC Conference on Computer and Communications Security. Utilizing this software on constructed reference SMC problems and measuring the costs is the main work of this thesis. The MP-SPDZ-project was created primarily for the purpose of benchmarking various SMC protocol variants in cryptographic research projects. Therefore, it is a core element of this Bachelor's Thesis. The team behind the project, Keller et al., offers a rich amount of documentation, which supports the test case construction. Table 3.1 displays the supported protocols of which many are analyzed in Chapter 5. The main reason why this SMC collection was chosen to be utilized is the practicability of the implemented virtual machine for high level Python-like code and the diverse selection of computation domains and adversary models.

CHAPTER 4

SEVAREBENCH: BENCHMARKING FRAMEWORK FOR SMC PROTOCOLS

SecreteTUM servare, “*to keep secret*” in Latin, or short sevarebench, is a bash-script framework that was developed by the author in the context of this thesis. It aims to connect the MP-SPDZ SMC protocol collection with the Chair of Network Architectures and Services’ testbed environment and enable a wide spectrum of functionality. The main feature is to imitate various environment settings for conducting SMC experiments and analyze their behavior under different circumstances. Some of its extended features are scheduled config runs, exception catching and error reporting, verbosity, expandability, and automatic test result parsing and exporting.

4.1 FUNCTIONALITY AND MODULES OVERVIEW

There are three ways to initiate an experiment run:

In the first and second line of Listing 4.1 is the direct way of starting sevarebench. The options with the desired parameters are named with their short or long name. The third line shows the config run variation, where the experiment run settings are defined in a file, for example see Listing 4.2

```
1 $ ./sevarebench.sh --experiment 33_matrix_add_float --nodes node1,node2,node3
2   --protocols mascot,atlas --input 10,20,...,100
3 $ ./sevarebench.sh --config configs/ex33/DisHon-quotas.conf node1,node2,node3
4 $ ./sewaresched.sh configs/ex32/Hon-quotas.conf node1,node2,node3 sevarelog
```

LISTING 4.1: Start sevarebench

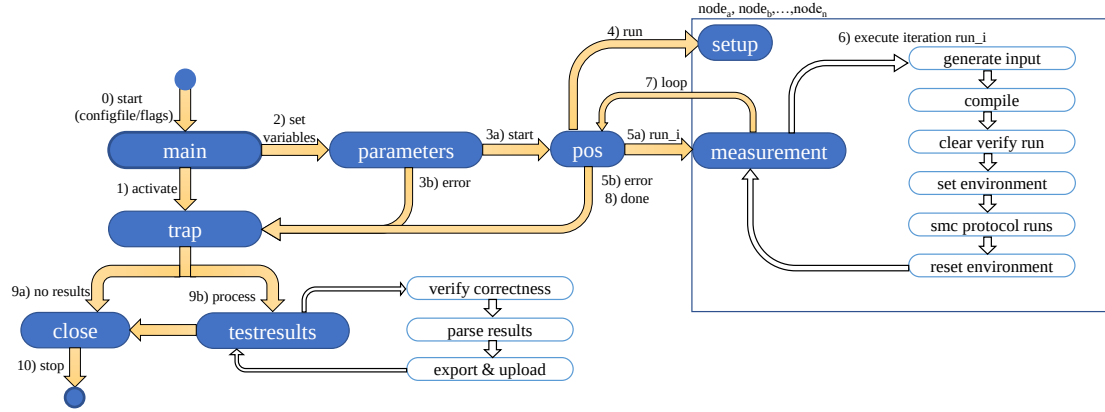


FIGURE 4.1: Benchmarking framework posSMCbench

```

1  nodes=node1,node2,node3
2  # mandatory parameters
3  experiments=31_scalable_search,35_set_intersection
4  protocols=shamir,malicious-shamir,spd2k,brain,yao,ccd
5  input=20,40,...,200
6  # optional parameters
7  cpu=16,8,4
8  cpuquota=240,180,120,60
9  freq=2.6,2.0
10 measureram
    
```

LISTING 4.2: Config file example

The config run mode supports the feature to name multiple experiments that will run consecutively with the same settings. The third option in line 4 of Listing 4.1 was created to schedule the next experiment to automatically follow after the currently running one. Every few minutes the availability of the test nodes is checked to determine whether the next run can be executed.

The experiment then starts the run in the main module `sevarebench.sh`, in Figure 4.1 located in the top left corner. A trap script gets activated to handle exceptions caught along the execution and gracefully close all process childs created upon exit. The demanded parameters get loaded in the helper module `parameters`, where the experiment settings are processed, checked for validity and displayed at their respective location. Essentially, the loop variable `file` for `pos` is generated, the protocols are split up into their domain and the test types get initialized. With no errors, which would be delegated to the trap that exits the program in 3b, the `pos` interface is utilized in the `pos` module to run the setup scripts on the hosts that install the required packages, compile the binaries for requested SMC protocols and prepares the environment manipulation. If this is accomplished, `sevarebench` starts the measurement module loop via `pos` in step 5a. For details on step 6 in Figure 4.1 view Section 4.2. At step 8, measurement data has been generated and uploaded to the management node which will now be processed even if the experiment did not fully complete, maybe because of an error or user interruption. The implemented trap logic detects that an experiment has begun measuring and starts the `testresult` module. Section 4.3 explains the `testresults` module in detail. Finally, `sevarebench` closes all subprocesses and stops.

4.2 MEASUREMENT MODULE

4.2.1 UNCONCEALED VERIFICATION COMPUTATION

An input generator function is called that produces random test values for each player to run the computation with. The amount and characteristics of the generated inputs is experiment specific and part of its definition. For instance, the integer matrix multiplication experiment generates $(inputsize)^2$ many integers between 1 and 5 for each player. Every time the same random numbers are output, fixed to the player's number as seed. These inputs are not required for compilation of a high level SMC program, only the arguments the program will be executed with must be provided. The implemented programs need the party and input size accessible in the arguments array. Therefore, the measurement module skips compilation if the program has already been compiled with the current input size, party-size combination in a previous iteration run, reducing overall experiment runtime costs. Part of an experiment's definition is also a script that

implements the solution to the experiment’s problem in an unconcealed manner. Before the host environment gets manipulated it is called with the generated input values and delivers verification results for proving the correctness of the secure computation in the measurement processing phase.

4.2.2 SMC IN A MANIPULATED HOST ENVIRONMENT

Pos creates the cartesian product of the defined loop variables for the iterations and the test type values combinations are used to set the environment in this step of the measurements module. Sevarebench supports the following test types:

- **-cpu** Limit the number of available virtual CPU cores/threads to the given value. This is achieved by setting `/sys/devices/system/cpu/cpu*/online` to false
- **-cpuquota** Cap the maximum share of total usable CPU time across all cores to a given value. Prepending a bash command with `systemd-run -scope -p CPUQuota=value%` achieves this by using cgroups in the background
- **-ram** limit the maximum RAM to a value in MiB. Using this option only makes sense if the system can swap RAM contents out, otherwise the entire system would crash. Therefore, this test type only takes effect on systems where a block device is attached at `/dev/nvme0n1`. If this is the case, 4 GiB (or a custom value defined with `-swap value`) will be used as swap memory. In the measurement module the unwanted RAM is allocated to only leave the defined value free.

Eisermann is collaborating in the development of sevarebench and will introduce the following network specific test types and measurement results in his Bachelor’s Thesis on benchmarking networking specific environmental effects on SMC protocol performance: `-latency`, `-bandwidth`, `-packetdrop`.

4.3 TESTRESULTS MODULE

If a test run iteration has begun, the trap calls the verification function of the testresults module upon finish or error. In case of an error, the last captured SMC protocol output is printed, usually containing valuable debug information. All found pos uploads will be processed, first for comparing the verification result with the SMC result to determine the validity of a run, and second for the measurement data dump. A csv-file that includes the experiment’s results, a table version derived from it and a summary file are the exported results. In the last step, these files are uploaded to a defined repository.

4.4 IMPLEMENTED SOLUTIONS FOR REFERENCE SMC PROBLEMS

Five elemental SMC problems were solved to allow the gathering of empirical data for Chapter 5, as well as provide a starting point for developing further solutions to more complex and extensive use cases. The MP-SPDZ project offers a compiler that translates high-level, Python-like code into bytecode that runs on an included, SMC specific virtual machine. This translation allows the reuse of the same high-level code for any of the supported protocols. During the development of the following five programs, we observe that the compatibility is limited to some degrees. For each of the three computational domains, the compiler creates a different bytecode. Even if it successfully compiles a program for one domain and the executed binary program produces the correct result, it is not guaranteed that compiling works for the other domains. Even minor details led to compilation errors for $\mathbb{F}_2$ while working in $\mathbb{Z}_{2^k}$ and $\mathbb{F}_p$. Another issue that was overcome is that a program that compiled fine for all computation domains only correctly worked for some, while protocols from other domains were returning completely wrong results. This was observed when directly operating on the container type “*Array*” of the compiler module, like adding two arrays together via the supported slice indexing. For $\mathbb{F}_2$ this approach returned wrong results. To achieve the goal of benchmarking every SMC category, code was written that compiles and delivers correctly for each computation domain. In the following, the developed code for the reference SMC problems will be presented that achieved compatibility for every domain, highlighting the adjusted part. The final programs were supported and inspired by the examples given by Keller that are included in the MP-SPDZ projects source code. Every problem was additionally solved in Python to allow verification of the computed SMC result. The Python code will not be further discussed.

- scalable search

```

1 def scalable_search(haystack, needle, size):
2     result = MemValue(sint(0))
3     @for_range(size)
4     def _(i):
5         result.write(result + (haystack[i] == needle))
6     print(result.reveal())

```

LISTING 4.3: High level source code for scalable search

This code is an adaptation of Kellers’ **scalable search** implementation in his **tpmpc tutorial**. It iterates over a set of integers (haystack) and compares each element with another element (needle). If the needle is found it returns “SMC-Result: Needle in Haystack? 1”

- element-wise matrix operations

```

1 program.bit_length = 64
2
3 size = int(program.args[1])
4 player = int(program.args[2])
5
6 inputs = Matrix(player, size*size, dtype)
7
8 for j in range(player):
9     for i in range(size*size):
10         inputs[j][i] = dtype.get_input_from(j)
11
12 @for_range(size*size)
13 def _(i):
14     result = MemValue(dtype(identity))
15     @for_range(player)
16     def _(j):
17         result.write(result (binop) inputs[j][i])
18
19 print(result.reveal())

```

LISTING 4.4: High level source code for matrix operations

Datatypes *dtype*: sint, sfix

Binary operators (*binop*): +, *

Identity elements identity: 0, 1

The solutions of the three arithmetic problems matrix integer addition, fixed-point addition and integer multiplication differ in small details. For integer arithmetic, the data type *dtype* is sint and for fixed-point sfix. Similarly, *emph(binop)* is * for multiplication and + for addition. The initial value of the result variable has to be loaded with the identity element of the respective operation, 1 for multiplication and 0 for addition. The sfix data type allows for custom precision with the command `print_float_precision(n)`, where n is the number of overall digits.

The amount of inputs and participants is given via parameters to the program and based on these sizes, the participants inputs will be read from a file into a matrix. The matrices of the participants are always of quadratic size and are abstracted to an array. The matrix rows contain the players values val_0 to val_i while the columns 0 to j represent each player's value at that respective index.

For compatibility reasons, the operations are done manually in two loops over the inputs matrix, summing or multiplying up column for column and revealing the final result of the secret computation after all elements of a column have been used. This approach, which involves using the container type *MemValue()* and immediately printing the result before starting with the next column, was

4.4 IMPLEMENTED SOLUTIONS FOR REFERENCE SMC PROBLEMS

followed to support $\mathbb{F}_2$ protocols. Otherwise, using the endorsed slice indexing led to arbitrary results.

- set intersection

```
1
2 @for_range(size)
3 def _(i):
4     result = MemValue(sint(0))
5     @for_range(1,player)
6     def _(j):
7         result.write(result + scalable_search(inputs[j], inputs[0][i], size))
8
9     result.write(if_else(result == 0, sint(0), inputs[0][i]))
10    print(result.reveal())
```

LISTING 4.5: High level source code for set intersection

The already implemented `scalable_search()` function from above helps with this problem and the loading procedure of the inputs matrix is the same as in the matrix operations programs' beginning. The outer loop iterates over the first participant input set, comparing each of its elements, or needles, with every input from the other participants. The inner loop iterates over the rows that represent the participants sets, or haystacks, calling the scalable search. The result variable stays 0 if none of the other players contain the current needle, which leads to the if else statement returning `sint(0)`, or if the needle is found, the result variable will contain a number $\geq \textit{needle}$ and the needle will be returned.

This approach leaks information about the first player's values, so it should only be applied for the use case, where a dataset is used for the first participant's input that the others are already familiar with.

CHAPTER 5

PERFORMANCE ASSESSMENT OF SMC PROTOCOLS

With the help of the developed benchmarking framework *sevarebench*, in a variety of experiment runs, the performance of the protocols from MP-SPDZ' collection is analyzed. Five different reference use cases were implemented, including a search problem, arithmetic computations and set operations, to assess the influence the underlying mathematical strategy of a protocol has on the performance. Furthermore, the cost of increased security, that could be gained by choosing a stricter adversary model, is a relevant question for the decision making process when considering the application of SMC. After an empirical based election of protocol implementations, a behavioral analysis is conducted by comparing metrics of host environment specific parameters.

The resulting tables and plots of the reference use case *scalable search* is presented and discussed in this Chapter, while the respective results of the other four reference use cases are only discussed here. The tables and plots for the experiments *matrix integer addition*, *matrix fixed point addition*, *matrix integer multiplication*, and *set intersection* are found in the Appendix A.

5.1 ELECTING PROMISING PROTOCOLS FOR REFERENCE SMC PROBLEMS

In two rounds, the 37 implemented protocol candidates compete against each other to single out the best performing solution for a given adversary model and SMC program. In the context of the third research question, these will then be analyzed more in-depth. During those steps, underperforming protocols will be gradually filtered, so that

only the most feasible candidates proceed for further assessment. This approach was chosen to present a concise answer of the research questions, specifically the study of the host environments effect. It is duly noted, that the protocols are run in their default setting as is. Many adjustments could be made that could influence the performance in a way, where the measurement of the optimized version of two candidates leads to a different reasoning and exclusion result. For fairness, all the protocols were used in their pre-configured state. In step one, the initial exclusion of underperforming candidates, for each reference problem, every supported protocol’s runtime will be compared to exclude candidates with the worst performance. In step two, given an adversary model, comparing the performance of protocols with candidates from different computation domains gives further insight into what mathematical strategy suits the given SMC program the most. A representative implementation for each of the four adversary models will be selected for the behavioral analysis in the next chapter.

5.1.1 INITIAL EXCLUSION OF UNDERPERFORMING CANDIDATES

Even though the goal of this thesis is to benchmark and analyze the behavior of SMC protocols in changing host environments, it is out of scope to achieve a meaningful in depth assessment for every candidate of the 37 supported. Therefore, similarly to a SMC protocol designer’s expected approach, in the first step a general performance overview was obtained. For designing problem specific SMC protocols, it can be a source of inspiration to first assess the generic protocols that exist and narrow them down to a subset of better performing ones. In this initial measurement, obviously bad performing candidates will be omitted in favor of others in the same SMC category. Also, if it was observed that a protocol with a stronger security type outperforms a less secure one, studying the slower one has little meaning. Furthermore, protocols with infeasible runtime or RAM costs will also be unselected in this step. The setup to conduct this experiment is chosen to deliver fast results by leaving the host environment untouched and limiting the element count variable to two values with approximately a factor of 10 distance between them. The column Adv. model stands for adversary model and uses a shortened description, where MalDishonset refers to the malicious, dishonest majority model, describing with the second part the behavior of the majority.

SCALABLE SEARCH

LISTING 5.1: Config file for initial run of Experiment 31 scalable search

```

1  experiments=31_scalable_search
2  # protocols
3  all
4  input=50,500
5  measureram
```

5.1 ELECTING PROMISING PROTOCOLS FOR REFERENCE SMC PROBLEMS

SMC category		Element count:	50	500	50	500
Adv. model	C. Domain	Protocol	Runtime [s]		RAM [GiB]	
MalDishonest	$\mathbb{F}_p$	mascot	4.75	34.30	0.37	0.38
MalDishonest	$\mathbb{F}_p$	lowgear	514.10	33.96	1.22	0.13
MalDishonest	$\mathbb{F}_p$	highgear	1,094.78	59.02	2.56	0.27
MalDishonest	$\mathbb{Z}_{2^k}$	SPDZ2k	5.54	34.98	0.80	0.81
MalDishonest	$\mathbb{Z}_2$ SS	tinier	1.07	6.59	0.08	0.10
MalDishonest	$\mathbb{Z}_2$ GC	real-bmr	44.08	441.01	0.50	1.57
CovertDishonest	$\mathbb{F}_p$	cowgear	14.48	34.27	0.14	0.12
CovertDishonest	$\mathbb{F}_p$	chaigear	70.98	58.87	1.22	0.29
SemiDishonest	$\mathbb{F}_p$	semi	1.29	11.11	0.06	0.06
SemiDishonest	$\mathbb{F}_p$	hemi	1.05	6.15	0.04	0.05
SemiDishonest	$\mathbb{F}_p$	temi	0.68	4.42	0.04	0.03
SemiDishonest	$\mathbb{F}_p$	soho	2.12	7.43	0.16	0.11
SemiDishonest	$\mathbb{Z}_{2^k}$	semi2k	0.28	2.34	0.03	0.03
SemiDishonest	$\mathbb{Z}_2$ SS	semi-bin	0.44	2.21	0.20	0.20
SemiDishonest	$\mathbb{Z}_2$ GC	yao	0.02	0.11	0.01	0.03
SemiDishonest	$\mathbb{Z}_2$ GC	yao -O	<0.01	0.09	0.01	0.03
SemiDishonest	$\mathbb{Z}_2$ GC	semi-bmr	5.88	56.80	0.12	0.61
MalHonest	$\mathbb{F}_p$	malicious-shamir	0.12	1.18	0.02	0.02
MalHonest	$\mathbb{F}_p$	malicious-rep-field	0.10	0.95	0.02	0.02
MalHonest	$\mathbb{F}_p$	ps-rep-field	0.08	1.00	0.02	0.02
MalHonest	$\mathbb{F}_p$	sy-rep-field	0.13	1.36	0.03	0.03
MalHonest	$\mathbb{Z}_{2^k}$	brain	0.22	1.34	0.03	0.03
MalHonest	$\mathbb{Z}_{2^k}$	malicious-rep-ring	0.09	0.81	0.02	0.02
MalHonest	$\mathbb{Z}_{2^k}$	ps-rep-ring	0.11	0.83	0.02	0.02
MalHonest	$\mathbb{Z}_{2^k}$	sy-rep-ring	0.10	0.84	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	malicious-rep-bin	0.20	1.81	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	malicious-ccd	0.28	2.90	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	ps-rep-bin	1.74	3.88	0.40	0.38
MalHonest	$\mathbb{Z}_2$ GC	mal-shamir-bmr	1.54	11.73	0.09	0.64
MalHonest	$\mathbb{Z}_2$ GC	mal-rep-bmr	0.98	7.77	0.10	0.75
SemiHonest	$\mathbb{F}_p$	shamir	0.07	0.92	0.01	0.02
SemiHonest	$\mathbb{F}_p$	atlas	0.19	1.75	0.02	0.02
SemiHonest	$\mathbb{F}_p$	replicated-field	0.05	0.70	0.01	0.02
SemiHonest	$\mathbb{Z}_{2^k}$	replicated-ring	0.03	0.44	0.01	0.01
SemiHonest	$\mathbb{Z}_2$ SS	replicated-bin	0.12	1.45	0.01	0.02
SemiHonest	$\mathbb{Z}_2$ SS	ccd	0.15	2.17	0.01	0.02
SemiHonest	$\mathbb{Z}_2$ GC	shamir-bmr	1.05	9.20	0.07	0.55
SemiHonest	$\mathbb{Z}_2$ GC	rep-bmr	0.59	4.54	0.06	0.43

TABLE 5.1: Experiment 31 scalable search preliminary results (Nodes: todd, ned and rod)

SMC category		Exclusions and reasoning
Adv. model	C. Domain	
MalDishonest	$\mathbb{F}_p$	HighGear is outperformed, omit, LowGear is a little slower but requires considerably less RAM than MASCOT, keep both this BMR version explodes in runtime and is therefore infeasible, omit
MalDishonest	$\mathbb{Z}_2$ GC	
CovDishonest	$\mathbb{F}_p$	CowGear and ChaiGear perform similar or worse than stricter security types, omit this adversary model for almost free security
SemiDishonest	$\mathbb{F}_p$	Temi outperforms Semi , Hemi and Soho in observed parameters. semi-bmr diverges away from yao very fast in observed parameters, Yao performs well for the 2-party case
SemiDishonest	$\mathbb{Z}_2$ GC	
MalHonest	$\mathbb{F}_p$	SY can be omitted in favor of the other candidates Brain is slower by a factor of two from the rest and needs many times more RAM
MalHonest	<i>modulo</i> 2^k	
MalHonest	$\mathbb{Z}_2$ SS	PS performs the worst of this group, omit the mal-shamir version has a slightly lower RAM reservation peak, but is almost half the speed of the mal-rep version
MalHonest	$\mathbb{Z}_2$ GC	
SemiHonest	$\mathbb{F}_p$	ATLAS is outperformed by Shamir and Rep3 in observed parameters, omit
SemiHonest	$\mathbb{Z}_2$ SS	CCD has a worse performance than Rep3 the shamir version of BMR performs worse than the rep version for all observed parameters
SemiHonest	$\mathbb{Z}_2$ GC	

TABLE 5.2: Experiment 31 scalable search preliminary results interpretation

Tabel 5.2 gives an overview of protocols, highlighted in bold, that were chosen to not further assess. The reasoning is based on results in Tabel 5.1, which shows measured runtimes and peak RAM usage for two specific input sizes. For each of the other four experiments, the resulting Tables can be found in the Appendix Section A.1.1

5.1.2 COMPARING PROTOCOLS OF THE SAME ADVERSARY MODEL

In this round, the protocols and their computation domain are compared with each other within their respective adversary model. In other words, the protocols on the same row in Table 3.1 are put into perspective with their runtimes, peak RAM usage and CPU frequency sensitivity. For a better overview, the resulting plots for the experiments *matrix integer addition*, *matrix fixed point addition*, *matrix integer multiplication*, and *set intersection* are found in the Appendix A.1.2. After determining the fastest candidate of a SMC category, the overall fastest protocol from that model will be chosen for the next section. Whenever a parameter is fixed, the theoretically least constraining value is selected, namely 8 cores for the CPU thread count, 240% for the CPU Quota and 3 GHz as CPU frequency. If the input size is fixed, always the highest value is selected.

5.1 ELECTING PROMISING PROTOCOLS FOR REFERENCE SMC PROBLEMS

SCALABLE SEARCH

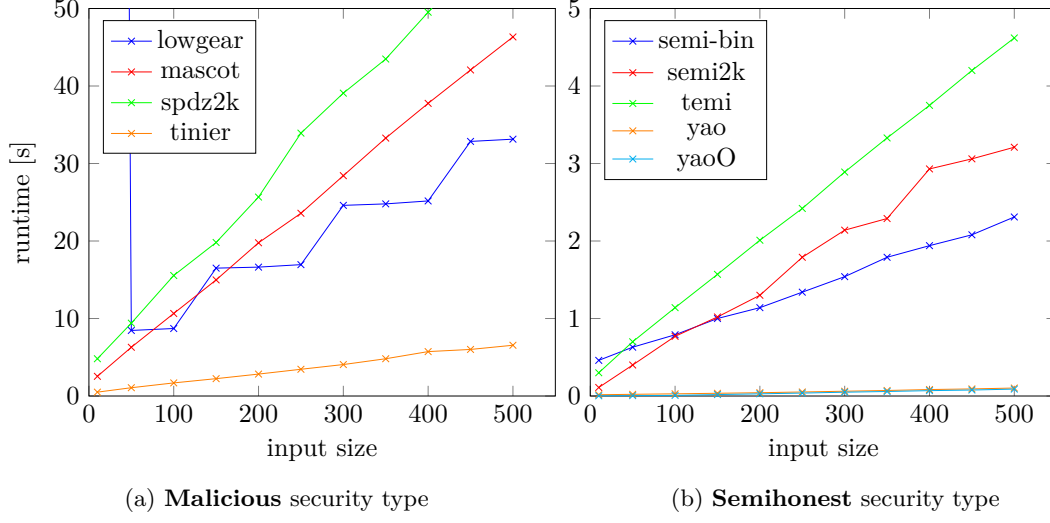


FIGURE 5.1: **Dishonest majority** protocols with their runtimes for scalable search on valga

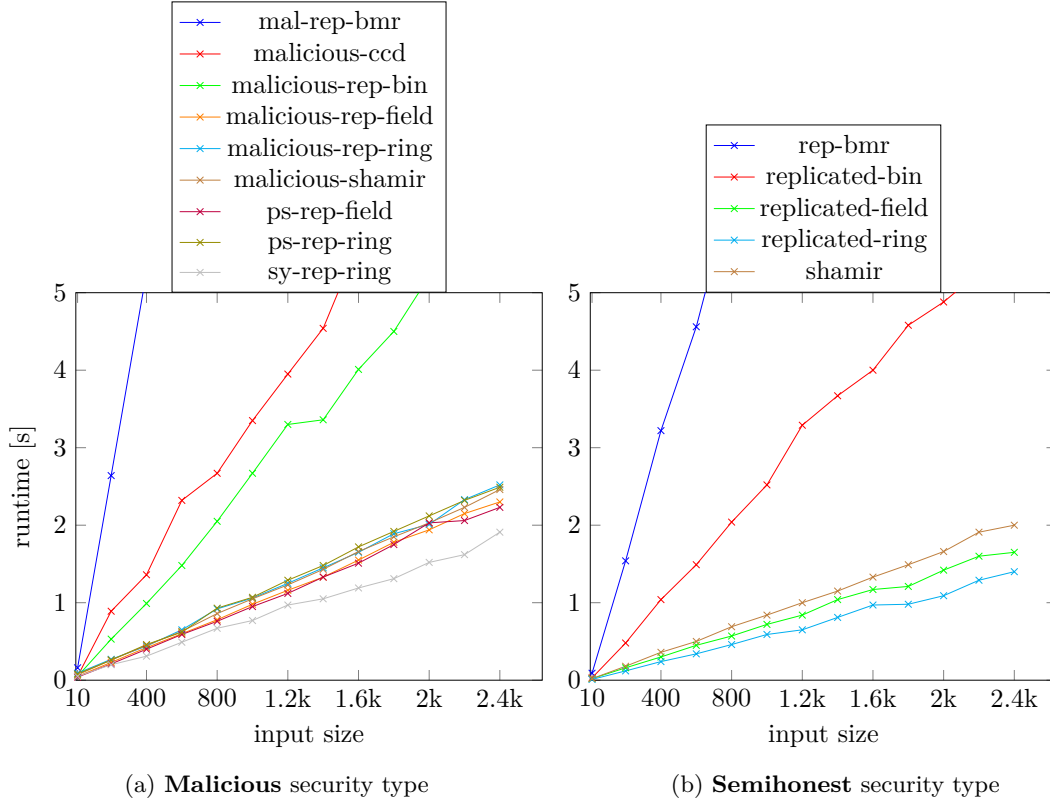


FIGURE 5.2: **Honest majority** protocols with their runtimes for scalable search on valga

CHAPTER 5: PERFORMANCE ASSESSMENT OF SMC PROTOCOLS

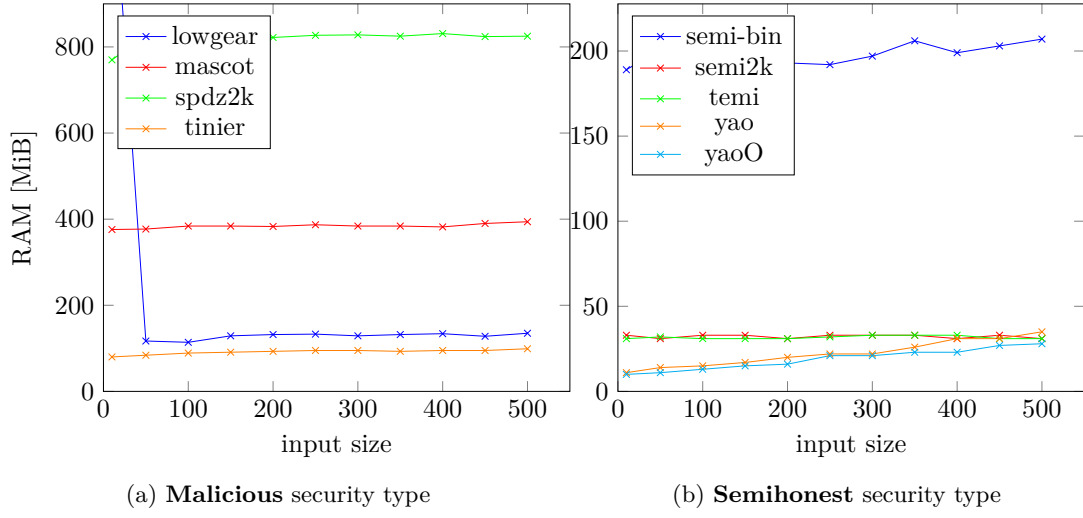


FIGURE 5.3: **Dishonest majority** protocols with their peak RAM usage for scalable search on valga

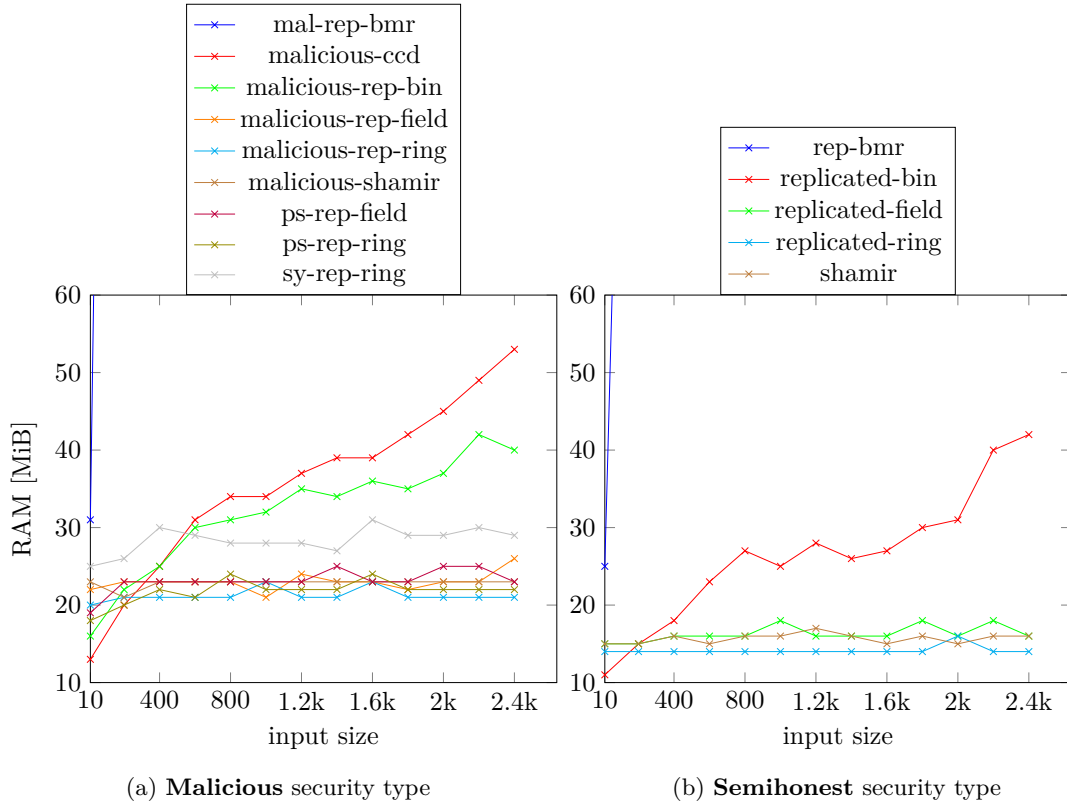


FIGURE 5.4: **Honest majority** protocols with their peak RAM usage for scalable search on valga

5.1 ELECTING PROMISING PROTOCOLS FOR REFERENCE SMC PROBLEMS

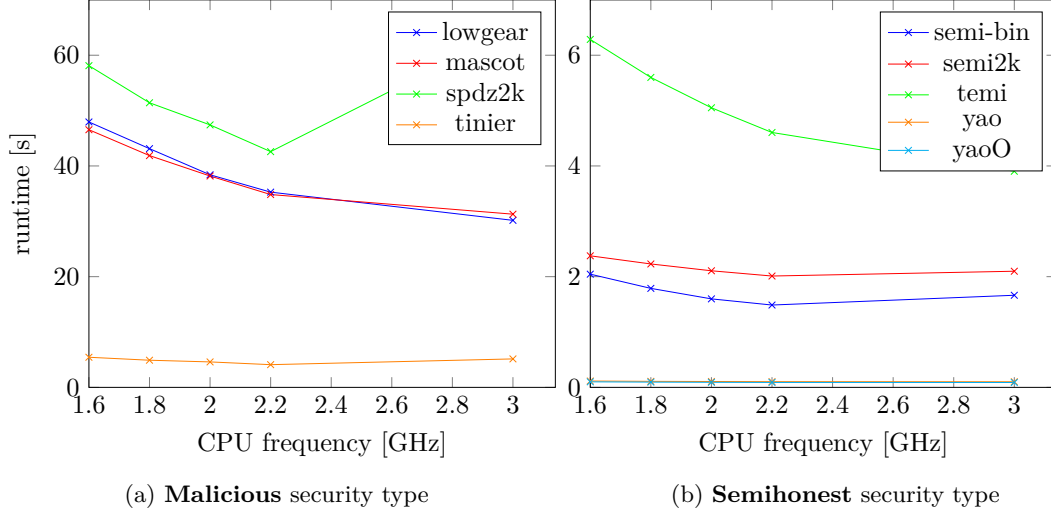


FIGURE 5.5: **Dishonest majority** protocols with their runtimes for scalable search based on frequencies with fixed input size 500 on valga

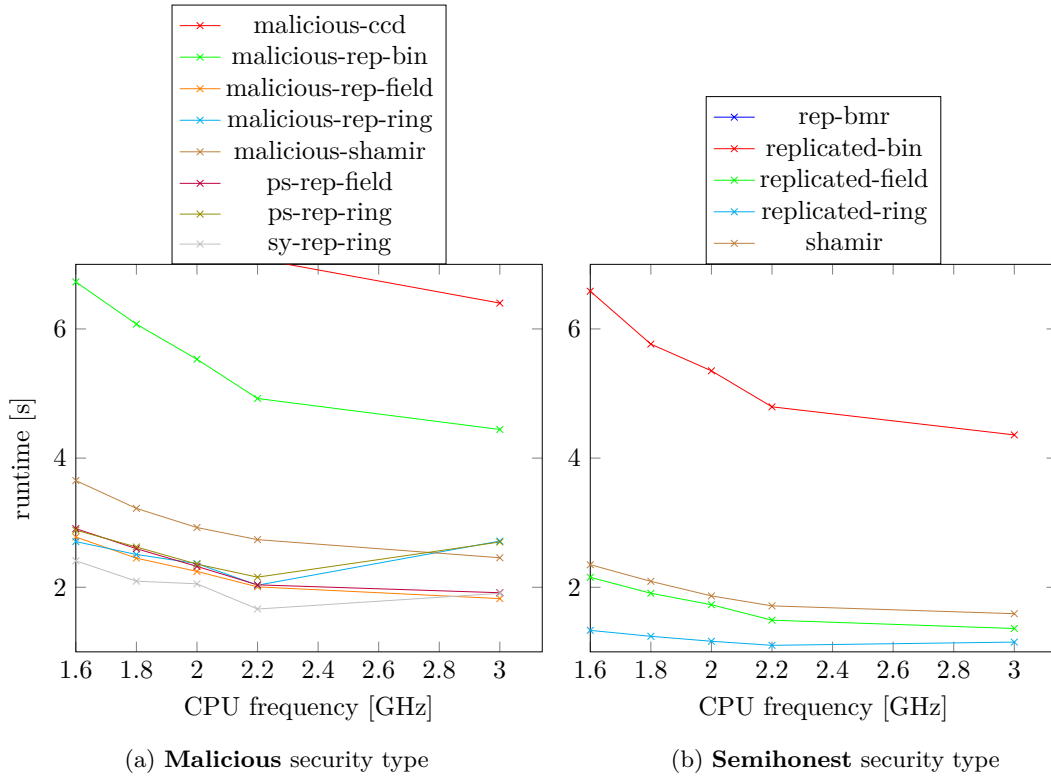


FIGURE 5.6: **Honest majority** protocols with their runtimes for scalable search based on frequencies with fixed input size 2400 on valga

Scalable Search: It becomes clear from Figure 5.1a that lowgear and tinier are the best candidates in the malicious dishonest model, while in Figure 5.1b the binary and ring version of semi are the fastest in the semihonest-dishonest majority model. After the long initialization phase of lowgear, which can also be found in Table 5.1, it beats mascot in their category with growing input sizes. Tinier and semi-bin, both of the computation domain $\mathbb{Z}_2$ SS, get selected as the representative of their adversary model to proceed to the next assessment. Of the two yao variants, yaoO is faster than the default yao variant. YaoO does not garble the circuit in chunks whenever received but all at once.

The computational domains of the malicious behavior, honest majority model (Figure 5.2a) have multiple candidates that can be compared. In $\mathbb{F}_p$, it is a close call, with all three candidates performing similarly. Observing the trend, ps-rep-field has the lead, malicious-rep-field comes in second and shamir goes in last. Of the $\mathbb{Z}_{2^k}$ protocols, sy-rep-ring has a lower runtime at every measure point and leads this domain, malicious-rep-ring and ps-rep-ring are too close and would require further measurements. The $\mathbb{Z}_2$ domain contains the slowest protocols, with malicious-rep-bin performing better. The overall fastest protocol candidate in this model is sy-rep-ring of domain $\mathbb{Z}_{2^k}$, which proceeds for further assessment. The most relaxed model semi honest, honest majority offers the best runtimes with replicated-ring performing best, the measurements show in Figure 5.2b.

When we compare the peak RAM consumption, the measurement results portray a general trend of securer protocols requiring more memory than protocols from a less secure adversary model. For most of the candidates, the RAM costs grow less with increased input sizes compared to the runtime, seemingly staying stable for some candidates. The malicious dishonest protocols consume the most RAM, observable in Figure 5.3a. For example, mascot starts out very high at almost 400 MiB for an input size of 10, but never crosses above 400 MiB even for a 50 times higher input size. For the initialization phase, lowgear consumes the most RAM, but then drops to a similar level as tinier. The scale of the y-axis in Figure 5.3b reduces from 800 MiB for the malicious security type to 200 MiB, with the semi-bin as the most demanding in RAM at around 200 MiB. The other candidates are located below 50 MiB with interesting trends, where $\mathbb{Z}_2$ protocols Yao and YaO eventually grow above $\mathbb{F}_p$ protocol temi. The malicious security type of the honest majority model offers protocols with low RAM requirement with the exception of mal-rep-bmr. Figure 5.4a barely captures the high RAM requirement of this bmr variant. At input size 2400, mal-rep-bmr needs over 2 GiB while the other candidates stay below 100 MiB. The binary protocols malicious-ccd and malicious-rep-bin are next in place in high RAM consumption, while protocols from the domain $\mathbb{F}_p$ and $\mathbb{Z}_{2^k}$ need

a constant low space. This is analogously observed for the semi honest security type of the same model in Figure 5.4b

All Figures 5.5a to 5.6b depicting the influence of the CPU frequency show a similar effect. The weakest processor of the group of nodes that run this series of tests is rated with 2.1 GHz base frequency and 2.6 GHz turbo boost, below the final tested parameter value of 3.0 GHz. Every candidate showed a better performance with an increasing CPU frequency until 2.2 GHz. From there different scenarios can be observed. The strongest reaction has *spd2k* performing even worse than with the lowest frequency value. Most of the other protocols showed little to no improved runtime performance.

5.2 METRICS OF CPU PARAMETERS FOR REFERENCE SMC PROBLEMS

The promising protocols considering runtime performance have been filtered out and will be assessed with metrics of processor conditions. The main categories that influence a program's runtime performance are the number of cores available, the frequency in which the processor operates and the allowed percentage of CPU time it can consume. Additionally, for SMC programs, the networking environment of the nodes has an important influence, which will be examined in a different thesis that is currently being developed.

The respective nodes are noted, only naming the first of triple composite, for instance *valga* referring to *valga*, *rapla* and *tapa*. Refer table 2.1 with all the used nodes. The surface plots style follow the axis pattern x-axis: CPU test type (frequencies [GHz], quotas [%], threads [1]), y-axis: input size (number of input elements) and z-axis: runtime [s]

The protocol finalists are

- Scalable Search:
 - *Malicious, Dishonest*: *tinier*
 - *Semihonest, Dishonest*: *semi-bin*, *yao*
 - *Malicious, Honest*: *sy-rep-ring*
 - *Semihonest, Honest*: *replicated-ring*
- Matrix Integer Addition:
 - *Malicious, Dishonest*: *mascot*

- *Semihonest*, *Dishonest*: *semi*
- *Malicious*, *Honest*: *malicious-rep-field*
- *Semihonest*, *Honest*: *replicated-field*
- Matrix Fixed-point Addition:
 - *Malicious*, *Dishonest*: *lowgear*
 - *Semihonest*, *Dishonest*: *semi2k*
 - *Malicious*, *Honest*: *ps-rep-field*
 - *Semihonest*, *Honest*: *shamir*
- Matrix Multiplication:
 - *Malicious*, *Dishonest*: *spdz2k*
 - *Semihonest*, *Dishonest*: *temi*
 - *Malicious*, *Honest*: *sy-rep-field*
 - *Semihonest*, *Honest*: *replicated-bin*
- Set Intersection (underperforming candidates): The protocol selection for set intersection would be the same as for scalable search. Instead of them, bad performing candidates have been chosen purposely to study their characteristics.
 - *Malicious*, *Dishonest*: *spdz2k*
 - *Semihonest*, *Dishonest*: *temi*
 - *Malicious*, *Honest*: *mal-rep-bmr*
 - *Semihonest*, *Honest*: *rep-bmr*

5.2.1 FREQUENCY

5.2 METRICS OF CPU PARAMETERS FOR REFERENCE SMC PROBLEMS

SCALABLE SEARCH

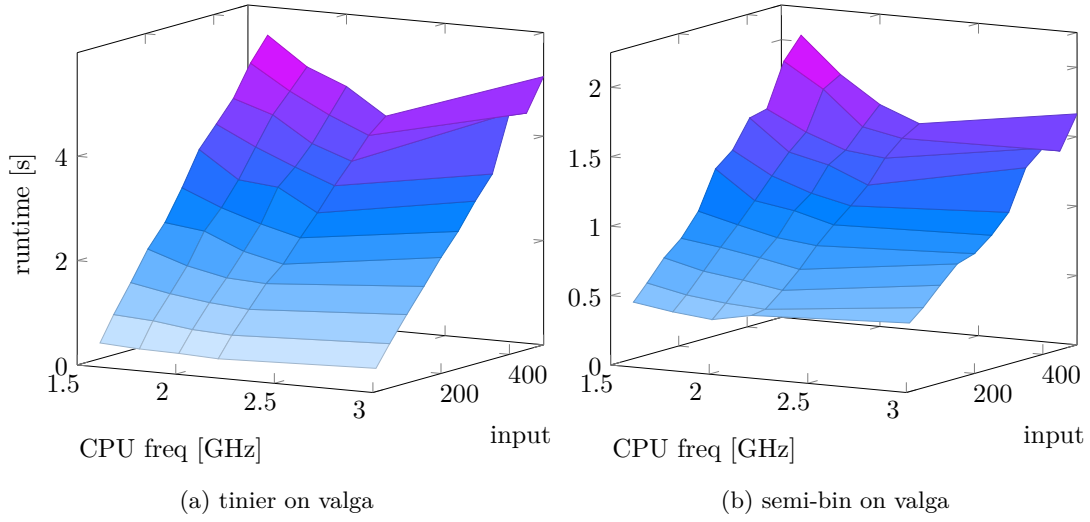


FIGURE 5.7: Dishonest majority protocol finalists reaction to CPU frequency change (Scalable Search)

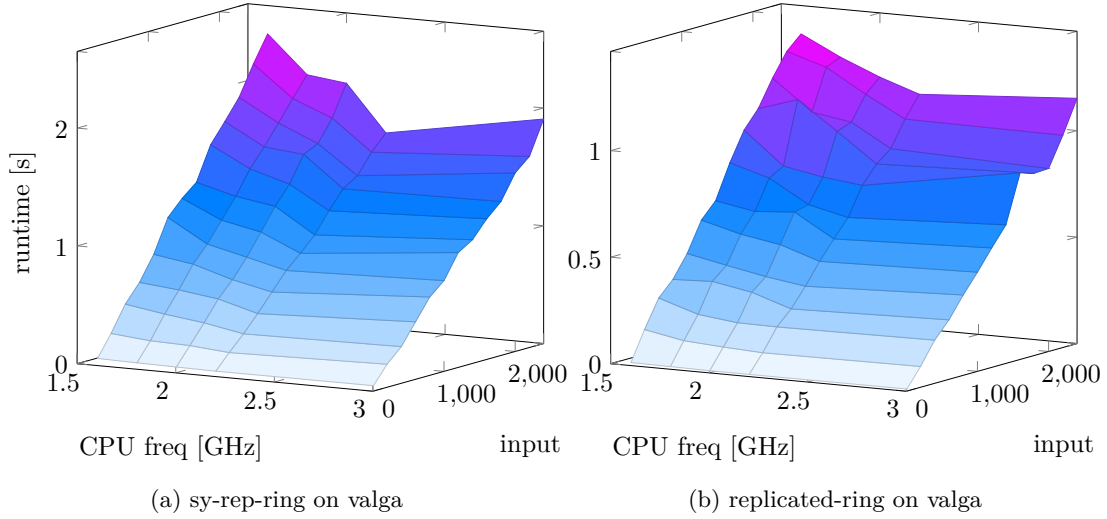


FIGURE 5.8: Honest majority protocol finalists reaction to CPU frequency change (Scalable Search)

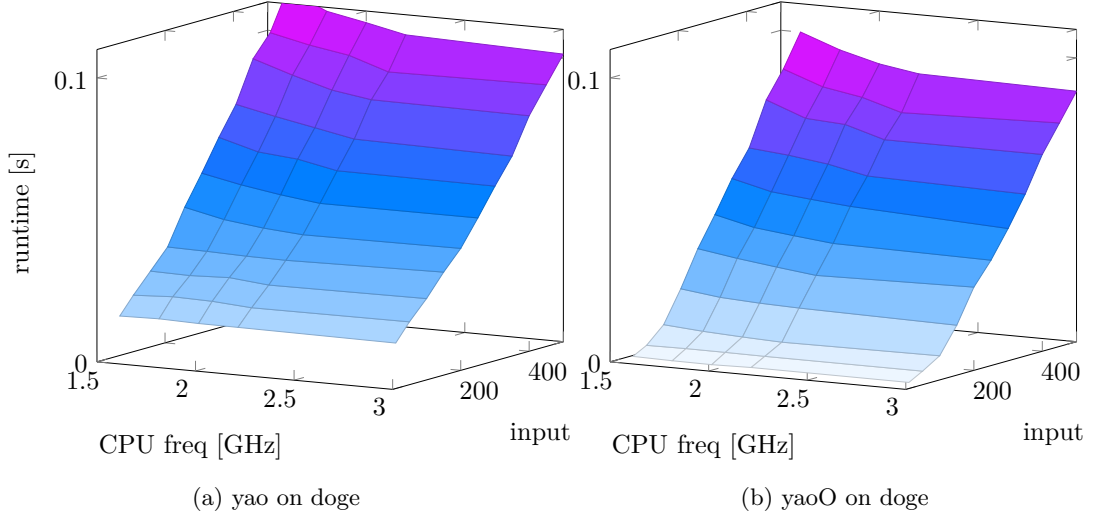


FIGURE 5.9: Garbled circuit protocol yaos reaction to CPU frequency change (Scalable Search)

The six protocol implementations show similar behavior with some artifacts that are present independently of the SMC category. For an increase in inputs, the effect of the frequency enforcement is becoming greater with an emerging exponential growth from values 2.2 GHz to the lowest value of 1.6 GHz, visibly in Figure 5.7 for input size 500. All graphs of the three-party case clearly show a minimum runtime line at 2.2 GHz while increasing again for 3 GHz. The higher the input size, the more visible this trend reversal gets. Also, some spikes are present for higher input sizes at 3 GHz. The two variants of the yao protocol in 5.9a and 5.9a do not show this behavior of higher runtimes at 3 GHz, not being affected by the frequency change as much as the others and performing good for input sizes up to 500. The yaoO version's measurement surface is overall shifted by approximately one millisecond down, implying a better performance for the tested values.

5.2.2 QUOTA

5.2 METRICS OF CPU PARAMETERS FOR REFERENCE SMC PROBLEMS

SCALABLE SEARCH

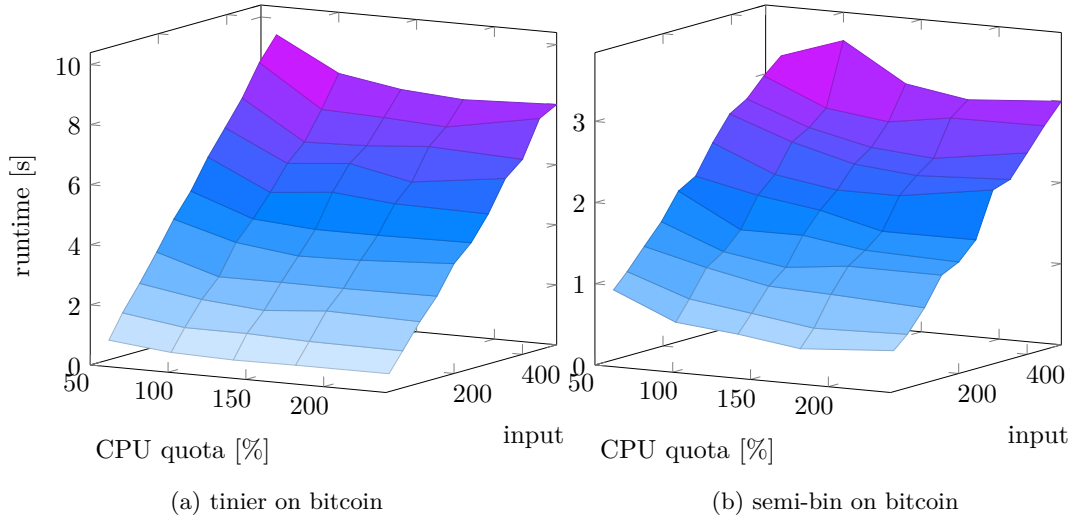


FIGURE 5.10: Dishonest majority protocol finalists reaction to CPU quota limit change (Scalable Search)

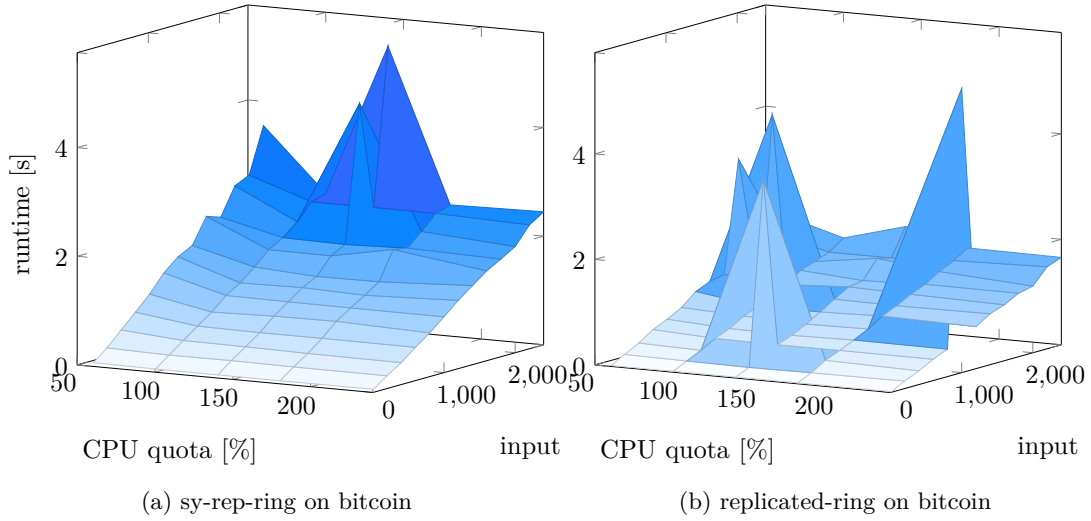


FIGURE 5.11: Honest majority protocol finalists reaction to CPU quota limit change (Scalable Search)

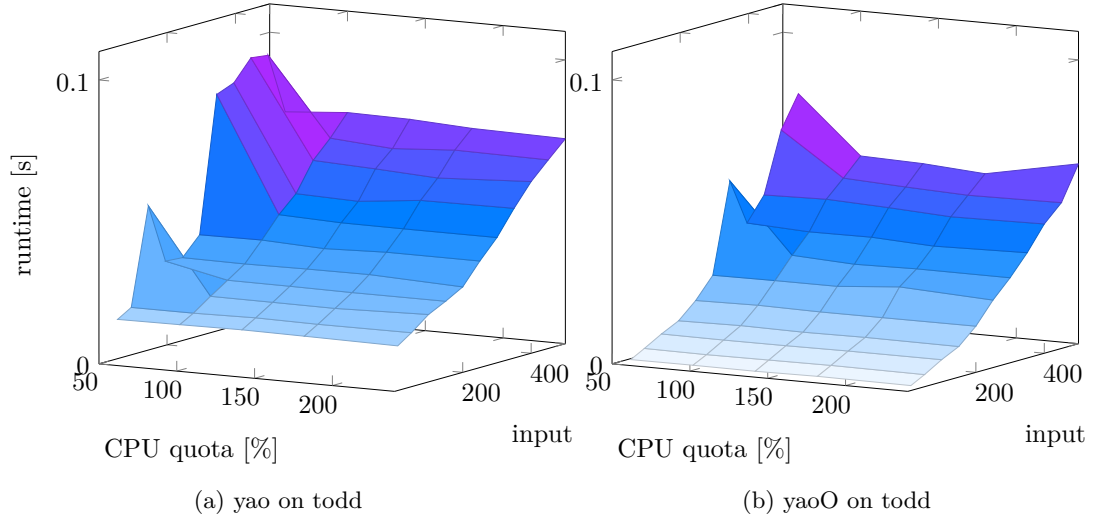


FIGURE 5.12: Garbled circuit protocol yaos reaction to CPU quota limit change (Scalable Search)

Figure 5.10 shows the surface plots of the dishonest majority protocols tinier for the malicious type on the left and semi-bin with semi-honest security on the right. The step from 60% limited total relative CPU time to 100% shows an increase in performance, especially for higher inputs. Random runtime peaks are observable, the most in Figure 5.11b. They cause the surface to look distorted even though it otherwise continues in a linear fashion. The surfaces from the yao protocols shown in Figure 5.12 have runtime peaks at a 60% quota limit but are not showing a clear trend of a greater influence by this test type.

5.2.3 CORES

5.2 METRICS OF CPU PARAMETERS FOR REFERENCE SMC PROBLEMS

SCALABLE SEARCH

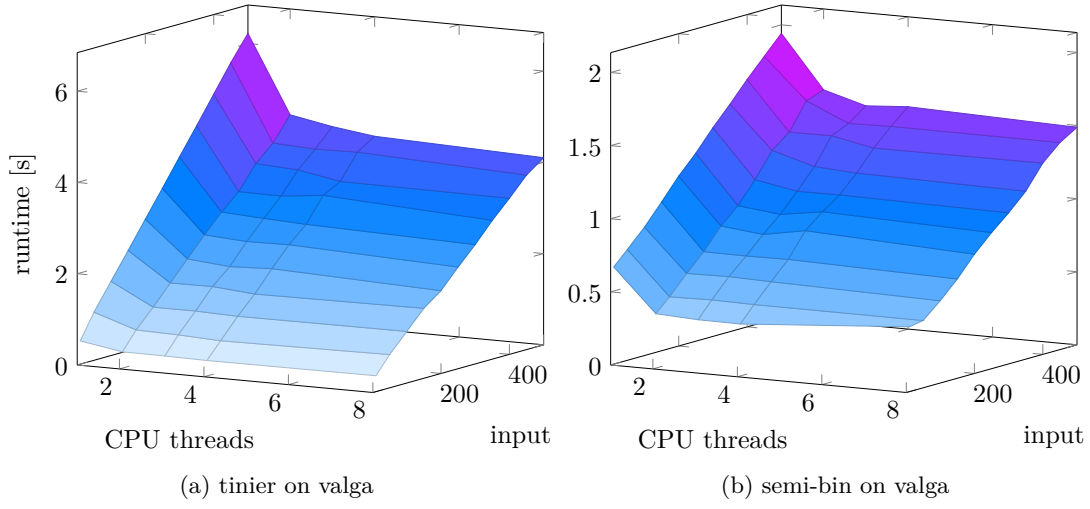


FIGURE 5.13: Dishonest majority protocol finalists reaction to available CPU thread number change (Scalable Search)

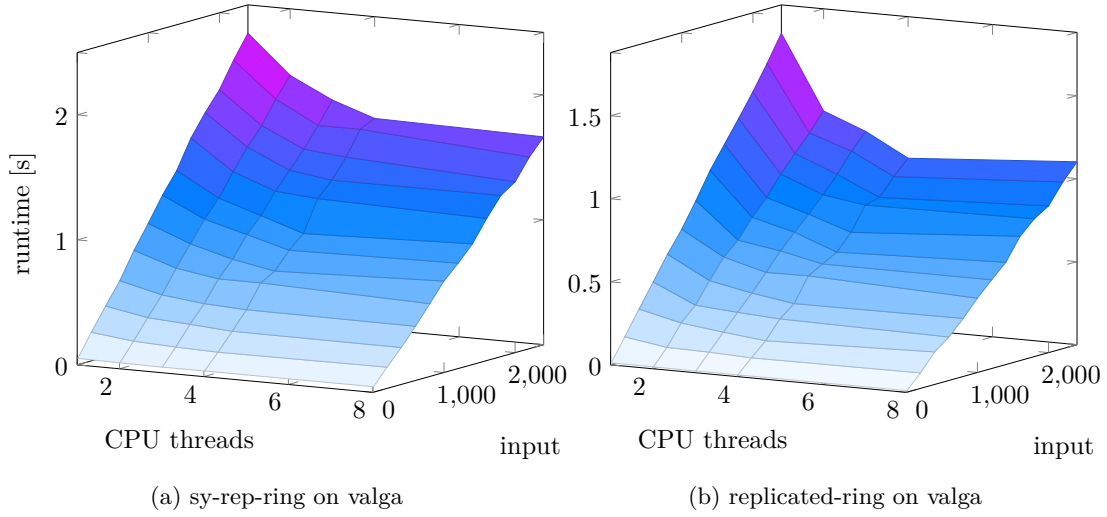


FIGURE 5.14: Honest majority protocol finalists reaction to available CPU thread number change (Scalable Search)

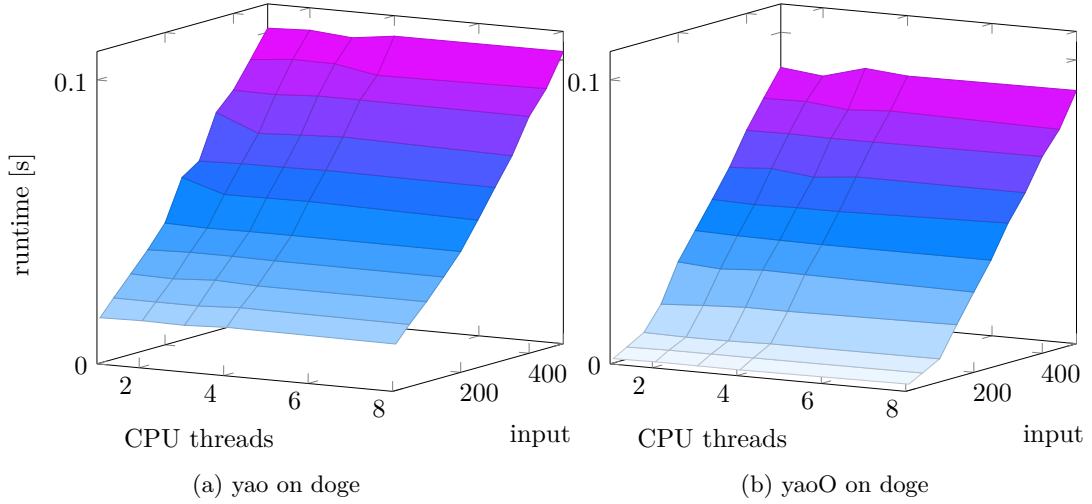


FIGURE 5.15: Garbled circuit protocol yaos reaction to available CPU thread number change (Scalable Search)

For protocol tinier on valga in Figure 5.13a, disabling all but one virtual CPU core leads to a significant increase in runtime compared to having two. Having more than two CPU cores shows no difference even for higher input values. With a growing number of inputs, the runtimes grow in a linear fashion, growing with a steeper slope with one core than with more. Semi-bin shows a similar behavior in Figure 5.13b with the difference of the slope at one core being parallel to the slopes at other core counts. Sy-rep-ring shows a benefit in having up to four cores at disposal, improving the runtimes in Figure 5.14a in three steps from one core to four. Each increased core count level has a more gentle slope. Having eight cores makes no more difference. Replicated-ring shows a steep drop from one core to two cores in Figure 5.14b in runtimes. The surface keeps dropping until four cores are available and it appears to grow back up at eight cores and input values above 1800. The yao protocol variants do not seem to be susceptible to a change in the available cores. The surfaces in Figure 5.15 are rising at all levels of cpu counts in a parallel manner.

5.3 ANALYZING THE MEASUREMENT RESULTS

Identifying the cause for every observed oddity is hardly possible. A key aspect to keep in mind and which has to be acknowledged are external influences. The testbed nodes, especially the coinbase hosts where many measurements took place, are connected via a switch with many other test nodes running experiments by other users. It even happened a few times that an entire test run aborted because of a pos timeout. Nodes connected to a switch that has other nodes linked and running MoonGen, a high-speed

packet generator that can saturate a 10 Gbit/s connection, are extremely constrained and do not offer laboratory-like conditions. Random measurement spikes as seen in Figure A.44b and A.30b can be attributed to this environmental handicap and offer an interesting glimpse into how real world experiment results could look like.

5.3.1 IMPLICATIONS OF OBSERVED BEHAVIOR

Initial Exclusion: SPDZ2k performs much better for arithmetic operations in contrast to the scalable search, leading the adversary model in runtime metrics. While runtimes are clearly growing for all protocols with an increase in inputs, the same cannot be observed for the peak RAM consumption. Some protocols start out with a high RAM demand, but stay stable across the measured input sizes, leading to a break even point. Figure 5.3b clearly shows that the protocols yao and yaoO from the domain garbled circuits start out with low RAM but with greater circuit sizes that have to be stored, eventually cross with the arithmetic domain protocols temi and semi2k at an input size of 500. Figure A.3b confirms this observation, which implies that protocols based on garbled circuits are preferred for scenarios with lower input sizes or little RAM restrictions. A general tendency of $\mathbb{F}_p$ and $\mathbb{Z}_{2^k}$ for being less RAM demanding even for higher input sizes is found.

Frequency: The observed growth in runtime in Figure 5.7 from its minimum at 2.2 GHz to 3 GHz is present independently of the SMC category. It can be therefore assumed that it is due to an environmental effect, most likely from the way the frequency modification was enforced. The used utility `cpupower frequency-set` offers two modes to set a specific value. Flag `-u` defines the maximum frequency value the processor governor may select, while `-f` sets a concrete value that the cores have to hold while under load. With `-u`, the CPU fluctuates between the lower and upper bounds, while for `-f` it very little change was observed. Most used processors are only supporting frequencies up to 2.2GHz, where we see the best runtimes in the graphs. Setting a higher value does not really make sense for models that have no boost frequency feature. With the exception of testnode valga's processors, which support turbo boost up to 3.1 Ghz, the other nodes are well below 3 GHz. The entire network of processors is constrained by the slowest processor in the group, so one fast CPU has no greater effect. For an interest in the behavior, it was also experimented with values beyond the theoretical limits, leading to this result where we can observe a decrease in performance. As mentioned above, values within the bounds of the processor's specifications, observed frequencies stayed stable, while with 3 GHz, random fluctuations within the limits are observed as if no frequency was set. This observation leads to the conclusion, that having set a legal frequency value for the , the governor actually enforces it, while a value out of the scope

makes the governor ignore that value and uses its default profile. The default profile might be more economical with the processor only scaling on demand and dynamically per thread. Summing up, the increase in runtime for an expectedly less constraining value can be attributed to little time it takes for the CPU to clock up and down in the dynamic setting, as opposed to staying in a high frequency.

The spikes in runtime, most noticeable for higher input sizes at 3 GHz in Figure 5.7, are assumed to originate from the pseudorandom functions nature of requiring a random amount of time to compute amplified by the dynamic frequency setting of the CPU governor.

Quota: The written programs were not optimized for a multithreaded execution. A keyword for constructing multithreaded loops exists in the high-level language of the MP-SPDZ compiler, but was not used to obtain more generalized results that are better comparable between the various protocols and across the domains and models. A 100% CPU quota permits a process to fully allocate a thread, theoretically the maximum a single threaded process would need. Yet some measurements, depicted in Figure A.39a and A.33a, that runtimes keep improving for protocols `spd2k` and `mascot` until flattening at 200%. On the other hand, Figure A.39b clearly shows that protocol `temi` does not profit from a CPU quota higher than 100%. These deviating observations imply that some protocol implementations can profit from having access to more than one thread, even for unoptimized programs. At 200%, all the surfaces presenting the CPU quota effect show a minimum in runtime not dropping further in efficiency. For the scalable search program, the protocol `tinier` and `semi-bin` show in Figure 5.10 a drop in runtime for higher input from the measurement step 100% to 140%. During a protocol run, the protocol process on the test servers is not statically allocated to a specific virtual core, but appears to jump between the available CPU threads, allocating 50% of two cores instead of 100% of one for a quota setting of 100. These fluctuations are higher for lower quota settings, causing an overhead in computation for the switching of the processes between the cores. The observed behavior could stem from this effect, implying that a 100% CPU quota is not as efficient as a concretely allocated core. There are more random runtime spikes observable in the quota measurements than in the surfaces of the other test types. We believe the way the quota limiting method works is putting an additional strain on the process run. Without specifying a CPU Weight, a priority setting defining the use of host resources, the default setting could be lower than running a program without encapsulation in `systemd.run`.

Cores: The protocol's reaction to a host environment with an overall limited count of virtual cores reflects to some degree the observations made with capping the CPU quota.

5.3 ANALYZING THE MEASUREMENT RESULTS

That some protocols show greatly improved runtimes with the step from a single core to two cores supports the hypothesis that internally these implementations use some kind of concurrency technique. The surface plot of protocol implementation `spd2k` behaves notably differently in Figure A.45a. While dropping in runtime from one core when adding a second the same way as `mascot` does in Figure A.41a, measurements resulted in worse runtimes at a core count of eight for every input value. This and `rep-bmrs` behavior is currently inexplicable. The malicious and semi-honest `rep-bmr` implementations similarly show much steeper slopes for the eight core parameter value at every input value in Figure A.48.

5.3.2 COMPARING PROTOCOLS OF A COMPUTATIONAL DOMAIN

Having determined the appropriate computation domain, even though the least applicable adversary model should have been decided upon in the research of the participants behavior, analyzing the performance of protocols in an even stricter security type and more pessimistic adversarial constellation can help identify cheap or free security. Furthermore, it could be of interest what the price of security is for various metrics beyond runtime, like memory consumption and energy requirement. The results of the experiments clearly show that the best performing candidate of a more relaxed security type outperforms stricter protocol implementations. The price factor becomes greater with every step into a higher security and majority type. The input sizes for the measurements series are kept the same within a majority type, but have a factor of approximately 10 distance between them. For the set intersection problem, for input size 5^2 , the best candidate of the lowest security adversary model semi-honest, honest majority is replicated-ring with a measured runtime of 0.065 seconds. The malicious version of this protocol is also the best for the adversary model malicious, honest majority, having a runtime of 0.175 seconds. This is a factor of 2.7 increase in runtime costs. The best protocol in the next higher security type, semi-honest, dishonest majority, is semi-bin with 0.694 seconds at input size 5^2 , which is a fourfold increase to this level. Tinier is the fastest of the protocols with the highest security promise and has a runtime of 0.902 seconds. In this case the extra security only costs 30% more in runtime. The factor between the lowest and the highest levels is 13.9, which is expected to increase with growing input sizes.

CHAPTER 6

CONCLUSION

Every protocol has its own characteristics. This becomes apparent when observing how for example tinier strongly outperforms other candidates of the same SMC category in search problems like the scalable search or set intersection, but on the other hand having the highest runtime time for arithmetic operations. SPDZ2k performs badly for scalable search, where a lot of comparisons are evaluated, while performing very well in arithmetic operations like the matrix addition. It is not enough to only consider a single factor, namely runtime, when looking for a suitable protocol. While there are seemingly little differences on the runtime charts like in Figure A.1a, and choosing mascot for being the fastest, even by a fraction, would be a sound choice, taking a look at the RAM requirement in Figure A.3a reveals that spdz2k can be more fitting with better RAM consumption albeit little longer runtimes.

In this thesis, default implementations of SMC protocols were compared with each other, resulting in an interesting but limited view. All reasoning is based on the MP-SPDZ specific version of an algorithm, where some might be better optimized and the implementations more thought through than others. It has a huge impact on performance, if an SMC protocol is implemented to have to regenerate random values each run instead of reusing them. An example shows lowgear in 5.1a, with this effect becoming visible as a stairway pattern. Many of the protocols that showed a high performance are only applicable for the three party case, again optimized for this specific scenario, while others allow more and require a more universal and complex implementation.

Protocols specializing on the 3-party case, in particular the rep3 implementations like replicated-field and replicated-ring prove with their observed performance, that optimizations can be made in various spots the more is known about the intended purpose.

Binary protocols need more RAM and grow in space for rising input values, while field and ring protocols stay stable.

No optimizations from the side of MP-SPDZ were made, but many protocols offer a variety of settings to enhance the runtimes.

6.1 CONTRIBUTIONS

Looking at table 3.1 with all the names of various SMC implementations, the characteristics of each individual are difficult to devise apart from the designers of the protocols. What to expect from a protocol’s performance, beyond being able to theoretically sort out the computation domain like assigning problems consisting of arithmetic integer operations to arithmetic protocols (modulo prime or power of two), important aspects involving real world applicability must be benchmarked beforehand. With the developed tool *sevarebench*, this benchmarking process is greatly reduced in complexity, allowing for a wide spectrum of test cases and in depth analysis. It was used to show how state-of-the-art SMC protocols perform for different reference use cases, proving that specific protocol types perform better for specific use cases. It helped to measure the cost of security, when considering the choices between the adversary model, and identify the most suitable computation domain for a given reference problem. The embedded testtypes for analyzing the performance behavior of changes on the host environment help assess SMC protocols from the angle of discovering bottlenecks and hardware requirements. It was shown for instance, that most protocols when run without keywords to activate multi threading do not profit from more CPU cores than four. Most of the implications showed that a multi-core environment has less impact on performance than fewer cores that are rated with a higher base frequency.

6.2 FUTURE WORK

MP-SPDZ offers mixed domain runs that can help in developing compound solutions. The application of SMC in more complex scenarios benefits from splitting up the computations to protocols from domains that are best suited for a specific part of a problem. SMC is strongly network dependent which requires a software developer of SMC protocols to know and benchmark the influence of network parameters. In this Bachelor’s Thesis, only host parameters observed under near ideal network conditions are assessed. Network parameters may be more important for different scenarios where the best performing SMC candidates in other categories than runtime and RAM usage, specifically energy consumption, data volume, and peer drop resistance are sought. If the fastest

protocol needs to transfer large datasets between the participants, it may not be applicable for a composition of nodes with some links having a high latency and low bandwidth. The three-party case is the most supported among the MP-SPDZ collection, but many protocols support more participants. The assessment of the influence of a growing group size on parameters such as RAM and runtime is an important future work, supporting the development of SMC protocols applicable for more real world scenarios. Sevarbench is a foundation that allows for further inclusion of various new extensions. For the depicted runtime results, the internal time measurement interface preshipped with the SMC protocols of MP-SPDZ was evaluated, but measuring the elapsed time externally showed a discrepancy between the results. The resulting tables will be extended with a new column for external runtime and CPU time measurements, increasing the verbosity of the experiment results.

CHAPTER A

APPENDIX

A.1 MEASUREMENTS FOR CHAPTER 5.1

A.1.1 MEASUREMENT TABLES FOR SECTION 5.1.1

INTEGER ADDITION

LISTING A.1: Config file for initial run of Experiment 32 matrix integer addition

```
1 experiments=32_matrix_add
2 # protocols
3 all
4 input=8,24
5 measureram
```

SMC category		Element count:	8^2	24^2	8^2	24^2
Adv. model	C. Domain	Protocol	Runtime [s]		RAM [GiB]	
MalDishonest	$\mathbb{F}_p$	mascot	0.99	1.07	0.25	0.25
MalDishonest	$\mathbb{F}_p$	lowgear	506.57	1.09	1.21	0.06
MalDishonest	$\mathbb{F}_p$	highgear	1,169.76	24.07	2.56	0.26
MalDishonest	$\mathbb{Z}_{2^k}$	SPDZ2k	0.88	0.98	0.16	0.16
MalDishonest	$\mathbb{Z}_2$ SS	tinier	2.34	17.45	0.09	0.10
MalDishonest	$\mathbb{Z}_2$ GC	real-bmr	128.25	1,142.80	0.72	3.55
CovertDishonest	$\mathbb{F}_p$	cowgear	6.76	1.08	0.13	0.06
CovertDishonest	$\mathbb{F}_p$	chaigear	70.72	24.11	1.22	0.26
SemiDishonest	$\mathbb{F}_p$	semi	0.01	0.06	0.01	0.01
SemiDishonest	$\mathbb{F}_p$	hemi	0.01	0.06	0.02	0.02
SemiDishonest	$\mathbb{F}_p$	temi	0.01	0.06	0.03	0.03
SemiDishonest	$\mathbb{F}_p$	soho	0.01	0.06	0.12	0.10
SemiDishonest	$\mathbb{Z}_{2^k}$	semi2k	0.01	0.06	0.01	0.01
SemiDishonest	$\mathbb{Z}_2$ SS	semi-bin	0.86	5.68	0.19	0.20
SemiDishonest	$\mathbb{Z}_2$ GC	yao	0.07	0.56	0.01	0.03
SemiDishonest	$\mathbb{Z}_2$ GC	yao -O	0.02	0.26	0.01	0.04
SemiDishonest	$\mathbb{Z}_2$ GC	semi-bmr	16.81	150.49	0.22	1.44
MalHonest	$\mathbb{F}_p$	malicious-shamir	0.01	0.05	0.02	0.01
MalHonest	$\mathbb{F}_p$	malicious-rep-field	0.01	0.05	0.01	0.01
MalHonest	$\mathbb{F}_p$	ps-rep-field	0.02	0.04	0.01	0.02
MalHonest	$\mathbb{F}_p$	sy-rep-field	0.01	0.04	0.01	0.02
MalHonest	$\mathbb{Z}_{2^k}$	brain	0.01	0.03	0.01	0.01
MalHonest	$\mathbb{Z}_{2^k}$	malicious-rep-ring	0.01	0.03	0.01	0.02
MalHonest	$\mathbb{Z}_{2^k}$	ps-rep-ring	0.01	0.04	0.01	0.01
MalHonest	$\mathbb{Z}_{2^k}$	sy-rep-ring	0.01	0.05	0.02	0.02
MalHonest	$\mathbb{Z}_2$ SS	malicious-rep-bin	0.56	5.05	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	malicious-ccd	0.86	8.06	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	ps-rep-bin	2.23	8.33	0.40	0.37
MalHonest	$\mathbb{Z}_2$ GC	mal-shamir-bmr	3.37	27.61	0.20	1.55
MalHonest	$\mathbb{Z}_2$ GC	mal-rep-bmr	2.38	17.71	0.23	1.85
SemiHonest	$\mathbb{F}_p$	shamir	0.01	0.09	0.01	0.01
SemiHonest	$\mathbb{F}_p$	atlas	0.04	0.14	0.02	0.02
SemiHonest	$\mathbb{F}_p$	replicated-field	0.01	0.04	0.01	0.01
SemiHonest	$\mathbb{Z}_{2^k}$	replicated-ring	<0.01	0.04	0.01	0.01
SemiHonest	$\mathbb{Z}_2$ SS	replicated-bin	0.40	4.81	0.01	0.02
SemiHonest	$\mathbb{Z}_2$ SS	ccd	0.64	6.20	0.01	0.03
SemiHonest	$\mathbb{Z}_2$ GC	shamir-bmr	1.85	14.93	0.17	1.32
SemiHonest	$\mathbb{Z}_2$ GC	rep-bmr	1.17	9.23	0.13	1.01

TABLE A.1: Experiment 32 matrix integer addition preliminary results (Nodes: todd, ned and rod)

SMC category		
Adv. model	C. Domain	Exclusions and reasoning
MalDishonest	$\mathbb{F}_p$	LowGear is a little slower but requires considerably less RAM than MASCOT, keep both, HighGear is outperformed this BMR version explodes in runtime and is therefore infeasible
MalDishonest	$\mathbb{Z}_2$ GC	
CovDishonest	$\mathbb{F}_p$	CowGear and ChaiGear run a little slower than stricter security types, omit for free security
SemiDishonest	$\mathbb{F}_p$	All deliver similar results with semi having the smallest RAM reservation, unselect soho and temi for worse memory consumption unselect semi-bmr for being too expensive in runtime and RAM, Yao performs well for the 2-party case, with only half the operations evaluated
SemiDishonest	$\mathbb{Z}_2$ GC	
MalHonest	<i>modulo</i> 2^k	PS and SY are unselected for being slower than Brain and Rep3 PS performs the worst of this group, omit the mal-shamir version and mal-rep version are deemed ineffective with much higher runtimes than protocols from other domains and with better security
MalHonest	$\mathbb{Z}_2$ SS	
MalHonest	$\mathbb{Z}_2$ GC	
SemiHonest	$\mathbb{F}_p$	ATLAS is outperformed by Shamir and Rep3 in observed parameters CCD shows a worse performance than Rep3 both BMR versions, shamir and replicated, perform bad for all observed parameters
SemiHonest	$\mathbb{Z}_2$ SS	
SemiHonest	$\mathbb{Z}_2$ GC	

TABLE A.2: Experiment 32 matrix integer addition preliminary results interpretation

FIXED-POINT ADDITION

LISTING A.2: Config file for initial run of Experiment 33 matrix fixed-point addition

```

1  experiments=33_matrix_add_float
2  # protocols
3  all
4  input=8,24
5  measureram

```

SMC category		Element count:	8^2	24^2	8^2	24^2
Adv. model	C. Domain	Protocol	Runtime [s]		RAM [GiB]	
MalDishonest	$\mathbb{F}_p$	mascot	0.65	0.77	0.08	0.08
MalDishonest	$\mathbb{F}_p$	lowgear	357.26	0.52	0.88	0.04
MalDishonest	$\mathbb{F}_p$	highgear	864.99	7.95	1.92	0.13
MalDishonest	$\mathbb{Z}_{2^k}$	SPDZ2k	1.27	1.37	0.16	0.16
MalDishonest	$\mathbb{Z}_2$ SS	tinier	2.04	15.30	0.08	0.10
MalDishonest	$\mathbb{Z}_2$ GC	real-bmr	202.44	2,003.74	0.53	1.89
CovertDishonest	$\mathbb{F}_p$	cowgear	2.16	0.47	0.06	0.04
CovertDishonest	$\mathbb{F}_p$	chaigear	22.49	7.98	0.62	0.13
SemiDishonest	$\mathbb{F}_p$	semi	0.01	0.10	0.01	0.01
SemiDishonest	$\mathbb{F}_p$	hemi	0.01	0.11	0.01	0.01
SemiDishonest	$\mathbb{F}_p$	temi	0.01	0.11	0.02	0.02
SemiDishonest	$\mathbb{F}_p$	soho	0.01	0.09	0.06	0.05
SemiDishonest	$\mathbb{Z}_{2^k}$	semi2k	0.01	0.10	0.01	0.01
SemiDishonest	$\mathbb{Z}_2$ SS	semi-bin	0.96	5.62	0.19	0.20
SemiDishonest	$\mathbb{Z}_2$ GC	yao	0.05	0.36	0.01	0.03
SemiDishonest	$\mathbb{Z}_2$ GC	yao -O	0.01	0.18	0.01	0.03
SemiDishonest	$\mathbb{Z}_2$ GC	semi-bmr	32.84	293.81	0.14	0.72
MalHonest	$\mathbb{F}_p$	malicious-shamir	0.01	0.07	0.02	0.01
MalHonest	$\mathbb{F}_p$	malicious-rep-field	0.01	0.07	0.01	0.02
MalHonest	$\mathbb{F}_p$	ps-rep-field	0.01	0.07	0.01	0.01
MalHonest	$\mathbb{F}_p$	sy-rep-field	0.01	0.07	0.01	0.01
MalHonest	$\mathbb{Z}_{2^k}$	brain	0.01	0.07	0.01	0.01
MalHonest	$\mathbb{Z}_{2^k}$	malicious-rep-ring	0.01	0.08	0.01	0.01
MalHonest	$\mathbb{Z}_{2^k}$	ps-rep-ring	0.01	0.06	0.01	0.01
MalHonest	$\mathbb{Z}_{2^k}$	sy-rep-ring	0.01	0.07	0.02	0.02
MalHonest	$\mathbb{Z}_2$ SS	malicious-rep-bin	0.41	3.43	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	malicious-ccd	0.52	4.72	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	ps-rep-bin	2.25	5.85	0.40	0.37
MalHonest	$\mathbb{Z}_2$ GC	mal-shamir-bmr	2.65	21.56	0.10	0.77
MalHonest	$\mathbb{Z}_2$ GC	mal-rep-bmr	1.29	10.75	0.12	0.90
SemiHonest	$\mathbb{F}_p$	shamir	0.02	0.14	0.01	0.01
SemiHonest	$\mathbb{F}_p$	atlas	0.04	0.16	0.01	0.01
SemiHonest	$\mathbb{F}_p$	replicated-field	0.01	0.06	0.01	0.01
SemiHonest	$\mathbb{Z}_{2^k}$	replicated-ring	0.01	0.06	0.01	0.01
SemiHonest	$\mathbb{Z}_2$ SS	replicated-bin	0.41	3.49	0.01	0.02
SemiHonest	$\mathbb{Z}_2$ SS	ccd	0.45	3.98	0.01	0.02
SemiHonest	$\mathbb{Z}_2$ GC	shamir-bmr	1.09	10.80	0.08	0.65
SemiHonest	$\mathbb{Z}_2$ GC	rep-bmr	0.64	6.34	0.07	0.50

TABLE A.3: Experiment 33 matrix fixed-point addition preliminary results (Nodes: dogecoin, dogecoin-cash and dogecoin-gold)

SMC category		Exclusions and reasoning
Adv. model	C. Domain	
MalDishonest	$\mathbb{F}_p$	LowGear is a little slower but requires considerably less RAM than MASCOT, keep both, HighGear is outperformed the BMR version with the highest security delivers bad performance again
MalDishonest	$\mathbb{Z}_2$ GC	
CovDishonest	$\mathbb{F}_p$	ChaiGear runs slower than stricter security types, omit for free security, textbfCowGear is a little faster than mascot but less secure
SemiDishonest	$\mathbb{F}_p$	All deliver similar results with semi having the smallest RAM reservation, unselect soho and temi for worse memory consumption unselect semi-bmr for being underperforming, Yao performs well for the 2-party case, with only half the operations evaluated
SemiDishonest	$\mathbb{Z}_2$ GC	
MalHonest	<i>modulo</i> 2^k	Rep3 is unselected for being slower than Brain, SY, and PS PS performs the worst of this group, omit the mal-shamir version and mal-rep version are deemed ineffective with much higher runtimes than protocols from other domains and with better security
MalHonest	$\mathbb{Z}_2$ SS	
MalHonest	$\mathbb{Z}_2$ GC	
SemiHonest	$\mathbb{F}_p$	ATLAS is outperformed by Shamir and Rep3 in observed parameters CCD shows a worse performance than Rep3 both BMR versions, shamir and replicated , perform bad for all observed parameters
SemiHonest	$\mathbb{Z}_2$ SS	
SemiHonest	$\mathbb{Z}_2$ GC	

TABLE A.4: Experiment 33 matrix fixed-point addition preliminary results interpretation

INTEGER MULTIPLICATION

LISTING A.3: Config file for initial run of Experiment 34 matrix integer multiplication

```

1  experiments=34_matrix_mul
2  # protocols
3  all
4  input=8,24
5  measureram

```

SMC category		Element count:	8^2	24^2	8^2	24^2
Adv. model	C. Domain	Protocol	Runtime [s]		RAM [GiB]	
MalDishonest	$\mathbb{F}_p$	mascot	2.81	4.05	0.35	0.38
MalDishonest	$\mathbb{F}_p$	lowgear	1,283.73	9.85	1.23	0.11
MalDishonest	$\mathbb{F}_p$	highgear	3,118.44	32.78	2.57	0.28
MalDishonest	$\mathbb{Z}_{2^k}$	SPDZ2k	2.16	3.49	0.32	0.34
MalDishonest	$\mathbb{Z}_2$ SS	tinier	49.49	449.66	0.09	0.11
MalDishonest	$\mathbb{Z}_2$ GC	real-bmr	10,177.90	-	5.04	-
CovertDishonest	$\mathbb{F}_p$	cowgear	15.62	9.78	0.16	0.11
CovertDishonest	$\mathbb{F}_p$	chaigear	82.30	32.73	1.22	0.28
SemiDishonest	$\mathbb{F}_p$	semi	0.21	0.76	0.06	0.06
SemiDishonest	$\mathbb{F}_p$	hemi	0.52	0.83	0.04	0.04
SemiDishonest	$\mathbb{F}_p$	temi	0.34	0.72	0.04	0.03
SemiDishonest	$\mathbb{F}_p$	soho	1.17	1.52	0.16	0.11
SemiDishonest	$\mathbb{Z}_{2^k}$	semi2k	0.10	0.56	0.03	0.03
SemiDishonest	$\mathbb{Z}_2$ SS	semi-bin	2.82	24.04	0.20	0.21
SemiDishonest	$\mathbb{Z}_2$ GC	yao	0.52	5.30	0.01	0.04
SemiDishonest	$\mathbb{Z}_2$ GC	yao -O	0.47	3.43	0.04	0.08
SemiDishonest	$\mathbb{Z}_2$ GC	semi-bmr	1,562.84	12,564.90	2.05	2.27
MalHonest	$\mathbb{F}_p$	malicious-shamir	0.09	0.40	0.02	0.02
MalHonest	$\mathbb{F}_p$	malicious-rep-field	0.06	0.28	0.02	0.02
MalHonest	$\mathbb{F}_p$	ps-rep-field	0.03	0.28	0.01	0.01
MalHonest	$\mathbb{F}_p$	sy-rep-field	0.03	0.26	0.02	0.02
MalHonest	$\mathbb{Z}_{2^k}$	brain	0.11	0.35	0.02	0.02
MalHonest	$\mathbb{Z}_{2^k}$	malicious-rep-ring	0.04	0.30	0.02	0.02
MalHonest	$\mathbb{Z}_{2^k}$	ps-rep-ring	0.03	0.26	0.01	0.01
MalHonest	$\mathbb{Z}_{2^k}$	sy-rep-ring	0.03	0.25	0.02	0.02
MalHonest	$\mathbb{Z}_2$ SS	malicious-rep-bin	1.57	15.59	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	malicious-ccd	6.74	52.42	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	ps-rep-bin	6.30	36.75	0.36	0.37
MalHonest	$\mathbb{Z}_2$ GC	mal-shamir-bmr	94.18	851.26	2.20	2.51
MalHonest	$\mathbb{Z}_2$ GC	mal-rep-bmr	39.22	357.85	2.68	3.00
SemiHonest	$\mathbb{F}_p$	shamir	0.04	0.34	0.01	0.01
SemiHonest	$\mathbb{F}_p$	atlas	0.12	0.98	0.02	0.02
SemiHonest	$\mathbb{F}_p$	replicated-field	0.03	0.28	0.01	0.01
SemiHonest	$\mathbb{Z}_{2^k}$	replicated-ring	0.03	0.24	0.01	0.01
SemiHonest	$\mathbb{Z}_2$ SS	replicated-bin	1.02	9.63	0.01	0.02
SemiHonest	$\mathbb{Z}_2$ SS	ccd	1.44	13.28	0.01	0.03
SemiHonest	$\mathbb{Z}_2$ GC	shamir-bmr	20.00	179.37	1.87	2.17
SemiHonest	$\mathbb{Z}_2$ GC	rep-bmr	10.77	95.62	1.40	1.67

TABLE A.5: Experiment 34 matrix integer multiplication preliminary results (Nodes: bitcoin, bitcoin-cash and bitcoingold)

SMC category		
Adv. model	C. Domain	Exclusions and reasoning
MalDishonest	$\mathbb{F}_p$	LowGear is faster but and requires considerably less RAM than MAS-COT after initialization, keep both, HighGear is outperformed this BMR is infeasible for being slow and RAM expensive, the run with input size 24^2 had to be aborted
MalDishonest	$\mathbb{Z}_2$ GC	
CovDishonest	$\mathbb{F}_p$	CowGear and ChaiGear run slower than stricter security types, omit for free security
SemiDishonest	$\mathbb{F}_p$	soho and hemi have a relatively bad runtime and RAM consumption unselect semi-bmr for being too expensive in runtime and RAM, Yao performs well for the 2-party case, with only half the operations evaluated
SemiDishonest	$\mathbb{Z}_2$ GC	
MalHonest	$\mathbb{F}_p$	shamir can be omitted in favor of the other candidates brain and rep are unselected for being slower than PS and SY PS and CCD perform the worse than rep-bin, omit the mal-shamir version and mal-rep version are deemed ineffective with much higher runtimes than protocols from other domains and with better security
MalHonest	<i>modulo</i> 2^k	
MalHonest	$\mathbb{Z}_2$ SS	
MalHonest	$\mathbb{Z}_2$ GC	
SemiHonest	$\mathbb{F}_p$	ATLAS is outperformed by Shamir and Rep3 in observed parameters
SemiHonest	$\mathbb{Z}_2$ SS	CCD shows a worse performance than Rep3
SemiHonest	$\mathbb{Z}_2$ GC	both BMR versions, shamir and replicated , perform bad for all observed parameters

TABLE A.6: Experiment 34 matrix integer multiplication preliminary results interpretation

SET INTERSECTION

LISTING A.4: Config file for initial run of Experiment 35 set intersection

```

1  experiments=35_set_intersection
2  # protocols
3  all
4  input=6,36
5  measureram

```

SMC category		Element count:	6	36	6	36
Adv. model	C. Domain	Protocol	Runtime [s]		RAM [GiB]	
MalDishonest	$\mathbb{F}_p$	mascot	14.98	430.99	0.38	0.41
MalDishonest	$\mathbb{F}_p$	lowgear	1,449.45	163.14	1.22	0.15
MalDishonest	$\mathbb{F}_p$	highgear	-	-	-	-
MalDishonest	$\mathbb{Z}_{2^k}$	SPDZ2k	12.77	361.45	0.80	0.83
MalDishonest	$\mathbb{Z}_2$ SS	tinier	1.56	23.36	0.09	0.08
MalDishonest	$\mathbb{Z}_2$ GC	real-bmr	-	-	-	-
CovertDishonest	$\mathbb{F}_p$	cowgear	15.97	162.33	0.16	0.15
CovertDishonest	$\mathbb{F}_p$	chaigear	90.02	256.98	1.23	0.32
SemiDishonest	$\mathbb{F}_p$	semi	3.26	98.32	0.06	0.06
SemiDishonest	$\mathbb{F}_p$	hemi	1.32	34.20	0.04	0.05
SemiDishonest	$\mathbb{F}_p$	temi	1.01	27.73	0.04	0.03
SemiDishonest	$\mathbb{F}_p$	soho	2.44	40.28	0.16	0.11
SemiDishonest	$\mathbb{Z}_{2^k}$	semi2k	0.92	29.98	0.03	0.03
SemiDishonest	$\mathbb{Z}_2$ SS	semi-bin	0.62	4.34	0.19	0.19
SemiDishonest	$\mathbb{Z}_2$ GC	yao	0.03	0.28	0.01	0.01
SemiDishonest	$\mathbb{Z}_2$ GC	yao -O	<0.01	0.20	0.01	0.02
SemiDishonest	$\mathbb{Z}_2$ GC	semi-bmr	22.32	571.55	0.12	1.40
MalHonest	$\mathbb{F}_p$	malicious-shamir	0.21	7.23	0.02	0.02
MalHonest	$\mathbb{F}_p$	malicious-rep-field	0.19	5.39	0.02	0.02
MalHonest	$\mathbb{F}_p$	ps-rep-field	0.14	5.26	0.02	0.02
MalHonest	$\mathbb{F}_p$	sy-rep-field	0.19	7.46	0.02	0.03
MalHonest	$\mathbb{Z}_{2^k}$	brain	0.32	7.29	0.03	0.03
MalHonest	$\mathbb{Z}_{2^k}$	malicious-rep-ring	0.13	4.74	0.02	0.02
MalHonest	$\mathbb{Z}_{2^k}$	ps-rep-ring	0.14	4.27	0.02	0.02
MalHonest	$\mathbb{Z}_{2^k}$	sy-rep-ring	0.16	4.37	0.02	0.03
MalHonest	$\mathbb{Z}_2$ SS	malicious-rep-bin	0.13	2.86	0.02	0.02
MalHonest	$\mathbb{Z}_2$ SS	malicious-ccd	0.40	4.75	0.01	0.01
MalHonest	$\mathbb{Z}_2$ SS	ps-rep-bin	1.94	5.78	0.36	0.40
MalHonest	$\mathbb{Z}_2$ GC	mal-shamir-bmr	1.61	42.13	0.08	1.49
MalHonest	$\mathbb{Z}_2$ GC	mal-rep-bmr	0.77	16.82	0.09	1.81
SemiHonest	$\mathbb{F}_p$	shamir	0.12	4.69	0.02	0.02
SemiHonest	$\mathbb{F}_p$	atlas	4.31	14.46	0.02	0.02
SemiHonest	$\mathbb{F}_p$	replicated-field	0.10	6.49	0.01	0.01
SemiHonest	$\mathbb{Z}_{2^k}$	replicated-ring	0.08	2.73	0.02	0.01
SemiHonest	$\mathbb{Z}_2$ SS	replicated-bin	0.09	2.50	0.01	0.01
SemiHonest	$\mathbb{Z}_2$ SS	ccd	0.10	2.95	0.01	0.01
SemiHonest	$\mathbb{Z}_2$ GC	shamir-bmr	0.51	8.99	0.07	1.26
SemiHonest	$\mathbb{Z}_2$ GC	rep-bmr	0.28	5.32	0.05	0.95

TABLE A.7: Experiment 35 set intersection preliminary results (Nodes: bitcoin, bitcoincash and bitcoin-gold)

A.1 MEASUREMENTS FOR CHAPTER 5.1

SMC category		Exclusions and reasoning
Adv. model	C. Domain	
MalDishonest	$\mathbb{F}_p$	HighGear measurement was aborted for taking too long, after pre-generation, lowgear showed good performance
MalDishonest	$\mathbb{Z}_2$ GC	BMR measurement was aborted for taking too long
CovDishonest	$\mathbb{F}_p$	CowGear and ChaiGear perform similar or worse than stricter security types, omit this adversary model for almost free security
SemiDishonest	$\mathbb{F}_p$	Temi outperforms Semi , Hemi and Soho in observed parameters. semi-bmr diverges away from yao very fast in observed parameters, Yao performs well for the 2-party case
SemiDishonest	$\mathbb{Z}_2$ GC	
MalHonest	$\mathbb{F}_p$	SY can be omitted in favor of the other candidates
MalHonest	<i>modulo</i> 2^k	Brain is clearly slower than the other candidates
MalHonest	$\mathbb{Z}_2$ SS	PS performs the worst of this group, omit
MalHonest	$\mathbb{Z}_2$ GC	the mal-shamir version has a slightly lower RAM reservation peak, but is almost half the speed of the mal-rep version, unselect both for being slower than stronger security types
SemiHonest	$\mathbb{F}_p$	ATLAS is outperformed by Shamir and Rep3 in observed parameters, omit
SemiHonest	$\mathbb{Z}_2$ SS	CCD has a worse performance than Rep3
SemiHonest	$\mathbb{Z}_2$ GC	the shamir version of BMR performs worse than the rep version for all observed parameters

TABLE A.8: Experiment 35 set intersection preliminary results interpretation

A.1.2 MEASUREMENT PLOTS FOR SECTION A.1.2

```

1 experiments=31_scalable_search
2 protocols=mascot,lowgear,spd2k,tinier,temi,semi2k,semi-bin,yao,yao0
3 input=10,50,100,...,500
4 measureram
5 freq=3.0,2.2,2.0,1.8,1.6
6 cpu=8,4,3,2,1

```

LISTING A.5: Experiment config file example: scalable search, Dishonest Majority protocols, testtypes: CPU frequencies and threads

```

1 experiments=32_matrix_add
2 protocols=mascot,lowgear,spd2k,tinier,semi,hemi,semi2k,semi-bin,yao
3 input=3,6,...,33
4 measureram
5 freq=3.0,2.2,2.0,1.8,1.6
6 cpuquota=240,180,140,100,60

```

LISTING A.6: Experiment config file example: matrix integer addition Dishonest Majority protocols, testtypes: CPU frequencies and quotas

MATRIX INTEGER ADDITION

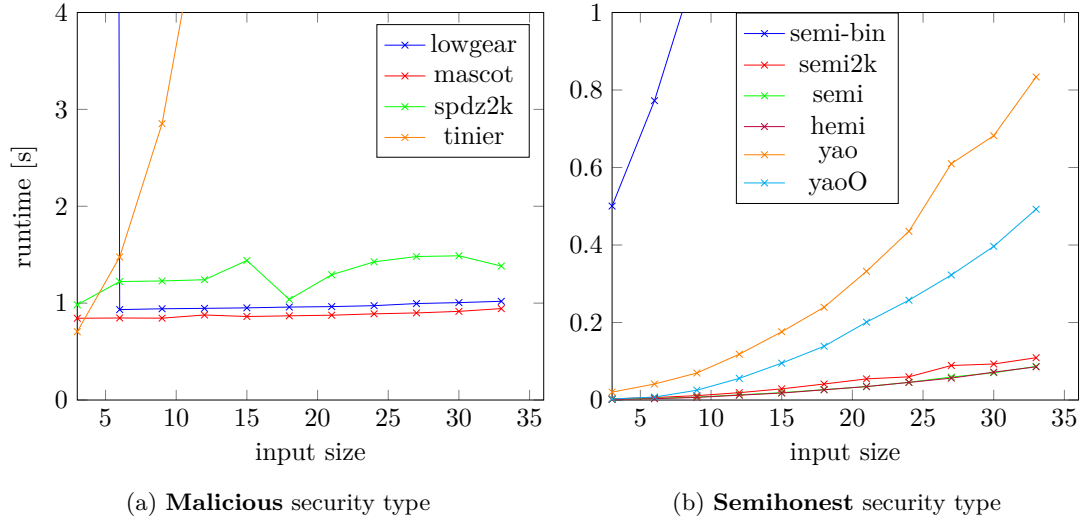


FIGURE A.1: **Dishonest majority** protocols with their runtimes for matrix integer addition on todd

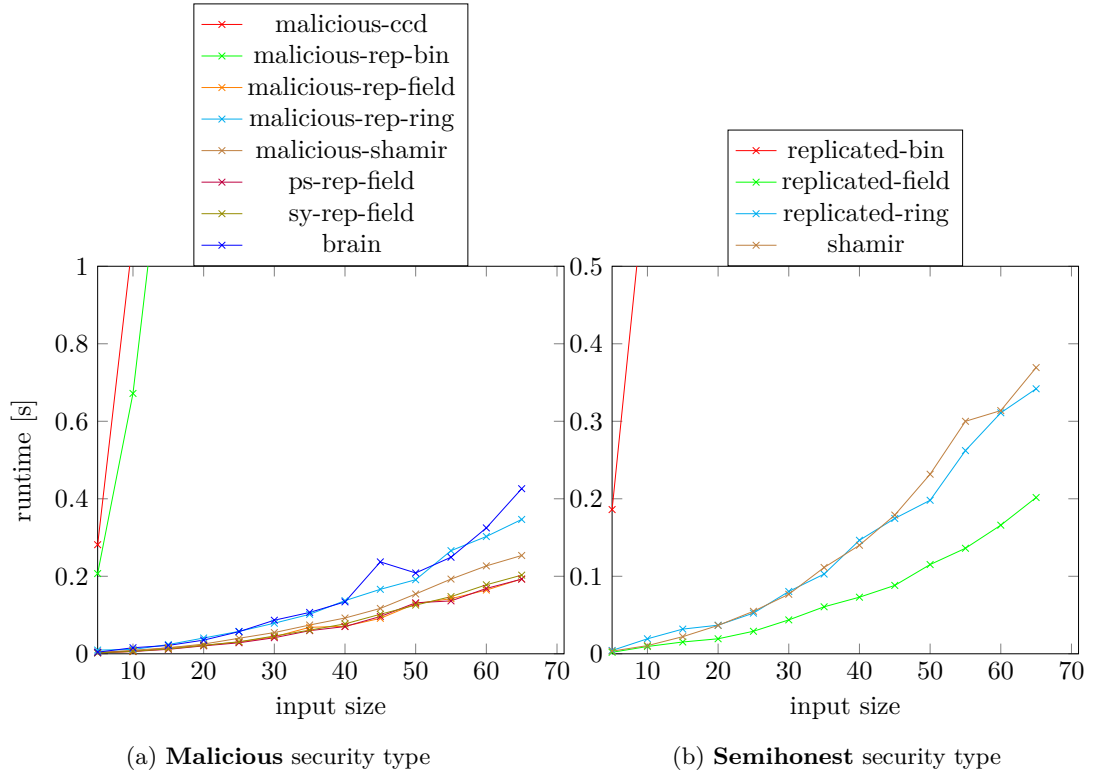


FIGURE A.2: **Honest majority** protocols with their runtimes for matrix integer addition on todd

A.1 MEASUREMENTS FOR CHAPTER 5.1

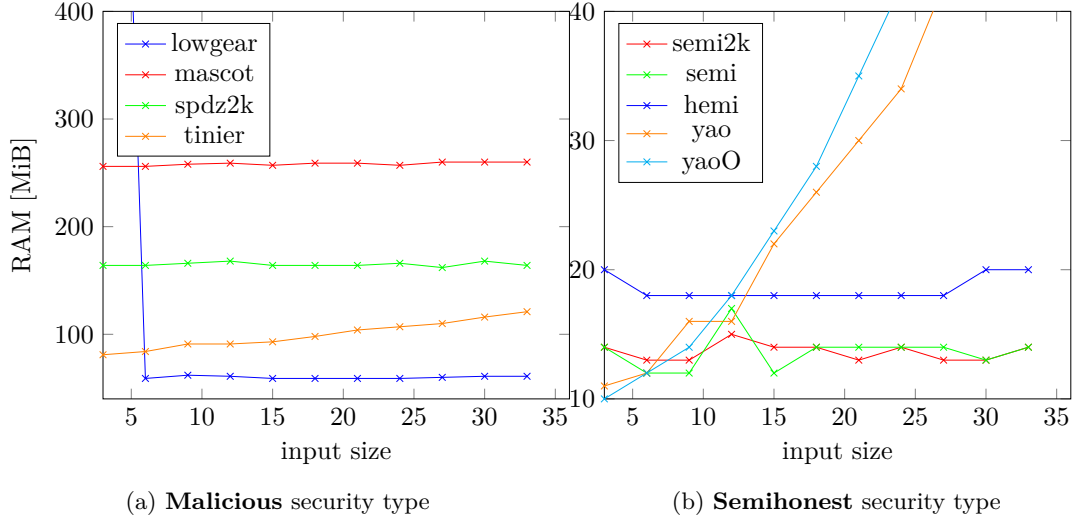


FIGURE A.3: **Dishonest majority** protocols with their peak RAM usage for matrix integer addition on todd

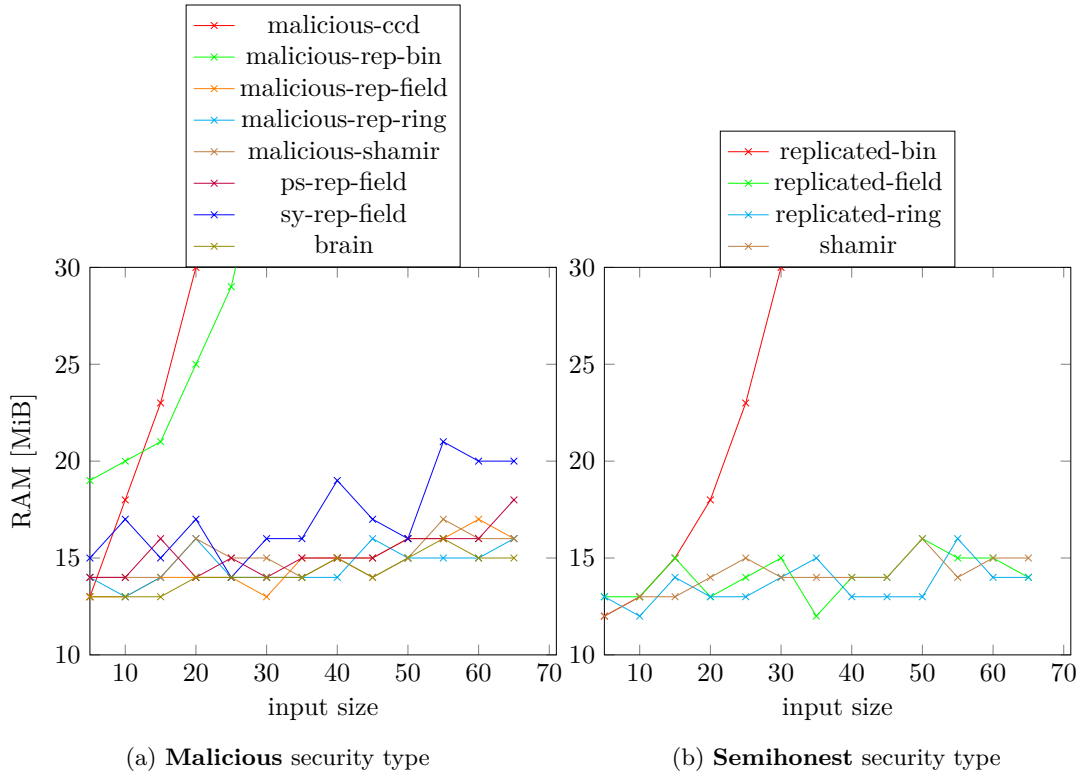


FIGURE A.4: **Honest majority** protocols with their peak RAM usage for matrix integer addition on todd

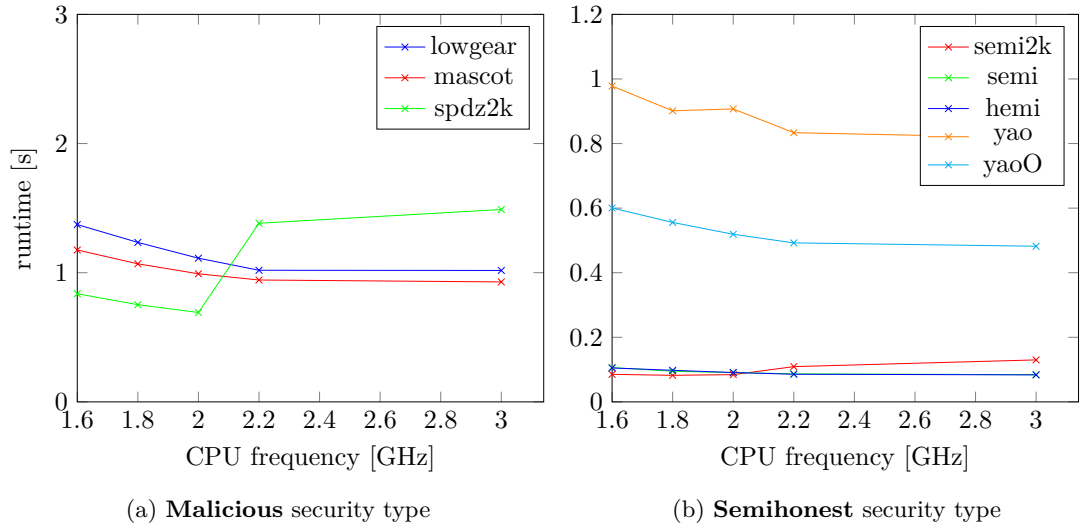


FIGURE A.5: **Dishonest majority** protocols with their runtimes for matrix integer addition based on frequencies with fixed input size 33^2 on todd

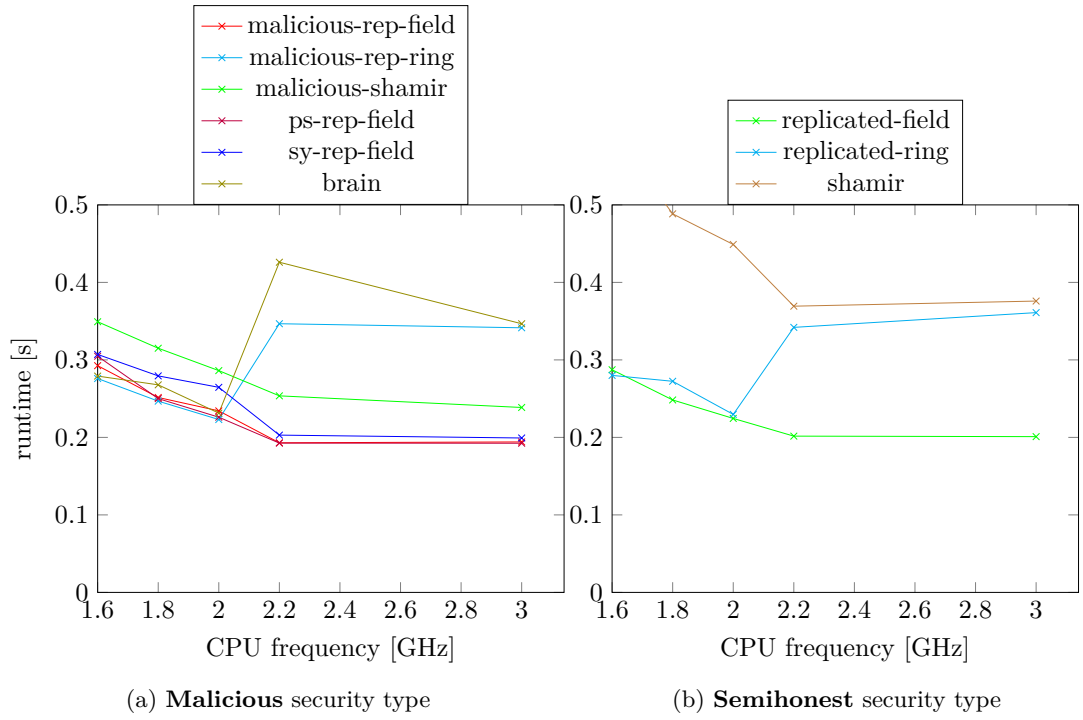
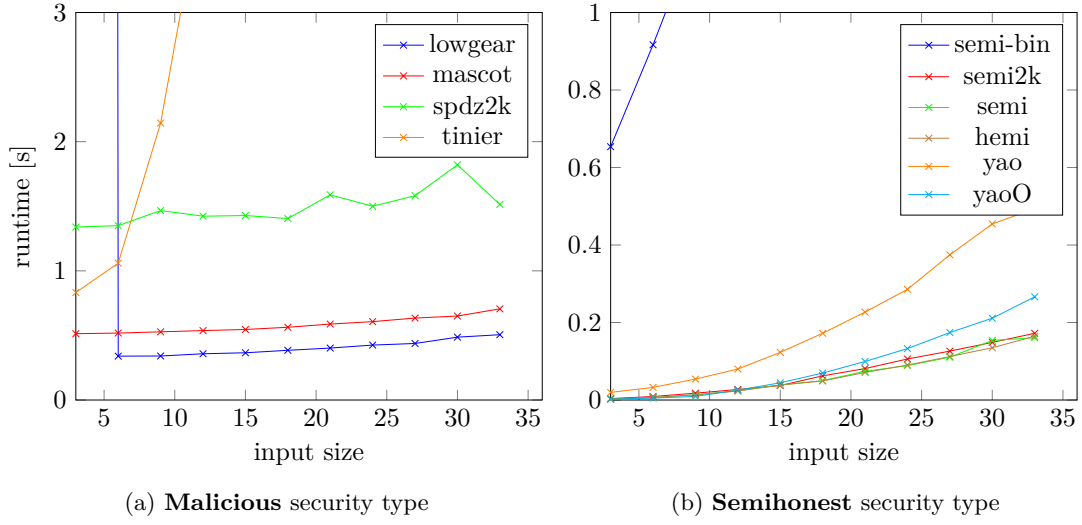
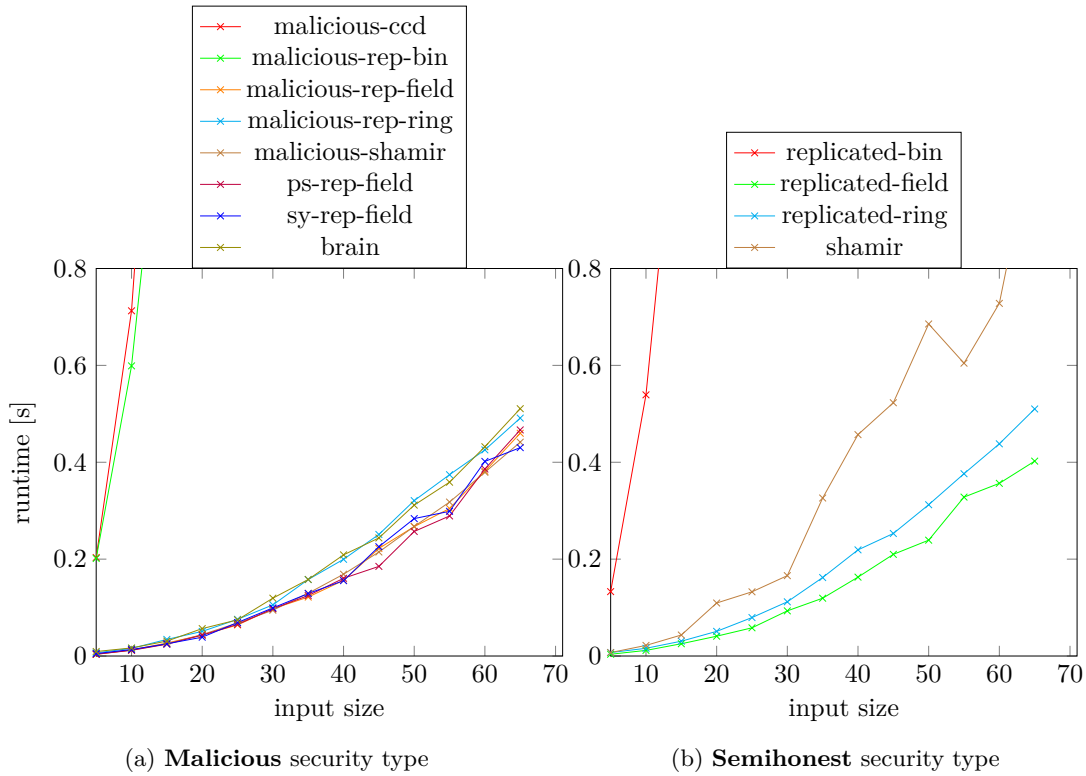


FIGURE A.6: **Honest majority** protocols with their runtimes for matrix integer addition based on frequencies with fixed input size 65^2 on todd

MATRIX FIXED-POINT ADDITION

FIGURE A.7: **Dishonest majority** protocols with their runtimes for matrix fixed-point addition on dogecoinFIGURE A.8: **Honest majority** protocols with their runtimes for matrix fixed-point addition on dogecoin

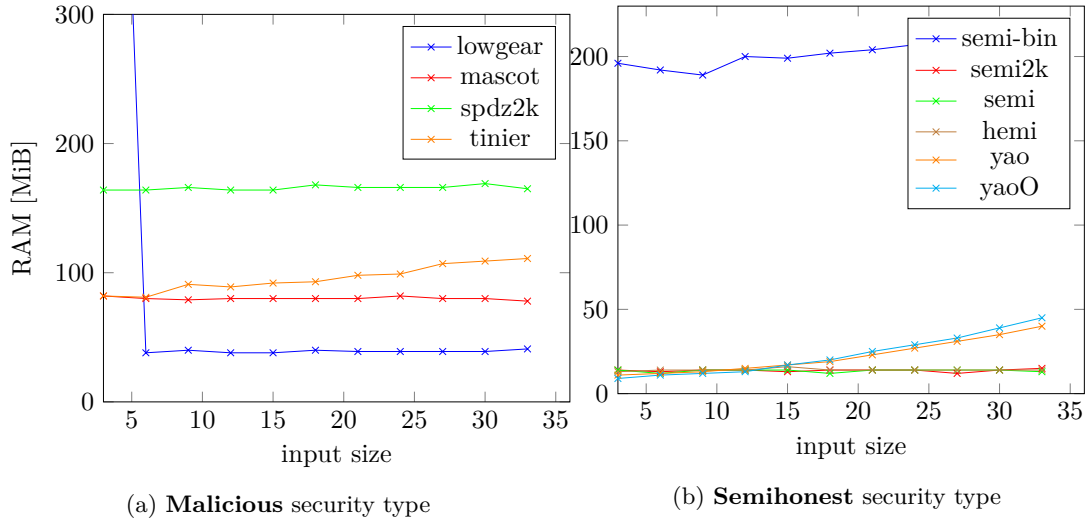


FIGURE A.9: **Dishonest majority** protocols with their peak RAM usage for matrix fixed-point addition on dogecoin

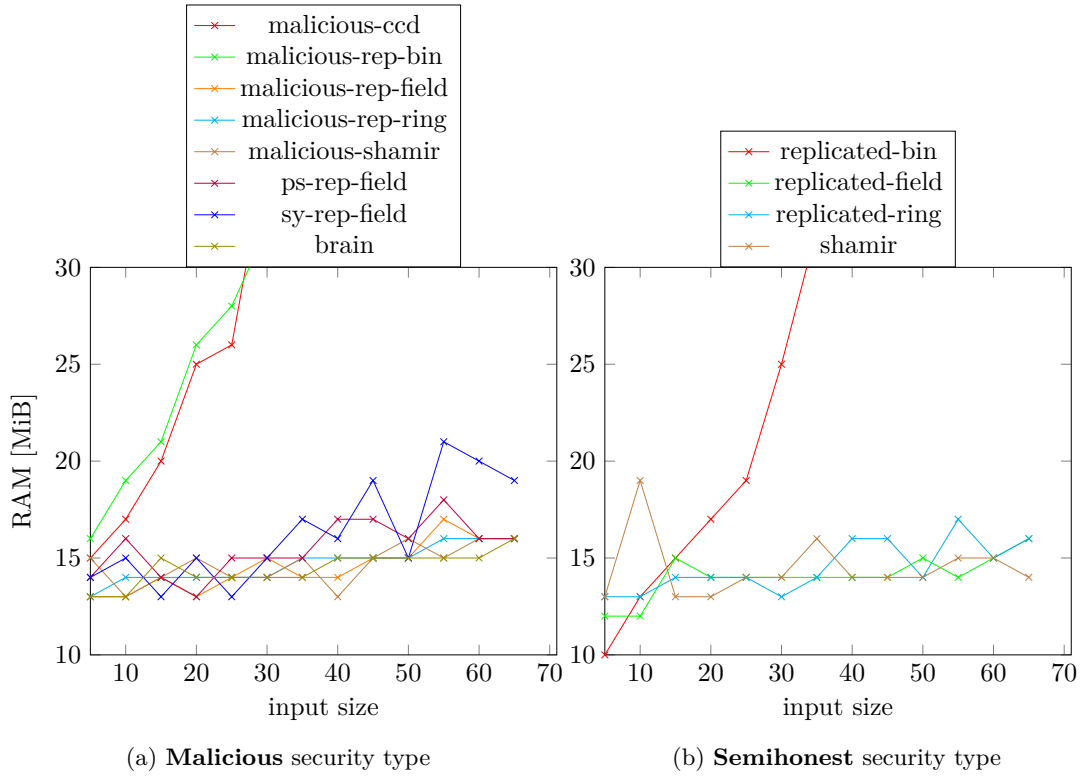


FIGURE A.10: **Honest majority** protocols with their peak RAM usage for matrix fixed-point addition on dogecoin

A.1 MEASUREMENTS FOR CHAPTER 5.1

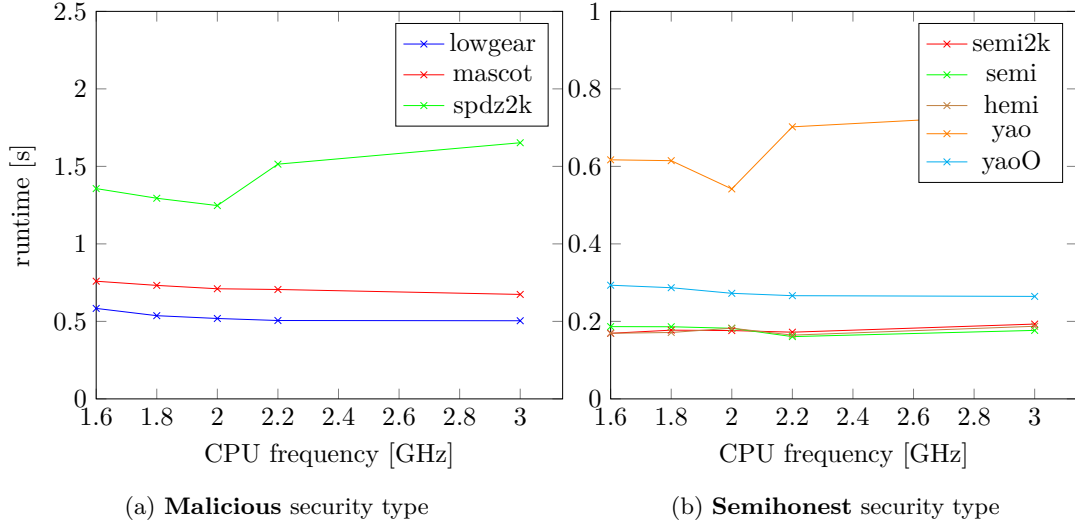


FIGURE A.11: **Dishonest majority** protocols with their runtimes for matrix fixed-point addition based on frequencies with fixed input size 33^2 on dogecoin

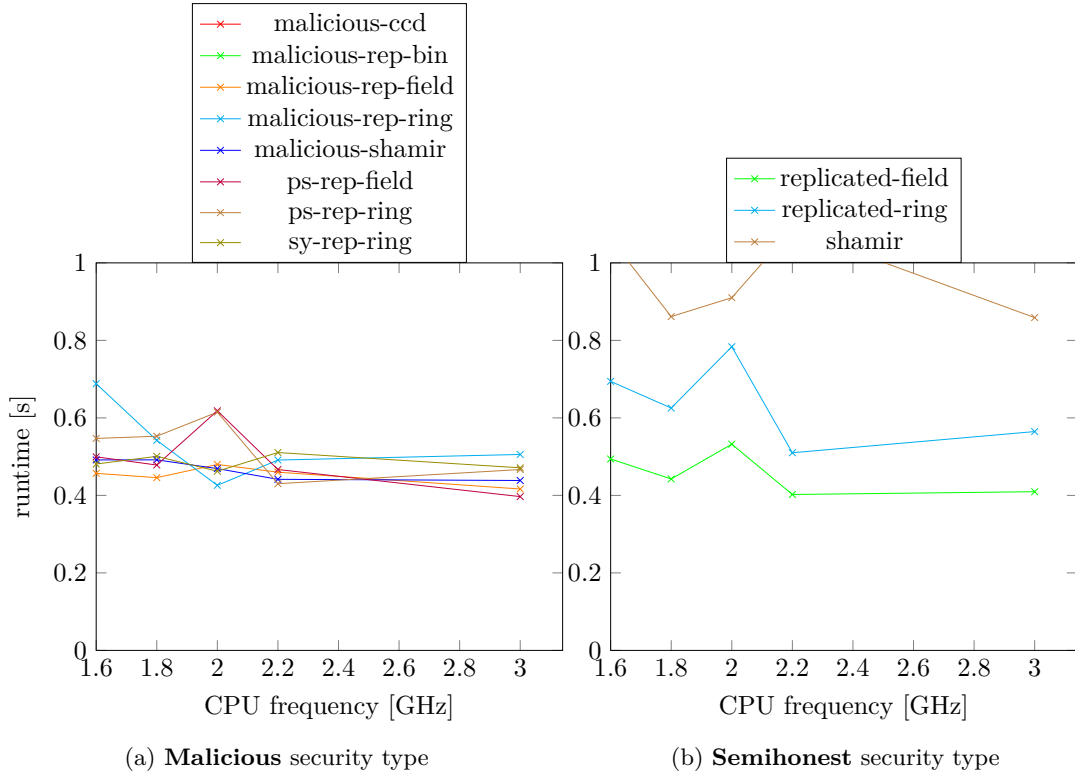


FIGURE A.12: **Honest majority** protocols with their runtimes for matrix fixed-point addition based on frequencies with fixed input size 65^2 on dogecoin

MATRIX INTEGER MULTIPLICATION

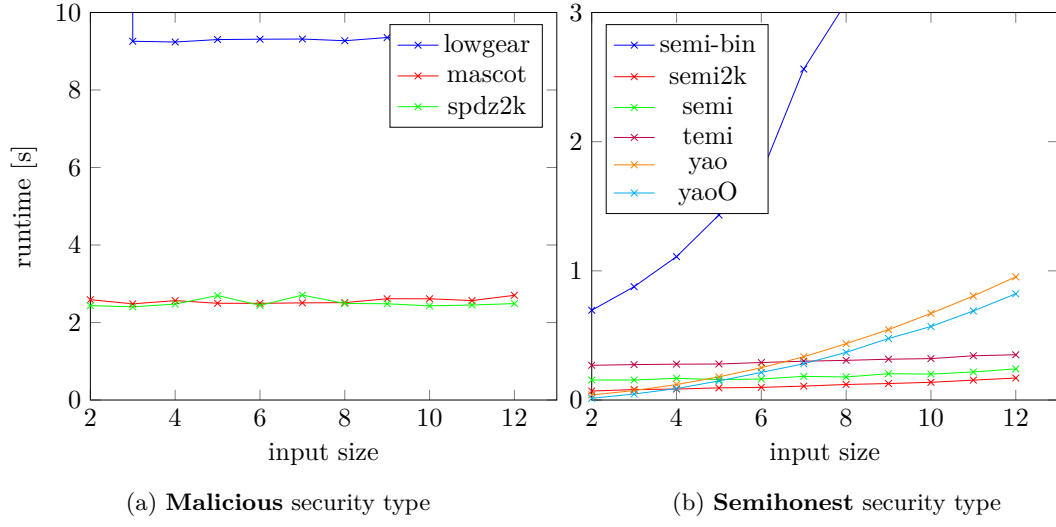


FIGURE A.13: **Dishonest majority** protocols with their runtimes for matrix integer multiplication on bitcoin

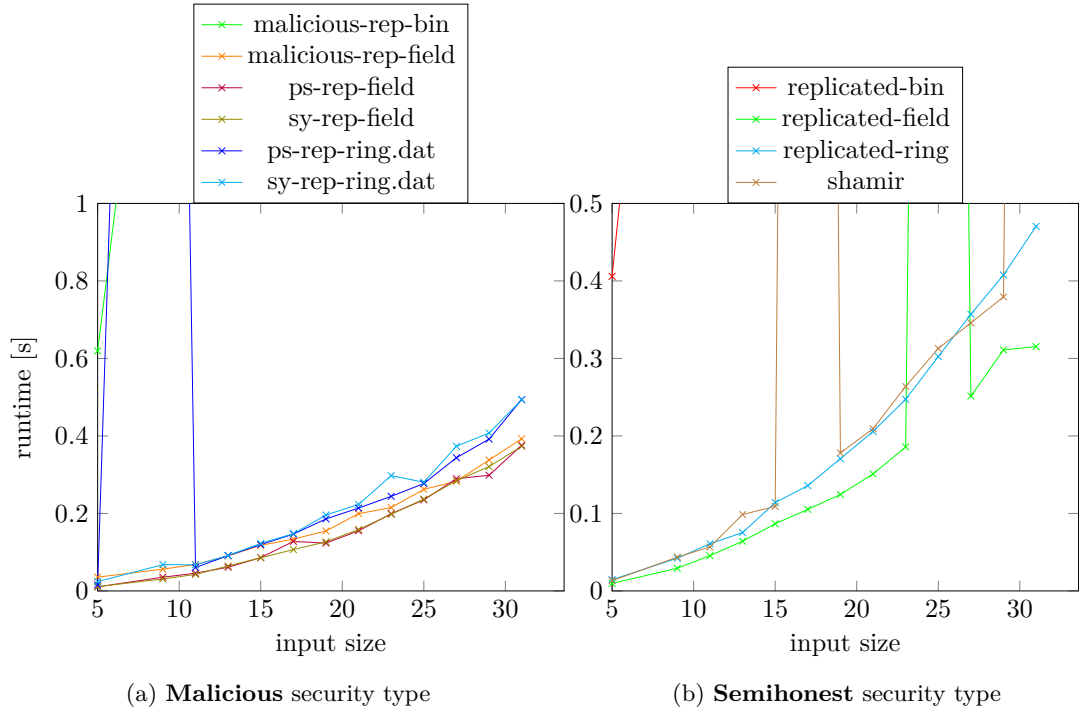


FIGURE A.14: **Honest majority** protocols with their runtimes for matrix integer multiplication on bitcoin

A.1 MEASUREMENTS FOR CHAPTER 5.1

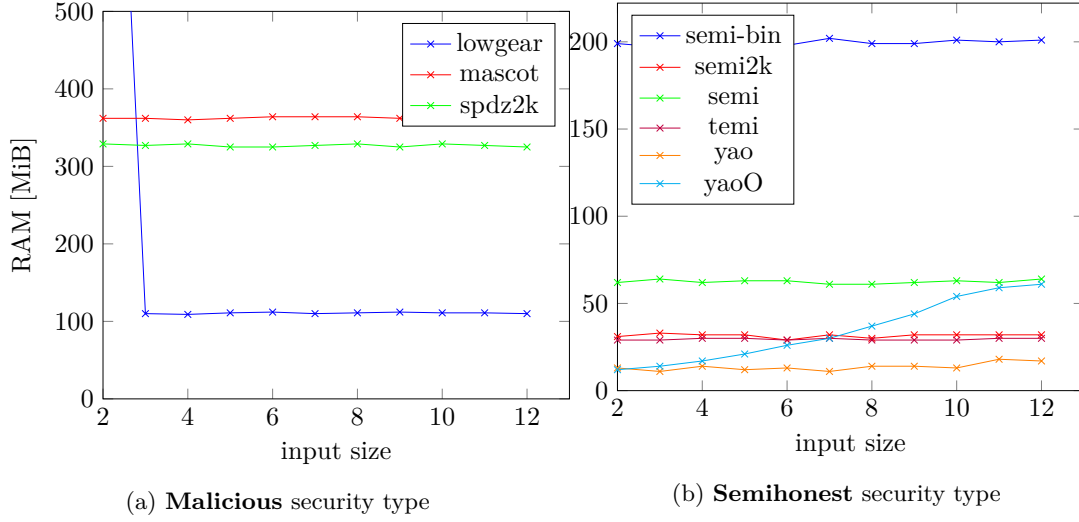


FIGURE A.15: **Dishonest majority** protocols with their peak RAM usage for matrix integer multiplication on bitcoin

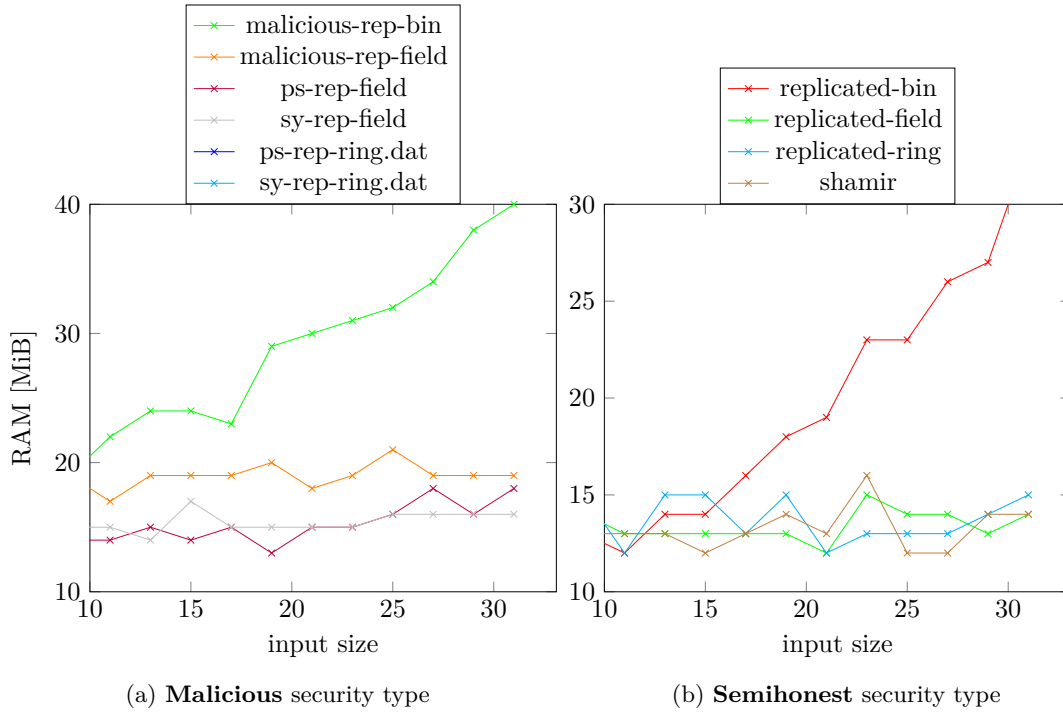
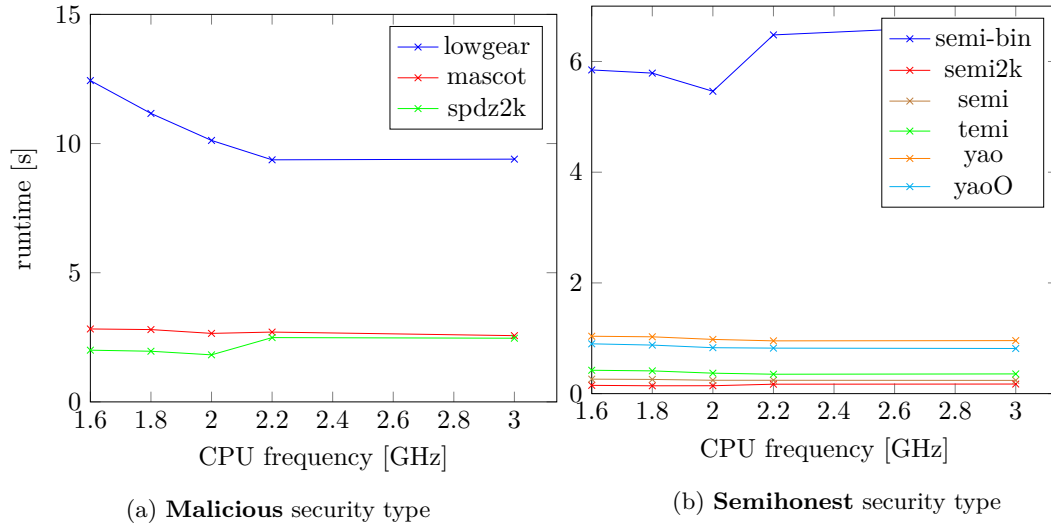
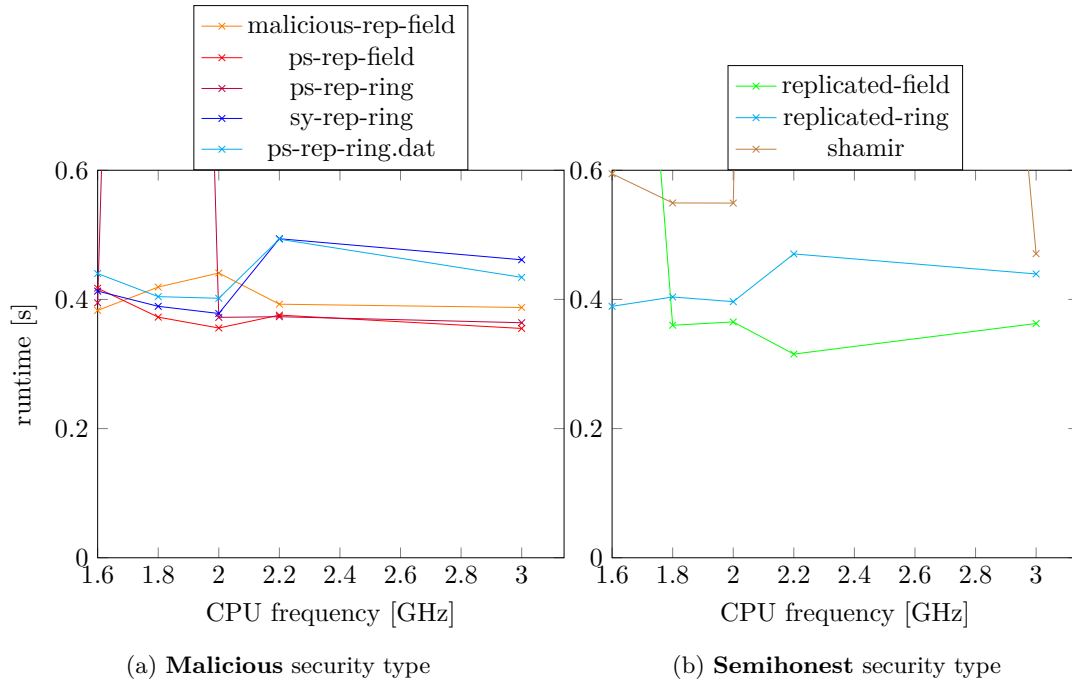
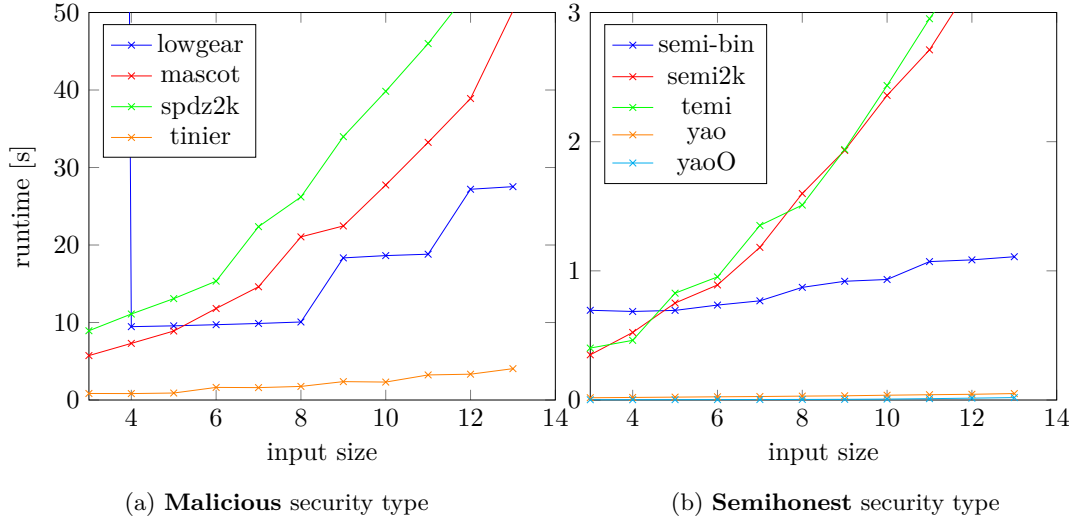
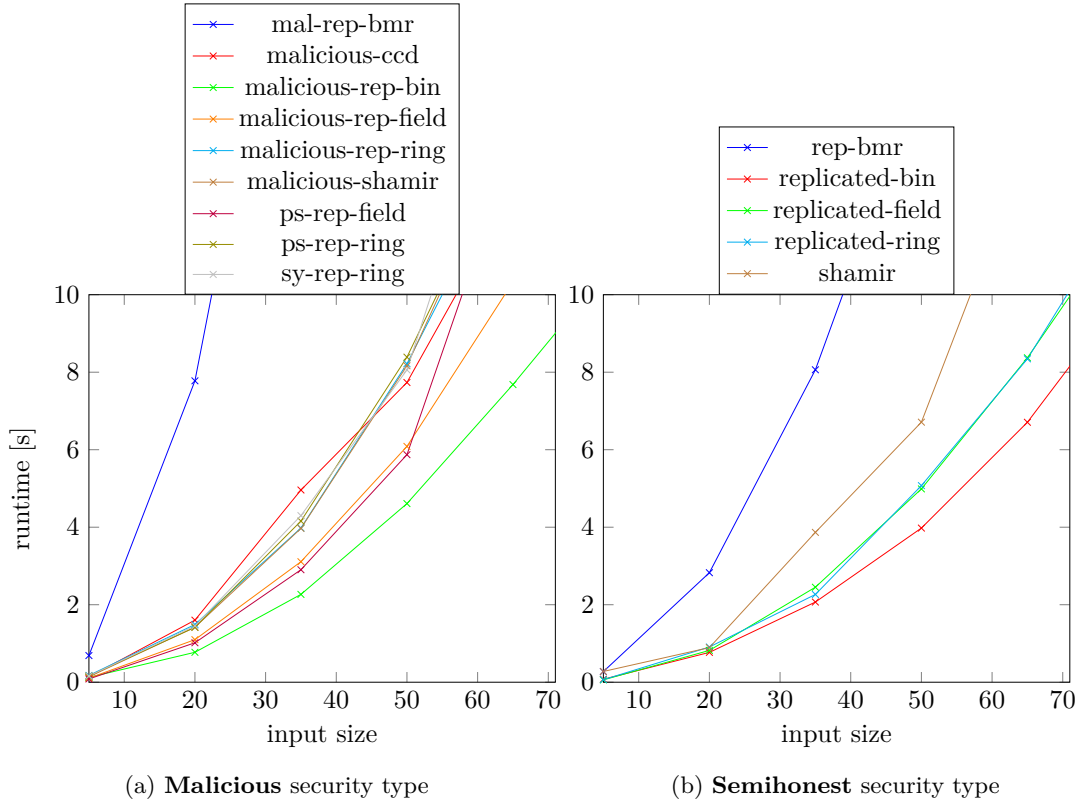


FIGURE A.16: **Honest majority** protocols with their peak RAM usage for matrix integer multiplication on bitcoin


 FIGURE A.17: **Dishonest majority** protocols with their runtimes for matrix integer multiplication based on frequencies with fixed input size 12^2 on bitcoin

 FIGURE A.18: **Honest majority** protocols with their runtimes for matrix integer multiplication based on frequencies with fixed input size 31^2 on bitcoin

SET INTERSECTION

FIGURE A.19: **Dishonest majority** protocols with their runtimes for set intersection on dogecoinFIGURE A.20: **Honest majority** protocols with their runtimes for set intersection on bitcoin

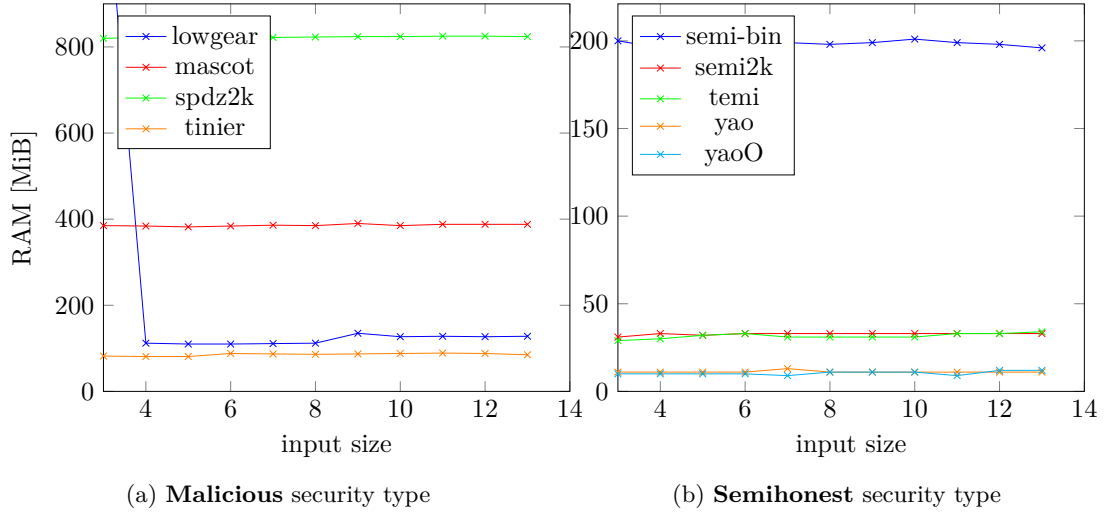


FIGURE A.21: **Dishonest majority** protocols with their peak RAM usage for set intersection on dogecoin

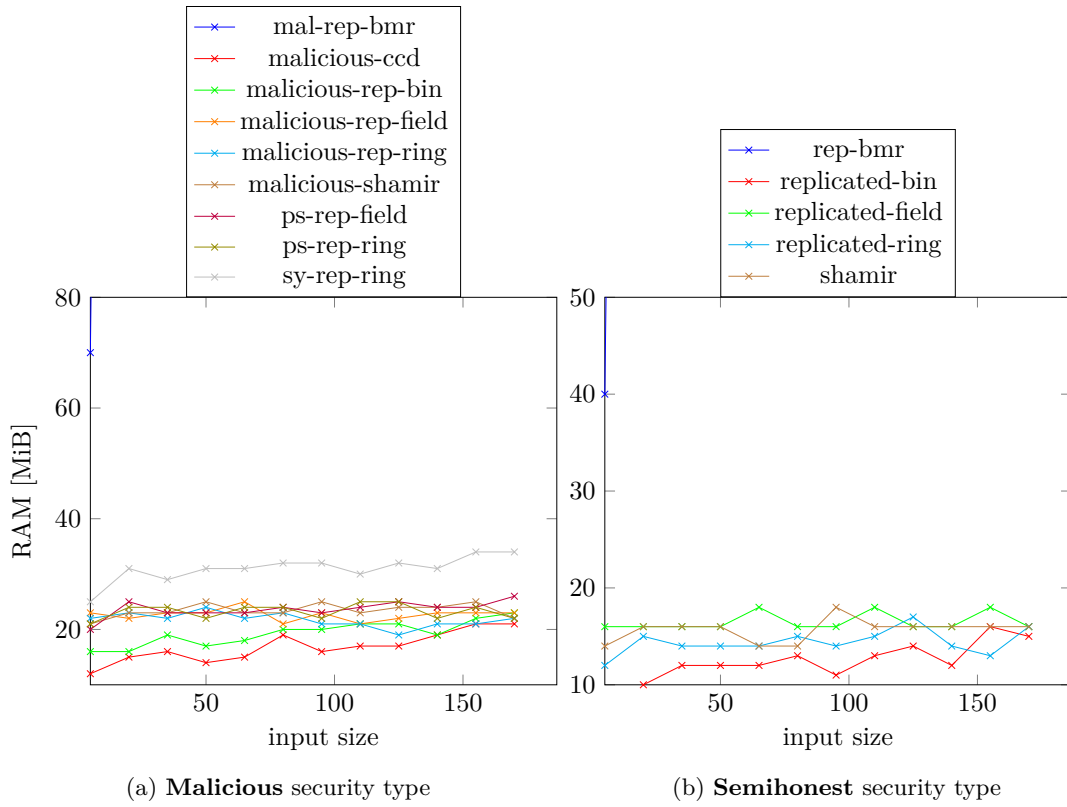


FIGURE A.22: **Honest majority** protocols with their on bitcoin

A.1 MEASUREMENTS FOR CHAPTER 5.1

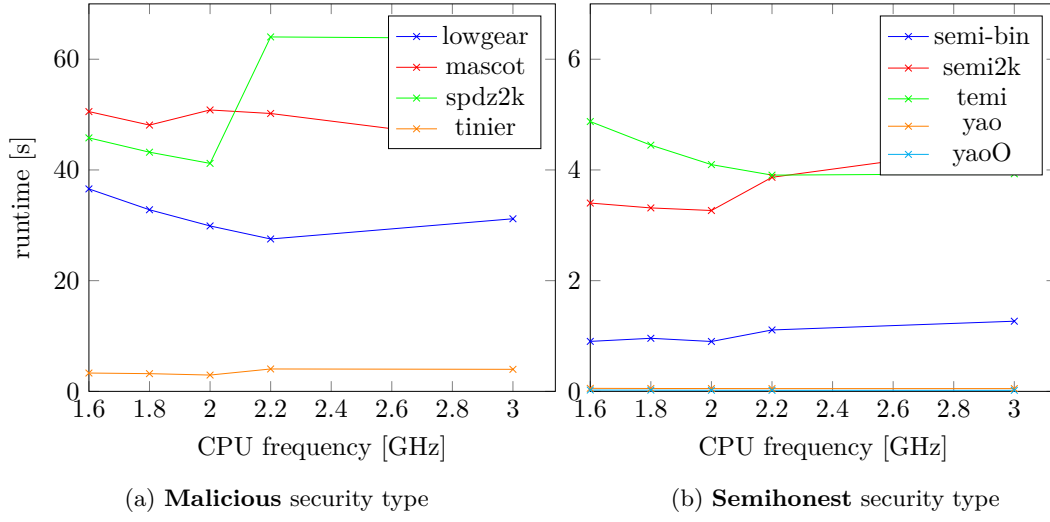


FIGURE A.23: **Dishonest majority** protocols with their runtimes for set intersection based on frequencies with fixed input size 13 on dogecoin

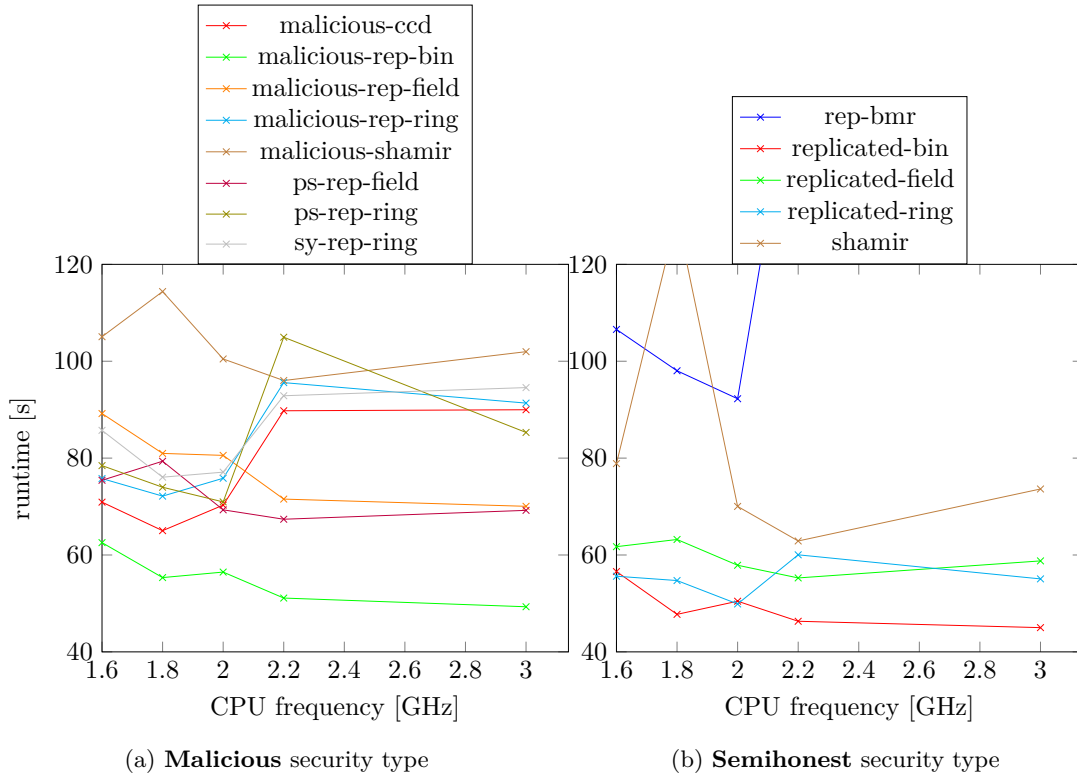


FIGURE A.24: **Honest majority** protocols with their runtimes for set intersection based on frequencies with fixed input size 170 on bitcoin

A.1.3 MEASUREMENT PLOTS FOR SECTION 5.2

FREQUENCY

INTEGER ADDITION

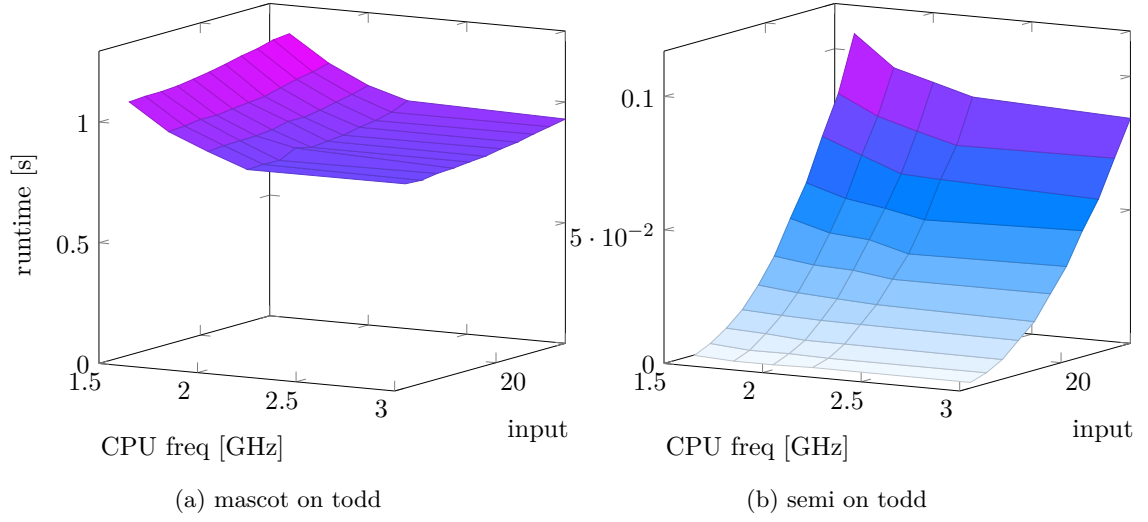


FIGURE A.25: Dishonest majority protocol finalists reaction to CPU frequency change (Integer Addition)

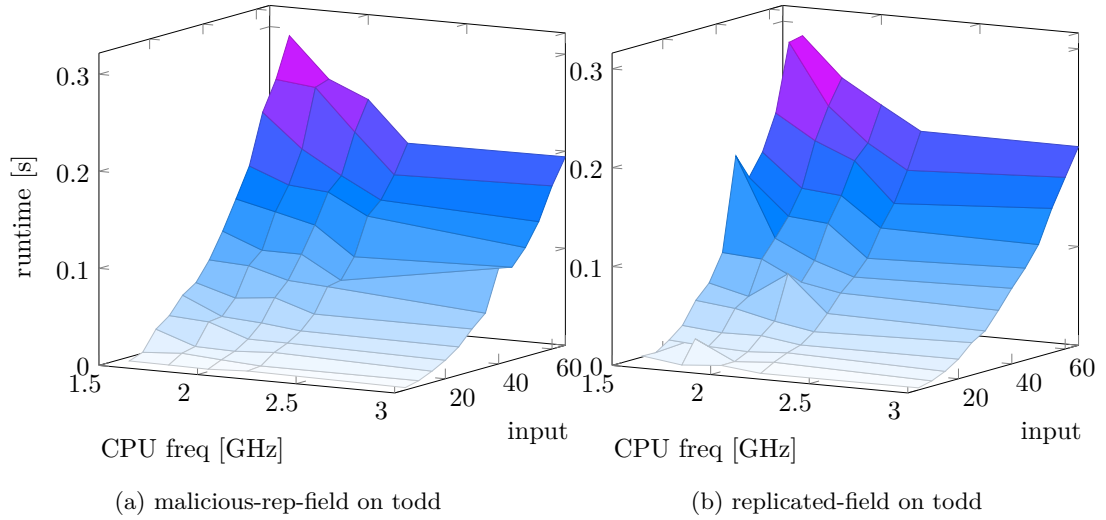


FIGURE A.26: Honest majority protocol finalists reaction to CPU frequency change (Integer Addition)

FIXED-POINT ADDITION

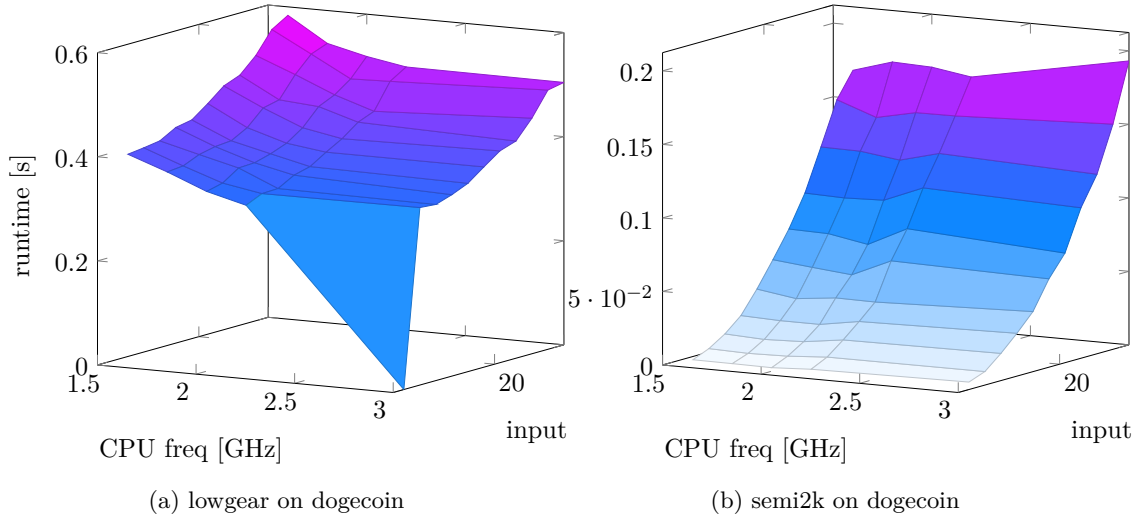


FIGURE A.27: Dishonest majority protocol finalists reaction to CPU frequency change (Fixed-point Addition)

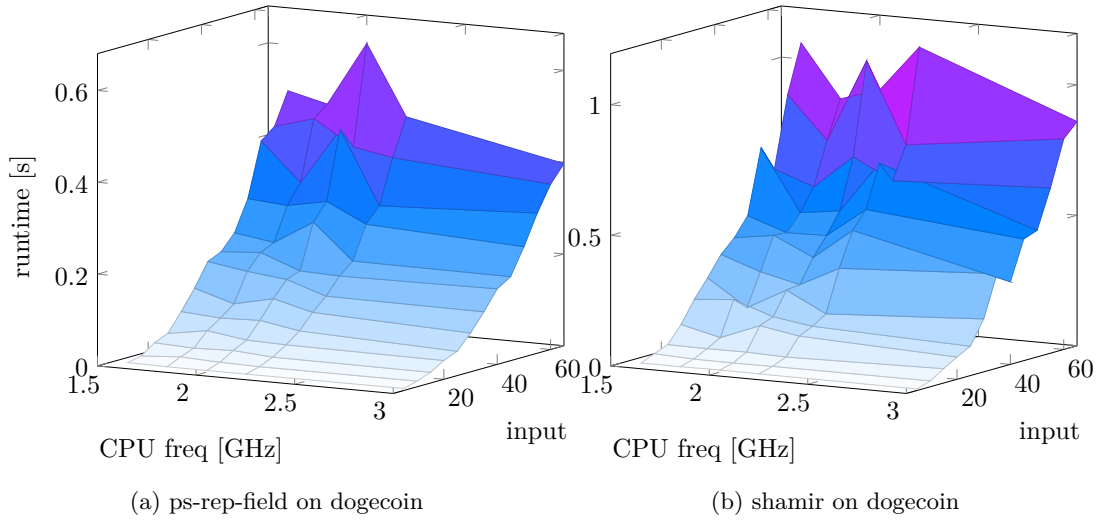


FIGURE A.28: Honest majority protocol finalists reaction to CPU frequency change (Fixed-point Addition)

For lowgear in figure A.27a, at input=3 and freq=3 the measured value was 265.511s. To scale the other values it was changed to 0. On its first run, lowgear pregenerates random triples that will be reused on subsequent runs.

INTEGER MULTIPLICATION

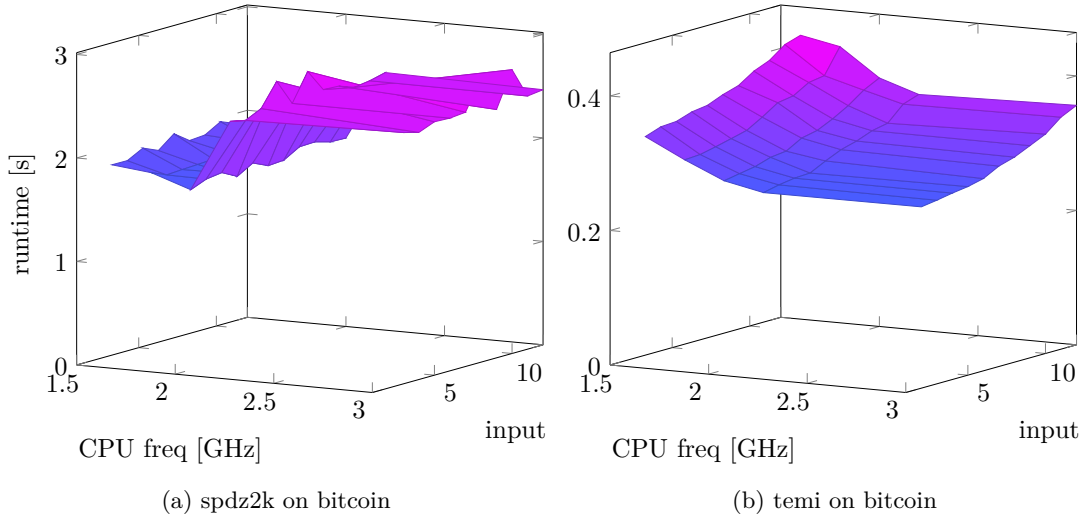


FIGURE A.29: Dishonest majority protocol finalists reaction to CPU frequency change (Integer Multiplication)

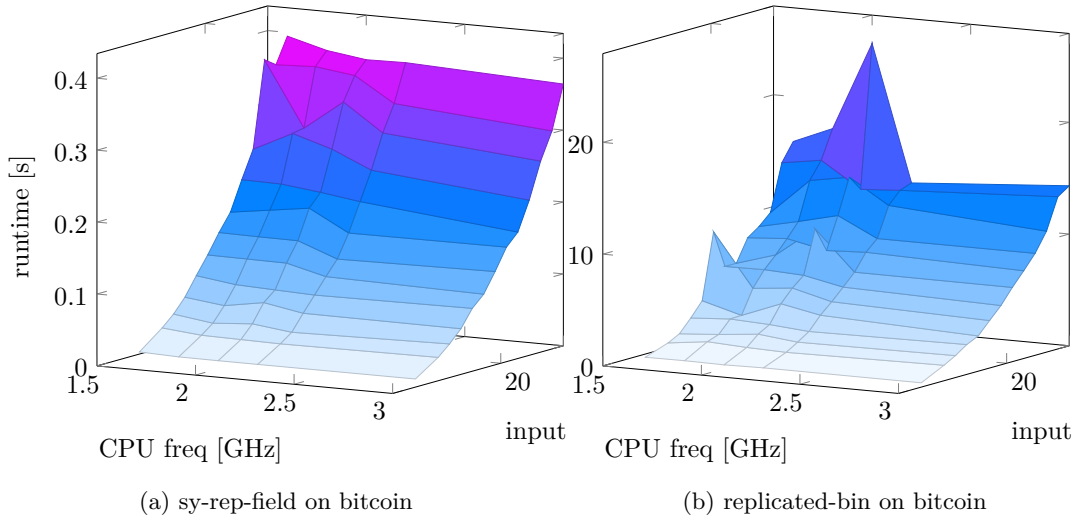


FIGURE A.30: Honest majority protocol finalists reaction to CPU frequency change (Integer Multiplication)

SET INTERSECTION

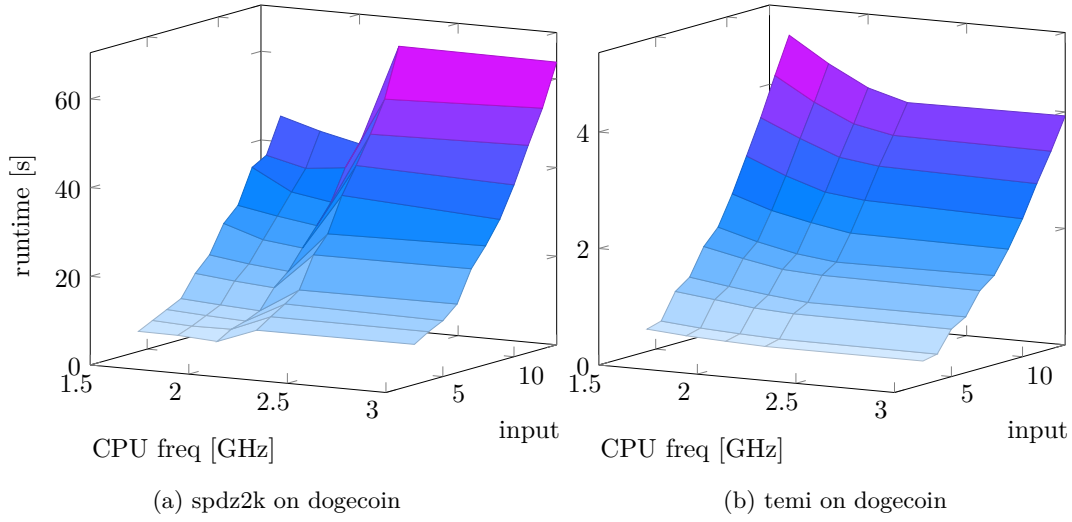


FIGURE A.31: Dishonest majority protocol finalists reaction to CPU frequency change (Set Intersection)

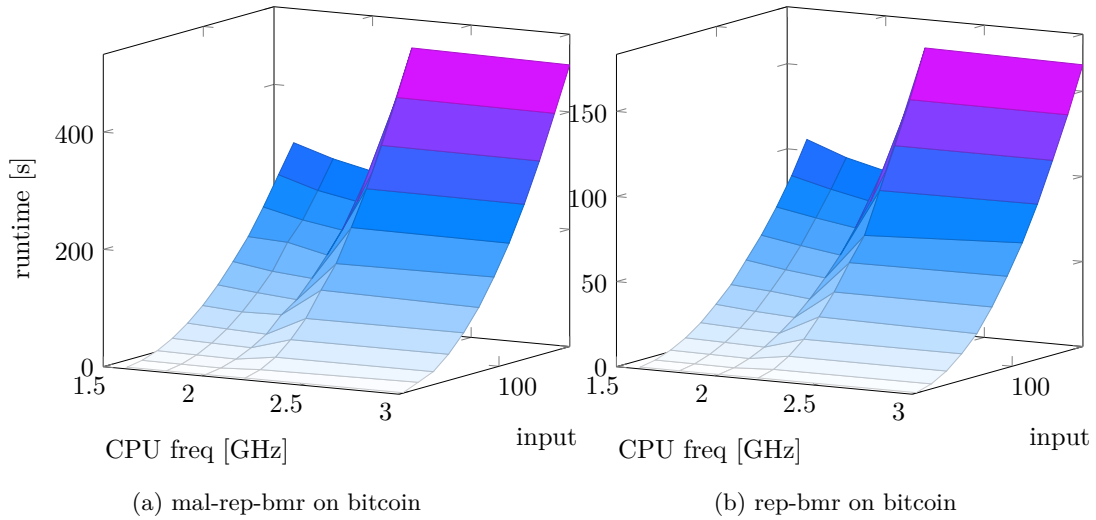


FIGURE A.32: Honest majority protocol finalists reaction to CPU frequency change (Set Intersection)

The otherwise smooth surfaces, especially for the rep-bmr versions, are surprising with the sharp rise in runtime for 2.2 GHz. This phenomenon was double checked and cannot currently be explained, as it only happens with the CPU thread count fixed to the maximum of 8, which is theoretically the least constraining. Choosing the next lower CPU thread count of 4 would have hidden this anomaly, but it was kept as it corresponds to the behavior seen in Figure A.48.

QUOTA

INTEGER ADDITION

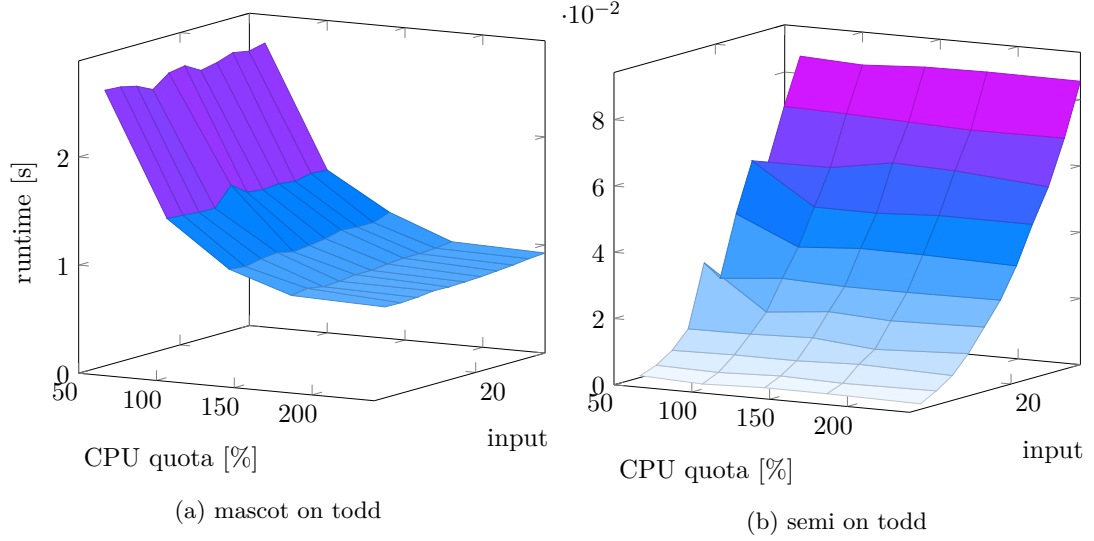


FIGURE A.33: Dishonest majority protocol finalists reaction to CPU quota limit change (Integer Addition)

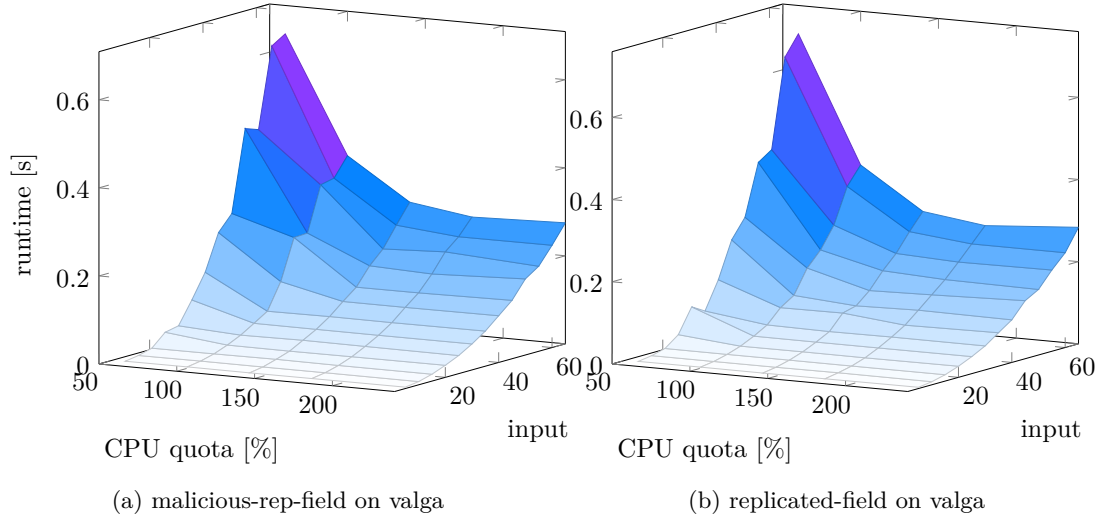


FIGURE A.34: Honest majority protocol finalists reaction to CPU quota limit change (Integer Addition)

FIXED-POINT ADDITION

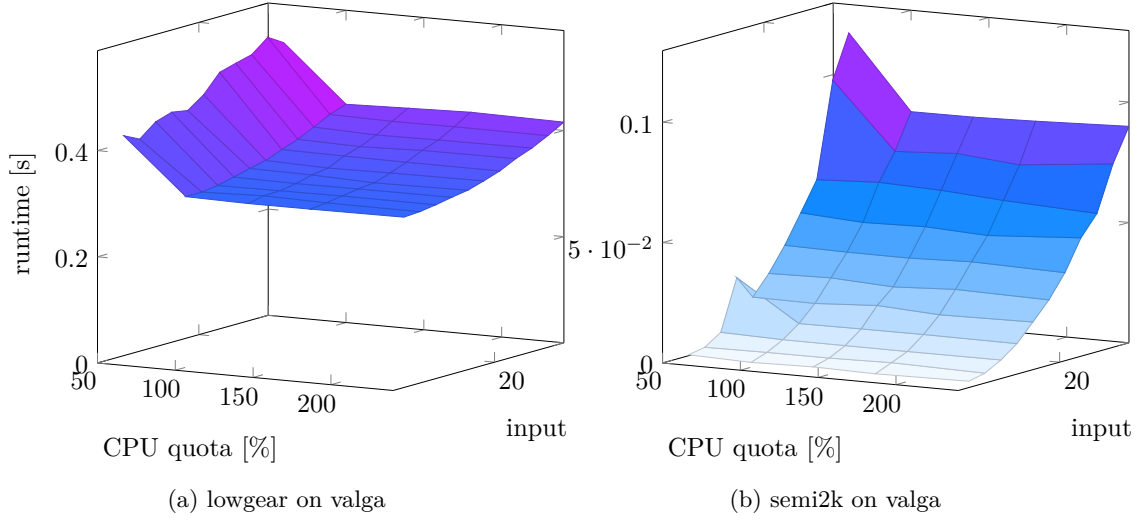


FIGURE A.35: Dishonest majority protocol finalists reaction to CPU quota limit change (Fixed-point Addition)

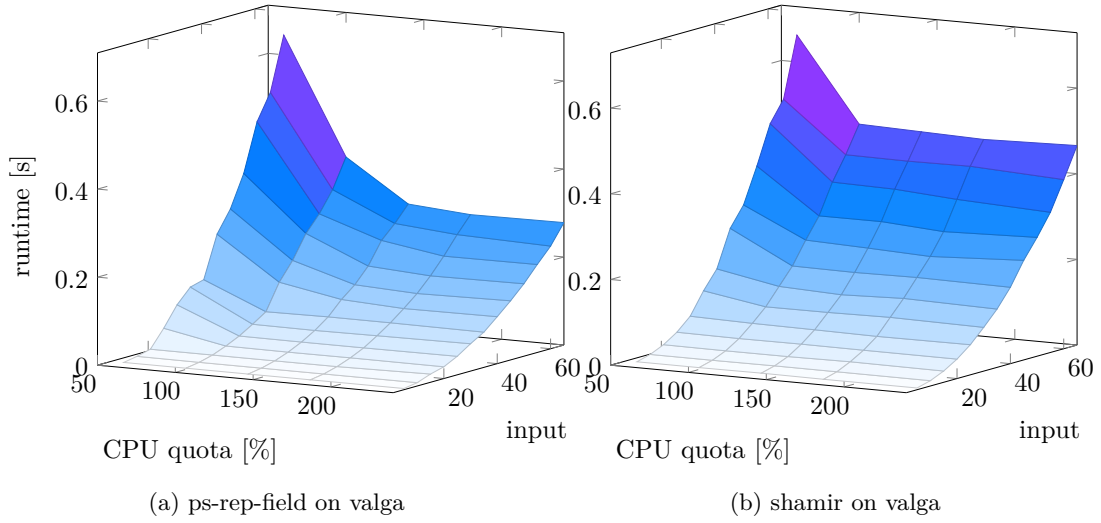


FIGURE A.36: Honest majority protocol finalists reaction to CPU quota limit change (Fixed-point Addition)

INTEGER MULTIPLICATION

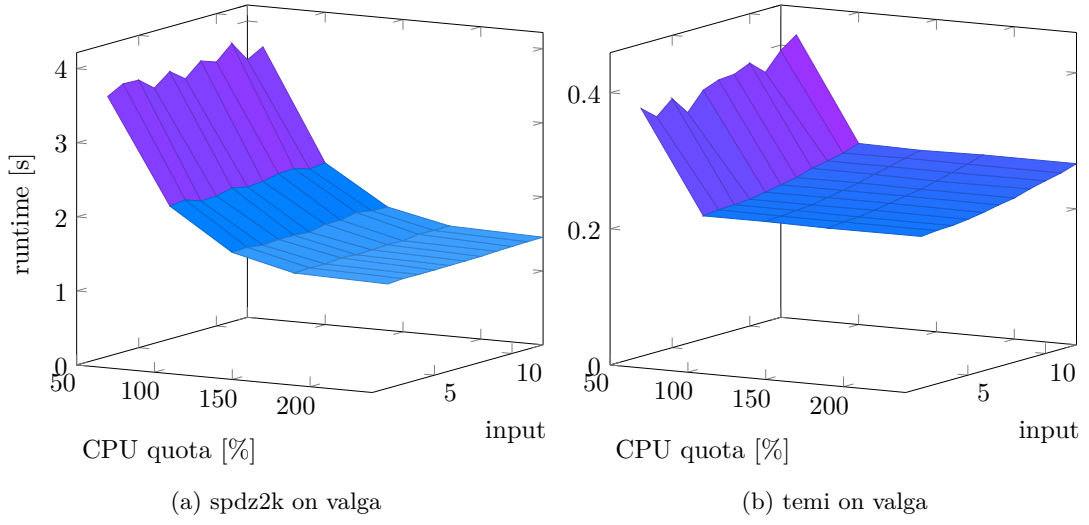


FIGURE A.37: Dishonest majority protocol finalists reaction to CPU quota limit change (Integer Multiplication)

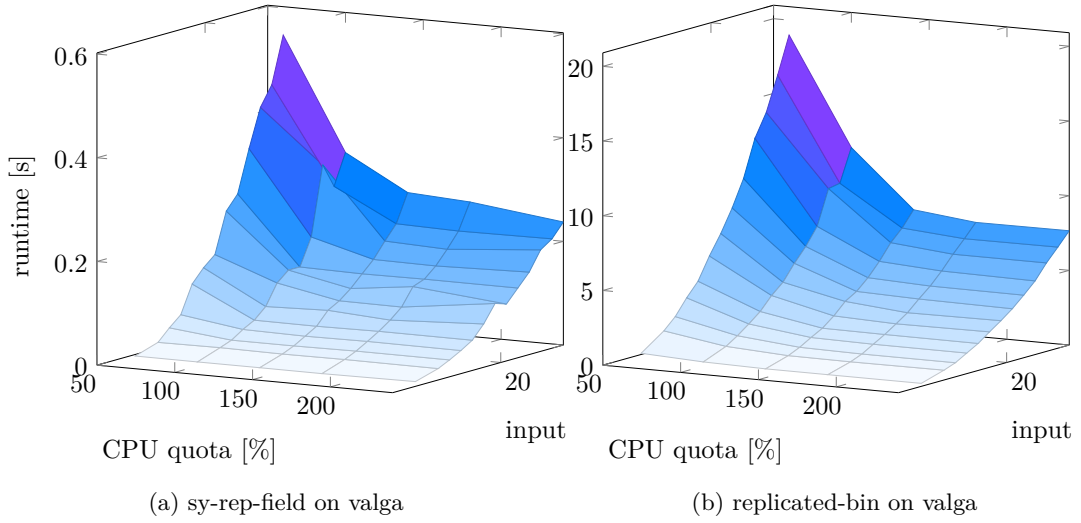


FIGURE A.38: Honest majority protocol finalists reaction to CPU quota limit change (Integer Multiplication)

SET INTERSECTION

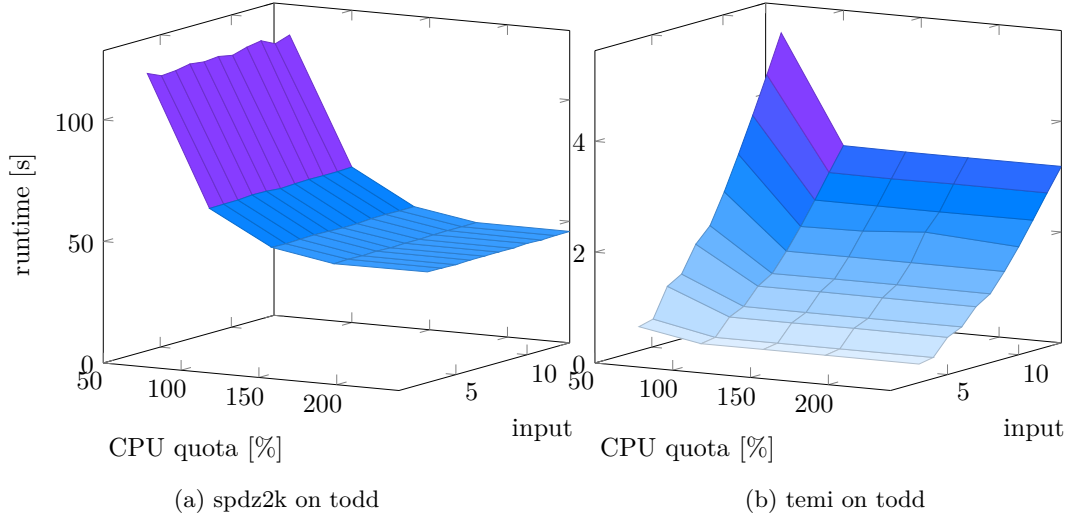


FIGURE A.39: Dishonest majority protocol finalists reaction to CPU quota limit change (Set Intersection)

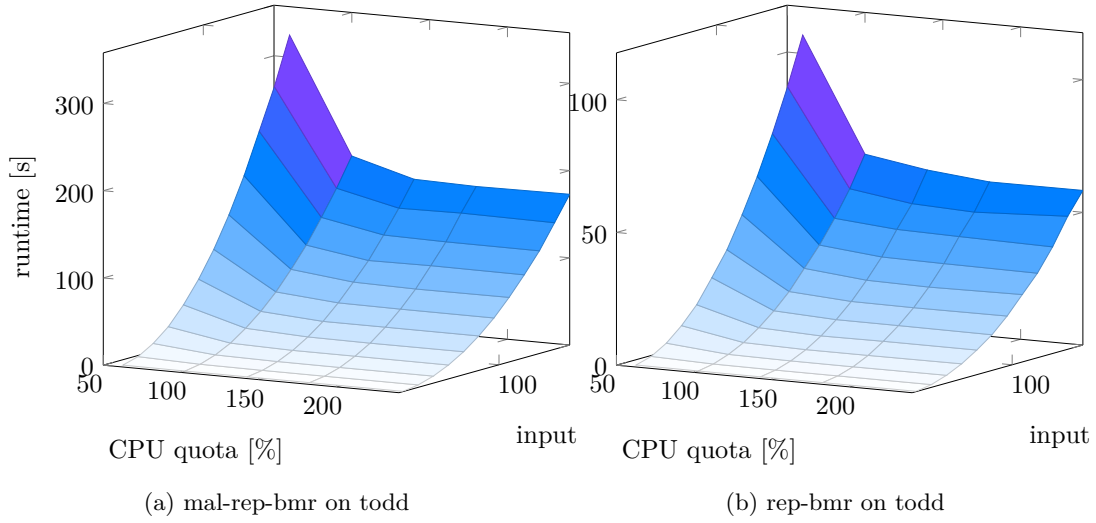


FIGURE A.40: Honest majority protocol finalists reaction to CPU quota limit change (Set Intersection)

CORES

INTEGER ADDITION

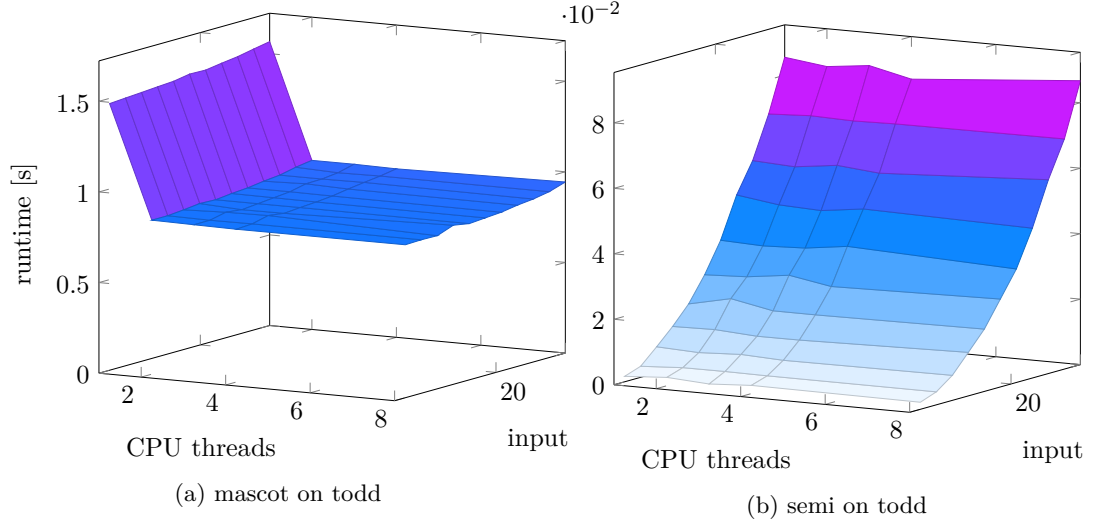


FIGURE A.41: Dishonest majority protocol finalists reaction to available CPU thread number change (Integer Addition)

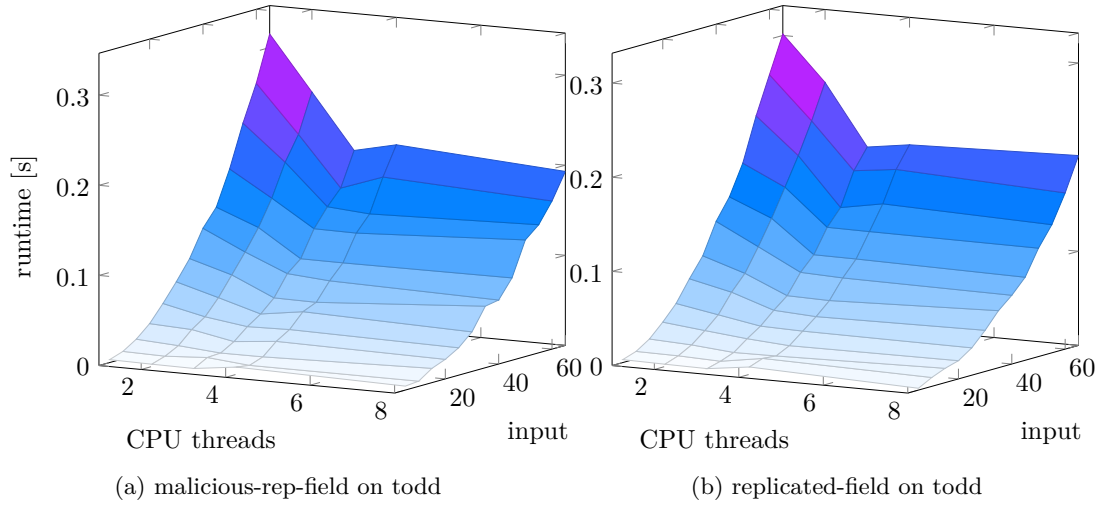


FIGURE A.42: Honest majority protocol finalists reaction to available CPU thread number change (Integer Addition)

FIXED-POINT ADDITION

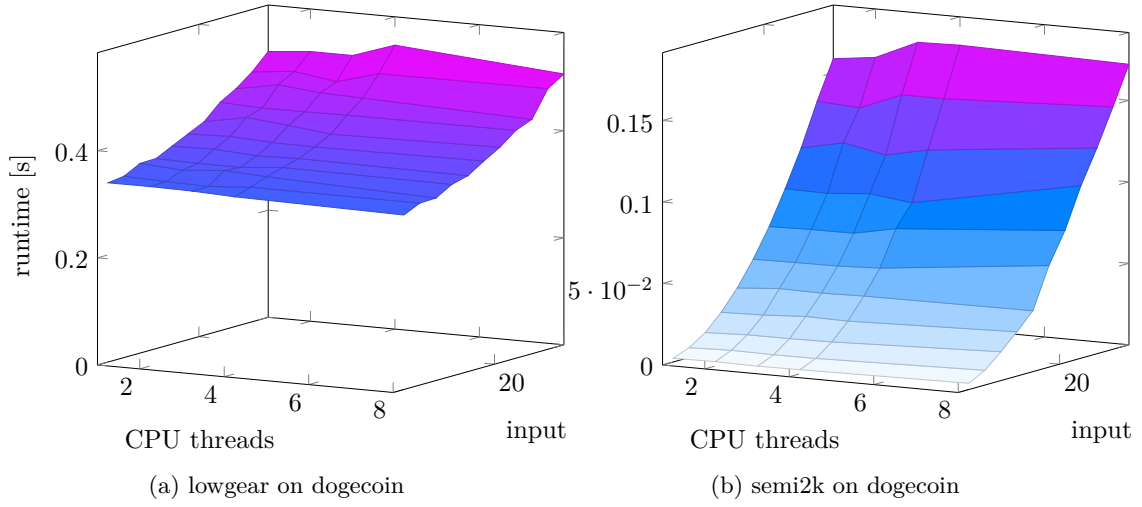


FIGURE A.43: Dishonest majority protocol finalists reaction to available CPU thread number change (Fixed-point Addition)

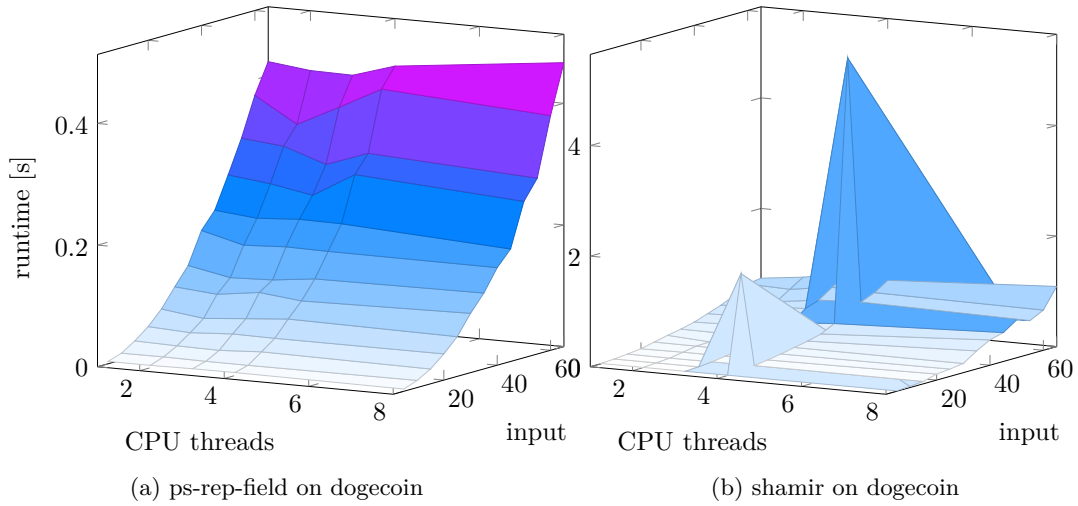


FIGURE A.44: Honest majority protocol finalists reaction to available CPU thread number change (Fixed-point Addition)

INTEGER MULTIPLICATION

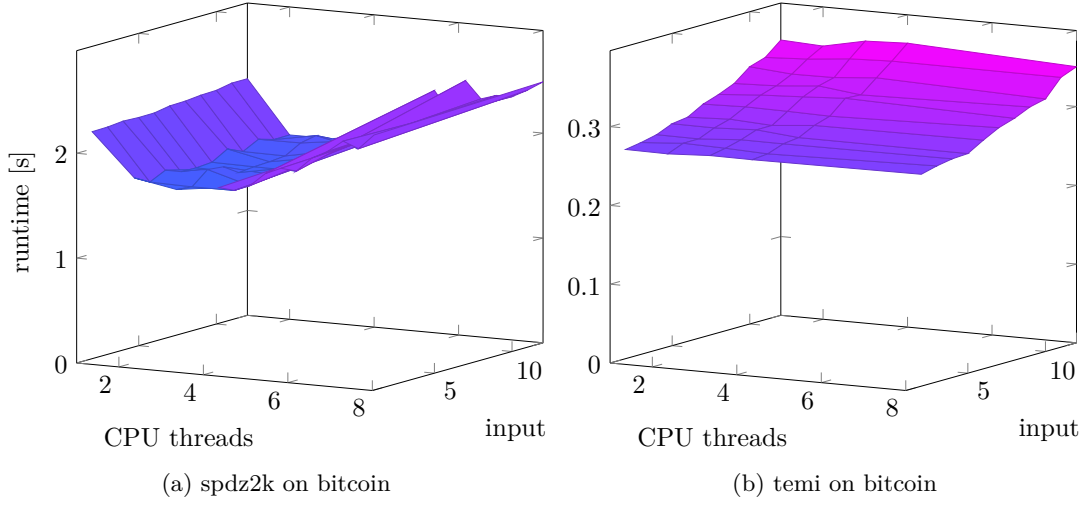


FIGURE A.45: Dishonest majority protocol finalists reaction to available CPU thread number change (Integer Multiplication)

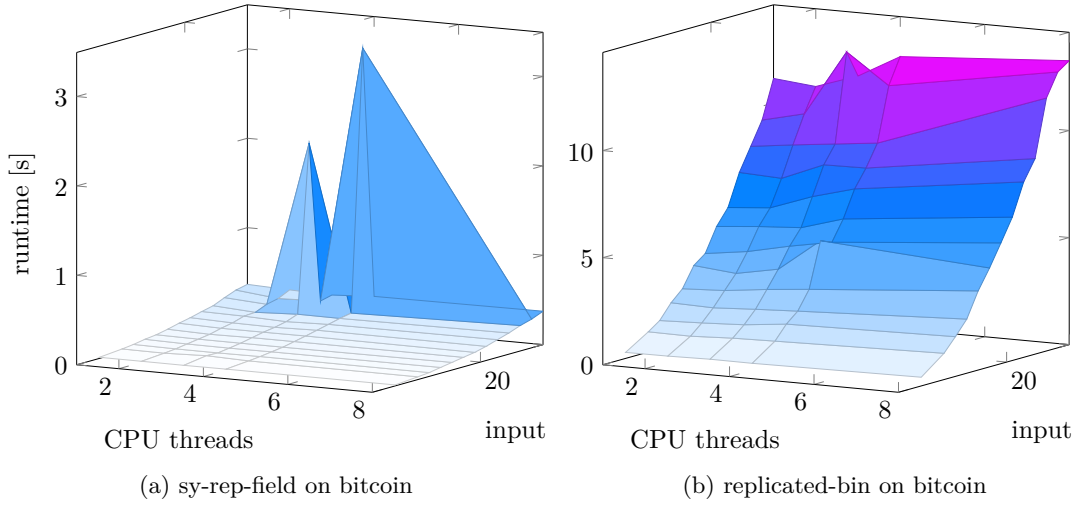


FIGURE A.46: Honest majority protocol finalists reaction to available CPU thread number change (Integer Multiplication)

SET INTERSECTION

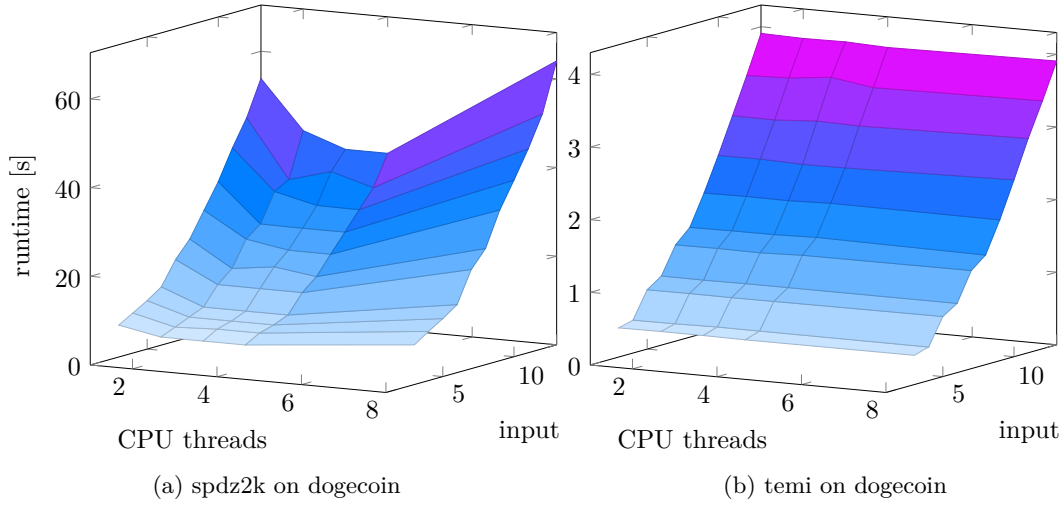


FIGURE A.47: Dishonest majority protocol finalists reaction to available CPU thread number change (Set Intersection)

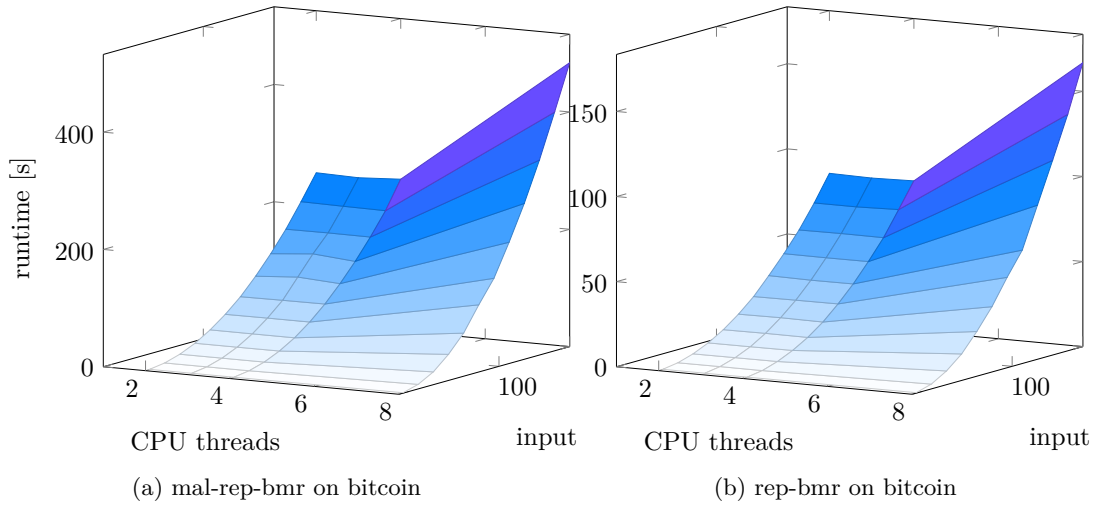


FIGURE A.48: Honest majority protocol finalists reaction to available CPU thread number change (Set Intersection)

A.2 TTP EXPERIMENT MEASUREMENT RESULTS

protocol	Element count [$\cdot 10^6$]	Runtime [s]
unsealed-mpc	10	25.7
unsealed-mpc	20	54.0
unsealed-mpc	30	76.4
unsealed-mpc	40	106.5
unsealed-mpc	50	134.9
unsealed-mpc	60	161.0
unsealed-mpc	70	198.3
unsealed-mpc	80	220.4
unsealed-mpc	90	243.4
unsealed-mpc	100	277.9

TABLE A.9: Experiment 31 scalable search conventional TTP experiment runtime results

A.3 LIST OF ACRONYMS

SMC	Secure Multiparty Computation, an area of cryptography
TTP	Trusted Third Party, knows all the secrets to be able to compute the result in a non SMC way
pos	plain Orchestrating Service, management tool for the testbed nodes
FRESCO	Framework for Efficient Secure Computation, generic SMC framework written in Java

BIBLIOGRAPHY

- [1] W. Du and M. J. Atallah, “Secure multi-party computation problems and their applications: A review and open problems”, in *Proceedings of the 2001 Workshop on New Security Paradigms*, ser. NSPW '01, New York, NY, USA: Association for Computing Machinery, 2001, 13–22, ISBN: 1581134576. DOI: 10.1145/508171.508174. [Online]. Available: <https://doi.org/10.1145/508171.508174>.
- [2] Y. Lindell, *Secure multiparty computation (mpc)*, Cryptology ePrint Archive, Report 2020/300, 2020. [Online]. Available: <https://ia.cr/2020/300>.
- [3] R. Talviste, *Secure multiparty computation (smc)*. [Online]. Available: <https://sunfish-platform-documentation.readthedocs.io/en/latest/smc.html#secure-multiparty-computation-smc>.
- [4] C. Zhao, S. Zhao, M. Zhao, Z. Chen, C.-Z. Gao, H. Li, and Y. an Tan, “Secure multi-party computation: Theory, practice and applications”, *Information Sciences*, vol. 476, pp. 357–372, 2019, ISSN: 0020-0255. DOI: <https://doi.org/10.1016/j.ins.2018.10.024>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025518308338>.
- [5] M. von Maltitz and G. Carle, “A performance and resource consumption assessment of secret sharing based secure multiparty computation”, *Data Privacy Management, Cryptocurrencies and Blockchain Technology*, 357–372, 2018, ISSN: 1611-3349. DOI: 10.1007/978-3-030-00305-0_25. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-00305-0_25.
- [6] K. W. Koch, “A benchmark framework for scale-mamba with applications to mpc-friendly prfs”, Dec. 2020. [Online]. Available: <https://diglib.tugraz.at/a-benchmark-framework-for-scale-mamba-with-applications-to-mpc-friendly-prfs-and-beyond-2020>.
- [7] M. Keller, “MP-SPDZ: A versatile framework for multi-party computation”, in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020. DOI: 10.1145/3372297.3417872. [Online]. Available: <https://doi.org/10.1145/3372297.3417872>.

- [8] S. Gallenmüller, D. Scholz, H. Stubbe, and G. Carle, “The pos framework: A methodology and toolchain for reproducible network experiments”, in *Proceedings of the 17th International Conference on Emerging Networking EXperiments and Technologies*, ser. CoNEXT ’21, New York, NY, USA: Association for Computing Machinery, 2021, 259–266, ISBN: 9781450390989. DOI: 10.1145/3485983.3494841. [Online]. Available: <https://doi.org/10.1145/3485983.3494841>.
- [9] J. I. Choi and K. R. B. Butler, “Secure multiparty computation and trusted hardware: Examining adoption challenges and opportunities”, *Security and Communication Networks*, vol. 2019, p. 1368905, 2019, ISSN: 1939-0114. DOI: 10.1155/2019/1368905. [Online]. Available: <https://doi.org/10.1155/2019/1368905>.
- [10] A. C. Yao, “Protocols for secure computations”, in *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, 1982, pp. 160–164. DOI: 10.1109/SFCS.1982.38.
- [11] D. Beaver, S. Micali, and P. Rogaway, *The round complexity of secure protocols (extended abstract)*, 1990.
- [12] S. Goldwasser, “Multi party computations: Past and present”, in *Proceedings of the Sixteenth Annual ACM Symposium on Principles of Distributed Computing*, ser. PODC ’97, New York, NY, USA: Association for Computing Machinery, 1997, 1–6, ISBN: 0897919521. DOI: 10.1145/259380.259405. [Online]. Available: <https://doi.org/10.1145/259380.259405>.
- [13] B. M. Leiner, V. G. Cerf, D. D. Clark, R. E. Kahn, L. Kleinrock, D. C. Lynch, J. Postel, L. G. Roberts, and S. Wolff, “A brief history of the internet”, *SIGCOMM Comput. Commun. Rev.*, vol. 39, no. 5, 22–31, 2009, ISSN: 0146-4833. DOI: 10.1145/1629607.1629613. [Online]. Available: <https://doi.org/10.1145/1629607.1629613>.
- [14] P. Bogetoft, D. L. Christensen, I. Damgård, M. Geisler, T. Jakobsen, M. Krøigaard, J. D. Nielsen, J. B. Nielsen, K. Nielsen, J. Pagter, M. Schwartzbach, and T. Toft, “Secure multiparty computation goes live”, in *Financial Cryptography and Data Security*, R. Dingledine and P. Golle, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 325–343, ISBN: 978-3-642-03549-4.
- [15] I. Damgård, M. Keller, E. Larraia, V. Pastro, P. Scholl, and N. P. Smart, “Practical covertly secure mpc for dishonest majority – or: Breaking the spdz limits”, in *Computer Security – ESORICS 2013*, J. Crampton, S. Jajodia, and K. Mayes, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 1–18, ISBN: 978-3-642-40203-6.
- [16] D. Evans, V. Kolesnikov, and M. Rosulek, “A pragmatic introduction to secure multi-party computation”, vol. 2, pp. 70–246, Jan. 2018. DOI: 10.1561/33000000019.

- [17] D. Rotaru, *Awesome-mpc*, <https://github.com/rdragos/awesome-mpc>, 2018.
- [18] M. von Maltitz, “Secure and privacy-preserving services based on secure multi-party computation”, Jul. 2019, ISBN: 978-3-937201-67-2. DOI: 10.2313/NET-2019-07-02. [Online]. Available: <https://www.net.in.tum.de/fileadmin/bibtex/publications/papers/dissertation-marcel-von-maltitz.pdf>.